

册01 · 逻辑与数

《数学基础》· 创世游戏系列丛书基础卷

从比特到命题：地基四件套

7 册 · 53 章 · 苏格拉底对话体自学教材

造物主（读者） $\rightleftharpoons$ 老谟（毒舌导师）—— 知识是推导出来的，不是背出来的

开源教材 · CC BY 4.0 · 在线版: lwq200.github.io/math-foundation

目录

1. 第 01 章 · 数——开关怎么表示量
2. 第 02 章 · 逻辑——开关怎么做判断
3. 第 03 章 · 集合——把东西圈起来
4. 第 04 章 · 函数——从一个到另一个
5. 第 05 章 · 证明方法入门——"显然"在数学里不算数
6. 第 06 章 · 证明实战——拿什么证明"素数有无穷多个"
7. 《逻辑与数》· 计算答案速查页

第 01 章 · 数——开关怎么表示量

本章从哪个问题出发：一台只会开和关的东西，凭什么装下一个数字？推导到哪个结论：二进制不是天选，是工程将就；负数靠补码重编码成加法；而**"一个结论凭什么对"——本章第一次把这个念头种进你的脑子：比特不会说话，全靠约定，而约定之下还有更硬的东西——证明。

核心认知：机器只会开和关。你看到的每一个数字，都是人跟机器约好的一套编码——比特本身没有任何意义，意义来自约定。**但约定不是"随便说说"：补码的每一条性质，都能被证明出来。**这一章，你除了学会编码，还要开始习惯一件事——**"显然"两个字，在数学里不算数，算数的是证明。**

引子

我把那台老电子表拆了。外壳撬开，露出里面一小块黑乎乎的芯片和几条走线，就再没有别的了。没有齿轮，没有刻度，没有一圈圈数字。这表上明明能显示 8:32、12:17，能计时，能倒着数——可我翻来翻去，只在里面找到一片不会说话的硅。

"就这么点东西，"我举着那块芯片冲老谟晃，"它凭什么记住'8 点 32 分'？它身上连个数字的影子都没有。"

老谟正给一台更旧的收音机拧螺丝，头也不抬："你这话说得好像数字是种长在机器里的东西。你自己想想——你把'3'这个念头，是怎么'放进'脑子里的？"

"我脑子……"我一愣，"我脑子里也没有一个写着'3'的小牌子啊。"

"那你凭什么觉得自己能记住'3'？"

这问题问得我胸口一堵。是啊，我凭什么？"3"不是颜色、不是气味、不是一块肌肉，我脑子里到底存了它的什么？

老谟终于抬头，把那颗拧下来的螺丝抛起来又接住："想明白'你的脑子怎么存 3'，你就想明白'这台表怎么存 8 点 32 分'了。二者是一回事——先把'3 长什么样'扔掉，只剩'3 和别的东西不一样'这一点。本章，就干这一件事。"

对话引擎

第 1 轮：凭什么非得是二进制

老谟："你一张口就'二进制'。可人天生爱十进制——十个指头数过来最顺手。机器是你造的，你凭什么不干脆让机器也用十进制？十个电压档位，对应 0 到 9，不行吗？"

造物主："因为……二进制简单？计算机算二进制快？"

老谟："快？二进制在数学上可一点都不比十进制'快'。你用二进制写个一百，得写七位（1100100），用十进制两位就完了。论省事，十进制才是省事的那个。你那'简单'两个字，是从哪儿偷来的？"

造物主："那.....难道是电路上，两种状态比十种状态容易做？"

老谟："这才沾到点边。你想想：一根导线上的电压，要区分'开'和'关'，得留多少安全区？要区分十档，又得留多少？"

造物主："两态只要一个门槛——高过它就是 1，低过它就是 0，中间一大段都是安全区。可要区分十个档，0 到 9 每个档之间得留极窄的缝.....稍有噪声、温度漂移、导线损耗，档位就串了。"

老谟："今天读出来是 7，明天变成 8。二极管能稳稳'开'和'关'，却很难'开到七分'。所以呢？"

造物主："所以是物理上不可靠，才退而求其次用两态？"

老谟："把'退而求其次'四个字给我划掉，换句话——**二进制不是数学的必然，是工程的便宜**。机器选二进制，是因为它容错、便宜、好做，不是因为它'天生正确'。这一点你要刻进骨头里：计算机里绝大多数'理所当然'，都是人跟物理讨价还价后的将就。"

造物主：（还是不服气，小声）"可我总觉得，二进制里藏着什么更'根本'的东西....."

老谟："你这话我听完就想给你鼓掌——可惜掌声是给'听懂了'的人的。藏着更根本的东西？你觉得它'根本'，恰恰是因为只有两种状态最稳、最不容易被物理噪声推翻——**稳定，就是那个'根本'**。稳定不是偶然的浪漫，是工程里最硬的事实。你先别急着给它披上哲学的外衣。"

知识锚点：为什么机器用二进制

【历史溯源】二进制不是新发明，是"老"约定。早在 **1679 年**，德国数学家**戈特弗里德·莱布尼茨（Gottfried Leibniz）**就在《二进制算术》里完整描述了二进制系统，甚至把它和中国的八卦（阴爻/阳爻）联系起来。但莱布尼茨当时只是把它当纯数学玩具——他绝没想过这东西会在三百年后统治全世界的计算机。**数学先想明白，工程后用它，中间隔了近三百年。**

【工程实践】工程上选二进制，图的不是"正确"，是**容错与成本**。一根导线上表示"开/关"（两态），只需一个电压门槛，门槛上下都是大片安全区，噪声、温漂、损耗都撼不动它；可要表示十档（0-9），十个电压区间就得紧贴在一起，每个档之间只留一条细缝，任何扰动都会让档位串号。**状态越少，越容错；越容错，越便宜、越好造。**这就是为什么芯片里清一色两态电路。

【概念深挖】**二进制（binary）**是一种只用 **0 和 1** 两个数字表示数的进制。人日常用十进制（逢十进一，0-9），机器用二进制（逢二进一，只有 0 和 1）。例如十进制的 **5**，写成二进制是 **101**（表示 $1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 4 + 0 + 1 = 5$ ）。**一个二进制位叫一个比特（bit）**，是信息的最小单位——它只回答一个问题：开，还是关。

第 2 轮：5 在二进制里到底长什么样

老谟："先别想着算。你告诉我，二进制的世界只有 0 和 1。你从 0 开始，一个一个往下写，写到 5，中间每个数怎么跳？"

造物主："从 0 开始.....0、1，然后是 2。可 2 没有第二个数字用了，只能进位——10。然后 3 是 11，4 是 100，5 是 101？"

老谟："你写出来了。停在这儿，别往下写。盯着这排数，告诉我：最低位（最右边那一位）有什么规律？"

```
0 → 0
1 → 1
2 → 10
3 → 11
4 → 100
5 → 101
```

造物主：".....最低位在 0、1、0、1 地跳。奇数结尾是 1，偶数结尾是 0！"

老谟："对。二进制的最低一位，就是'这个数是奇数还是偶数'。那我现在问你：5 是奇数，它的最低位一定是 1——你把它抹掉，剩下的是谁？"

造物主："抹掉最低位.....101 去掉 1 剩 10，就是 2？可 5 抹掉一位怎么是 2？"

老谟："你反过来想：二进制每往左一位，值是翻倍的（1、2、4、8...）。所以'5 抹掉最低位'，等于'5 除以 2 丢掉了那点零头'。5 除 2 商 2 余 1，商 2 就是 10，余 1 就是最低位。你顺着这个'除 2、取余'往下走，会得到什么？"

造物主：".....5 除 2 余 1，商 2；2 除 2 余 0，商 1；1 除 2 余 1，商 0。余数从下往上读：1、0、1，就是 101！我这是把'最低位=奇偶'这条路倒着走回去了。"

老谟："对。你不是背了个'除二取余'的口诀，你是从'最低位是奇偶'这一步一步把它踩出来的。这口诀是**你推导出来的**，不是天上掉的。"

造物主：".....可我还有个不安：我说'最低位是奇偶'、'往左一位翻倍'，这俩是我数出来的规律。可它们**凭什么一直对**？我数到 5 对，数到 100 还对吗？"

老谟："你问出了一个比'怎么换算'深得多的问题——**你凭什么相信一条规律对所有的数都成立**？这个问题，今天先记下，留到第 05 章我们专门给它一把'数学归纳法'的钥匙。现在你先把'怎么算'练熟——但记住你这个不安，它是学好数学的第一步。"

知识锚点：整数十进制 → 二进制

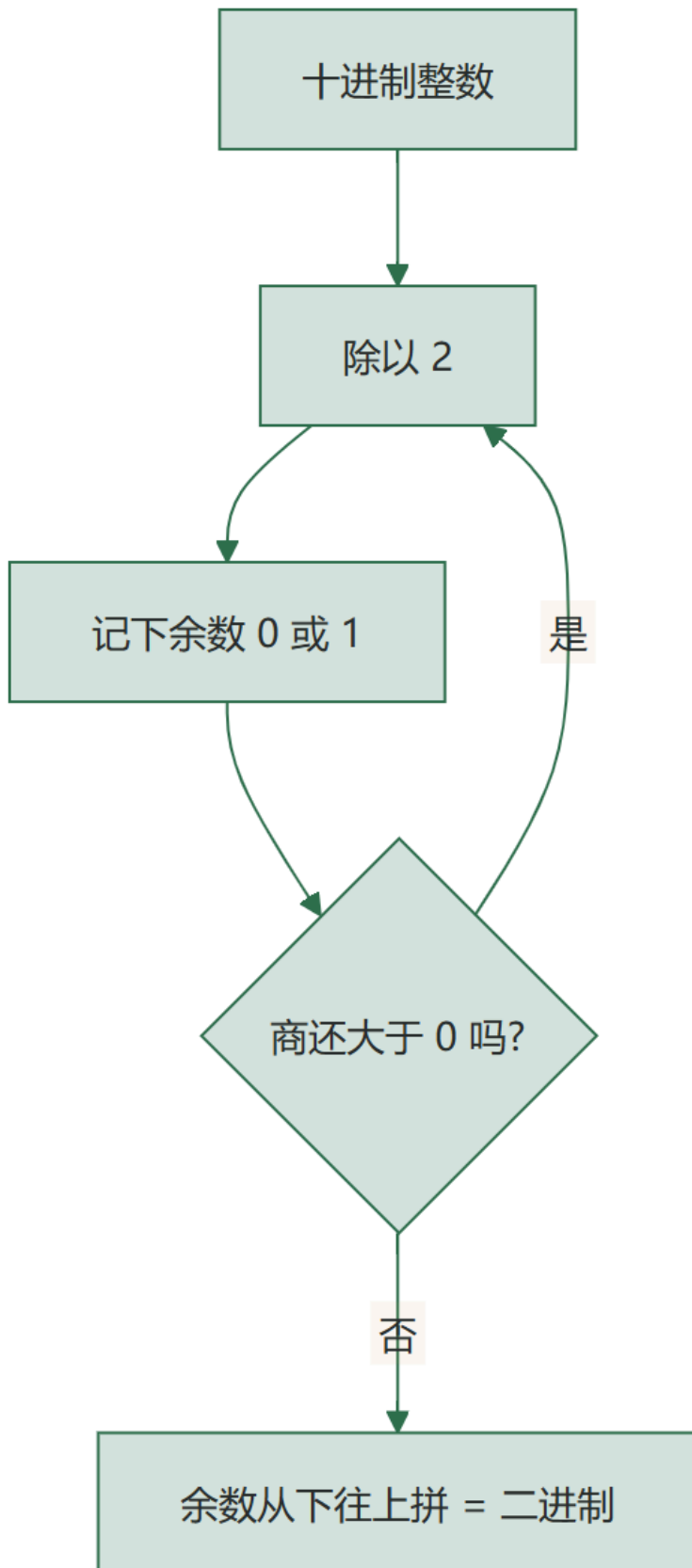
【历史溯源】进制转换的方法不是凭空来的，它和**位值制计数法 (place-value)** 一脉相承。十进制的"逢十进一"、二进制的"逢二进一"，本质都是**每一位代表一个固定的权值**：从右往左，十进制是 1、10、100...，二进制是 1、2、4、8...。这个"权值"思想源自古代位值计数系统（古巴比伦六十进制、古印度十进制），几千年后才被系统化成今天的进制换算规则。

【工程实践】工程师为什么从不用"除二取余"以外的方法？因为**除二取余**只靠重复的除法和取余就能完成——极其机械、可循环，特别适合交给计算机自动执行（一个 `while` 循环就能写完）。进制转换在工程里不是"心算游戏"，而是程序里一行代码的事。

【概念深挖】整数转二进制用**除二取余**：不断除以 2，记下余数（0 或 1），再从下往上拼。例：

```
5 ÷ 2 = 2 余 1 → 最低位 1
2 ÷ 2 = 1 余 0 →      0
1 ÷ 2 = 0 余 1 → 最高位 1
从下往上读：101
```

最低位是奇偶，往左每一位翻倍——抓住这两条，整数的二进制就是一张你随时能亲手画出来的表。



读图说明：这是一台“剥洋葱机”。每次从数的腰部剥下一层（最低位），剥下来的余数就是这一层的真相，剩下的商接着剥；一直剥到商变成 0，把剥下来的所有皮从里往外拼起来，就是二进制。机器只要会“除以 2、记余数、判断商是否为 0”三件事，就能无穷无尽地把整数转成二进制。

第 3 轮：负数，才是真正的坎

老谟："二进制装正数，你算是摸到门了。现在来硬的：开关只有 0 和 1，你怎么让它表示 '-5'？"

造物主："这还不简单？留一位当符号位，比如最左边是 1 就代表负号。10000101 表示 -5，00000101 表示 +5。"

老谟："听起来很讲道理。"他顿了顿，"那我问你， $5 + (-5)$ 等于多少？"

造物主："等于 0 啊。"

老谟："用你的符号位法，让机器算一遍。它要把两个字节喂进加法电路：00000101 加 10000101。"

造物主："我算一下.....00000101 加 10000101。个位 $1+1=2$ ，进 1.....算完是 10001010，也就是 -10。等一下，5 加 -5 怎么会是 -10？"

老谟："你发现矛盾了。说说，你想怎么绕过去？你的符号位法，问题出在哪？"

造物主："我给它'贴了个标签'——最左边那位我单独当符号用。可贴了标签的数，和正数根本不是同一套逻辑。机器做加法时，得先看懂标签、判断谁正谁负、该加还是该减——一套电路被我拆成了两套。"

老谟："那怎么办？"

造物主："与其给负数贴标签，不如**把负数挪到正数圈里**，让加法电路一视同仁地加？"

老谟："挪？用一个固定长度，比如 8 位，能表示 0 到 255。你想怎么把 -5 挪进去？"

造物主："如果我把 -5 定义成 256 减 5，也就是 251？"

老谟："251 在 8 位里写成什么？"

造物主："11111011。"

老谟："那 $5 + 251$ 等于多少？"

造物主："256。可 8 位装不下 256，最高位溢出了，剩下的.....是 0。"

老谟："成了。你亲手把 -5 编码成 251（也就是 $256-5$ ），正数 5 就是 5，两者相加溢出后刚好归 0——**减法，被你变成加法了**。这套把戏，叫补码（two's complement）。你刚才那一下'256 减它'，就是补码的灵魂。"

造物主："所以我不是发明了负数，我是发明了'怎么把负数藏进正数里'的编码。"

老谟："这就不赖了。可你还欠一件事——你说'取反加一'能造补码，你凭什么信？'溢出后归零'这个说法，能不能**证明**？我们下一轮，就给补码立个证明。"

知识锚点：补码 (two's complement)

【历史溯源】符号位法的坑，历史上整整一代机器都摔过。最早的商用存储程序计算机 **EDSAC (1949, 剑桥, Maurice Wilkes 团队)** 用的就是"符号 + 幅度" (sign-magnitude) 式的表示，结果 ± 0 并存、加减要判符号，电路处处别扭。真正的转折是 **1964 年 IBM System/360**：这台机器正式把**补码 (two's complement)** 定为整数的标准表示，从此加减统一成一套逻辑，符号位不再特殊。有趣的是，补码的思想在 1945 年冯·诺依曼的《EDVAC 报告初稿》里就已现雏形。

【工程实践】补码替硬件工程师**省了一块芯片**。如果按符号位法，ALU（算术逻辑单元）得同时备着"加法器"和"减法器"两套电路，再加一个判符号的开关在两者间切换。而补码把 $-b$ 编码成 $(2^n - b)$ 后， $a - b$ 就变成了 $a + (2^n - b)$ ——**减法器整个消失，机器里只剩一个加法器**。

【概念深挖】补码 (two's complement) 是什么：

在 n 位二进制里，模是 2^n 。一个数 x 的 (n 位) 补码，**正数就是自己，负数 $-x$ 定义为 $2^n - x$** 。于是 $a - b = a + (2^n - b)$ ，只要结果再取低 n 位（溢出自然丢弃），就等价于减法。符号位不再是"标签"，而是补码最高位天然带出来的：最高位为 1 的数，按无符号读本来就大于 2^{n-1} ，正好对应负数区间。

✗ 符号位法（原码）： $-5 \rightarrow 10000101$ (加 $+5 \rightarrow 10001010 = -10$ ，错)
 ✓ 补码法： $-5 \rightarrow 11111011$ (加 $+5=00000101 \rightarrow 1\ 00000000$ ，溢出剩 $00000000 = 0$ ，对)

踩坑注释：原码（符号位法）还有个更尴尬的毛病——它有两个 0（00000000 和 10000000 都表示零），白白浪费一个编码，还让"比较是否为零"要多判一次。补码只有一种 0，且负数区间比正数多表示一个（如 8 位补码能表示 -128 到 127），这都是编码约定带来的红利。

第 4 轮：亲手造一次——从"看懂"到"能造"

老谟："补码你懂了，可'懂'和'能造'隔着一道河。现在把河蹚了——给你一个整数，你给我写出它在 8 位补码下的样子。来，-7。"

造物主："-7 的 8 位补码.....按公式，256 减 7 得 249，249 的二进制是 11111001。我算出来了。"

老谟："算得对，但你在'背公式'。真造补码的人，从不用'256 减它'——那太慢，硬件也没法直接做减法。硬件用的是另一招，你试着找找。"

造物主："另一招？我只知道 $-x = 256 - x$哦，等一下。补码的核心是'溢出归零'，那能不能从 +7 出发，反着把它翻过去？"

老谟："说下去。"

造物主：" $+7$ 是 00000111。要让它和某个数相加得 0.....如果我把每一位都翻转，00000111 变 11111000，再.....加 1？11111000 加 1 等于 11111001！这正好是刚才那个 249！"

老谟："成了。你不是背出来的，你是造出来的。**取反加一**（NOT 一下，再加 1）——这就是硬件造补码的真实动作。它只需要几个取反门加一个加法器，连减法都不用做，就把 -7 从 $+7$ 翻出来了。"

造物主："可我还是有点虚——'取反加一'和'256 减它'，真的永远是一回事吗？会不会哪天碰上一个数，两条路算出不一样的结果？"

老谟："你又问到一个'凭什么'了。这正该证明——下一轮，我们把这个'两条路永远一样'，用一个证明钉死。"

知识锚点：补码的"取反加一"——数学 vs 工程

【概念深挖】数学上， $-x$ 定义为 $2^n - x$ ，方便你理解"溢出后归零"。但硬件要"省"，就换了个**等价动作：按位取反再加 1 (NOT + 1)**。对正数 x 取反得 $\sim x$ ， $\sim x + 1$ 恰好等于 $2^n - x$ （在 n 位内），所以两者是同一个补码，只是工程上更好造。

【实现思考】把上面手动造补码翻译成代码，就是"看懂 → 能造"的落地：

```
def to_twos_complement(x, n):
    mask = (1 << n) - 1
    if x >= 0:
        return x & mask          # 正数就是自己
    return (~abs(x) + 1) & mask   # 负数：对 |x| 取反再加 1

print(bin(to_twos_complement(-7, 8)))    # 0b11111001
print((to_twos_complement(-7, 8) + 7) & 0xFF) # 0
```

踩坑：Python 里 $\sim x$ 对负数的语义是 $\sim(x) = -(x+1)$ （不是"把每一位翻转"），所以对负数直接 $\sim x + 1$ 会出错——必须先取绝对值 $\text{abs}(x)$ ，对正数 $|x|$ 做"取反加一"，才符合教材"对 $+7$ 取反加一得到 -7 "的教学逻辑。上面代码实测：`to_twos_complement(-7, 8)` 输出 `0b11111001`，`(...+7) & 0xFF` 输出 `0`，与注释一致。（更简洁的写法是直接 `return x & mask`，Python 负数 `-7 & 0xFF` 自动给出补码 249；这里用 $\sim \text{abs}(x) + 1$ 是为了贴合"取反加一"的推导。）

为什么这么造： $\sim x + 1$ 只需几个 NOT 门（取反）接一个加法器，不需要减法电路。**换条路会怎样：**若按字面"256- x "做，硬件得先造减法器，又绕回原码的老路。同一个数学对象，为方便工程改个等价动作，这就是"约定"在硬件层面的又一次落子。

第 5 轮：会造还得会还原——补码的自治

老谟："造补码你会了。现在反过来——给你一串补码 `11111001`，你怎么知道它代表 -7，而不是 249？"

造物主："8 位里最高位是 1，所以它是负数。可它到底是 -7 还是别的？我.....得把它'还原'回原数？"

老谟："怎么还原？你刚才的取反加一是'造'，那'还原'该是什么？"

造物主：".....再造一遍？取反加一，再取反加一，不就回来了？`11111001` 取反得 `00000110`，加 1 得 `00000111`，是 +7！所以它代表 -7。"

老谟："对。补码对自己再'取反加一'一次，就能还原回原数的相反数。这正好说明补码是'自治'的——同一个操作，造是它，还原也是它。你现在既会造、又会还原了。可'自治'这两个字，跟'证明'有什么关系？"

造物主：".....关系？'自治'是说它转一圈能回来，'证明'是说.....它凭什么能回来？"

老谟："你这句话，就是本章埋的那颗种子：**'凭什么'三个字，就是证明的起点。**你已经会背补码的公式了，可你还没'证明'过补码。下一节严格化走廊，我们就把'取反加一自治'这个'凭什么'，写成一个像模像样的证明——那也是你这一生写的第一个证明的雏形。"

知识锚点：补码的自治性——造与还原是同一个动作

【概念深挖】补码最漂亮的性质是**自治**：对任意补码再"取反加一"一次，就得到它的相反数。因为 $(\sim x + 1)$ 再取反加一 $= x$ （在 n 位内）。所以 `to_twos_complement(-7, 8)` 得 `11111001`，而 `to_twos_complement(11111001 按无符号读 = 249, 8)` 的结果在 8 位里就是 -7 的补码——同一个函数，造负数、还原负数，用的是同一个动作。这就是为什么硬件只需要"取反加一"这一个电路，既管编码又管解码。

【工程实践】若给你一个 16 位补码的电路图，要把"取反加一"的逻辑放进去，你会把它放在加法器的哪一侧？——答案：取反是"一堆 NOT 门"放在加数输入端，加 1 是给加法器最低位进一个"1"。你会发现，造和还原共用的就是这一小段：NOT 门 + 进位。这也解释了为什么补码能在硅上省下整整一个减法器。

第 6 轮：落点——比特不认识任何数，但证明认识

老谟："绕了一圈，回到你修表时的那个问题。你现在告诉我：那块表里，'8 点 32 分'到底被存成了什么？"

造物主："被存成一串比特。8 是 1000，32 是 100000.....被存成若干个 0 和 1。"

老谟："那这串比特，自己知道它是 8 点 32 分吗？"

造物主："不知道。它连'几点'都不懂。同一串比特，按二进制读是个数，按别的方式读可能是别的....."

老谟："说到底——"

造物主："说到底，机器只知道开和关。我写的每一个数字，全是我给比特串贴的标签。比特本身没有任何意义，意义来自约定。正数是一套约定，补码是把负数也藏进同一套约定里，让加法不必挑肥拣瘦。"

老谟："可你今天比样书里多学会了一样东西——你不再只是'知道'约定，你还开始问'凭什么'了。你问'最低位是奇偶凭什么一直对'，问'取反加一和 256 减它凭什么一样'——这两个'凭什么'，就是证明的种子。你这一生要写的证明，全是从这种不安开始的。"

造物主：".....所以比特不会说话，约定让它开口；而约定自己，还得靠证明来给它撑腰？"

老谟："这话记好了。比特是哑的，约定让它说话；约定是松的，证明让它站住。下一章，我们让比特从'会报数'升级到'会判断'——而'判断'里，藏着你证明之路的第一块正式台阶。"

严格化走廊

位置说明：对话负责"直觉"——比特为什么这样编码。本走廊负责"严谨"——把"取反加一自治"这个"凭什么"落成可复写的证明。这是全系列第一册的第一条严格化走廊，也是读者接触"证明"的第一次正式亮相：**每一步写清楚用了哪条定义/哪条代数规则，绝不说"显然"**。

证明类型标注：本章的三条证明都属于**"直接计算/代数"型**——直接由定义展开、按代数规则变形（没用到反证、归纳、构造那些"招数"）。完整的证明方法分类（直接/反证/归纳/构造）会在第 05 章系统展开，这里先给你一个概念：**证明也是分"招式"的**，看懂"它用的是哪一招"，比看懂"它证了什么"更进一步。

定义：n 位补码 (n-bit two's complement)

设 $n \geq 1$ 。在 n 位二进制中，模 $M = 2^n$ 。**整数 x 的 n 位补码**记作 $\text{comp}(x)$ ，定义为：

- 若 $0 \leq x \leq 2^{n-1} - 1$ （非负）： $\text{comp}(x) = x$ 的 n 位二进制表示。
- 若 $-2^{n-1} \leq x \leq -1$ （负数）： $\text{comp}(x) = x + M = x + 2^n$ 的 n 位二进制表示（取低 n 位）。

等价记法：对负数 $-a$ ($a \geq 1$)， $\text{comp}(-a) = 2^n - a$ 。

n 位截断：任何整数 t 的 n 位表示，都取 $t \bmod 2^n$ （即丢弃高于 $n-1$ 位的部分）。

定义域注记： $\text{comp}(x)$ 的定义只显式覆盖合法补码区间 $[-2^{n-1}, 2^{n-1}-1]$ ；但定理 1 里会出现 $\text{comp}((x+y) \bmod 2^n)$ ，当 $(x+y) \bmod 2^n$ 落在溢出区 $[2^{n-1}, 2^n)$ （比如 $127+1$ 的 10000000 ）时， comp 定义没直接写。**这条就按“边界”一节处理：**溢出的 mod 值，按补码语义解读（ 10000000 读作 -128 ）。换句话说—— **comp 的“本义”是给合法区间的数编码；超出区间的结果，靠“模 2^n 再按补码读”这条约定兜住。**

定理 1（补码加法闭合）：在 n 位内，补码加法等价于模 2^n 加法

对任意 n 位补码表示的整数 x, y ： $(\text{comp}(x) + \text{comp}(y)) \bmod 2^n = \text{comp}((x + y) \bmod 2^n)$ 。

直觉：补码把“减法”藏进“加法”，靠的就是“溢出即取模”。这个定理说：**在 n 位里做补码加法，跟“先正常相加、再取模”是一回事。** $5 + (-5) = 0$ 的验证，正是它的特例。

证明：

- 第 1 步：由补码定义， $\text{comp}(x)$ 表示的数 $\equiv x \pmod{2^n}$ ， $\text{comp}(y)$ 表示的数 $\equiv y \pmod{2^n}$ 。（因为非负数就是 x ；负数是 $x + 2^n \equiv x \pmod{2^n}$ 。）
- 第 2 步：因此 $\text{comp}(x) + \text{comp}(y) \equiv x + y \pmod{2^n}$ 。
- 第 3 步：取 n 位（即 $\bmod 2^n$ ），得 $(\text{comp}(x) + \text{comp}(y)) \bmod 2^n = (x + y) \bmod 2^n$ 。
- 第 4 步：而 $\text{comp}((x+y) \bmod 2^n)$ 正是 $(x+y) \bmod 2^n$ 的 n 位表示（定义）。■

读法：整个证明只用了“补码 $\equiv$ 原数 $\pmod{2^n}$ ”这一件事，加上模算术的同余相加。**先看懂‘补码和原数同余’，再看懂‘同余相加保持同余’，证明就通了。**这就是证明的骨架：把“看起来玄”的东西，翻译成“已经懂的规则”。

定理 2（取反加一自洽性）： $\sim x + 1 \equiv 2^n - x \pmod{2^n}$

设 x 是 $0 \leq x < 2^n$ 的整数， $\sim x$ 表示对 x 的 n 位每一位取反（NOT）。

证明：

- 第 1 步： x 的 n 位二进制中，每一位非 0 即 1，所以 x 的每一位与 $\sim x$ 的对应位相加恒为 1。故 $x + \sim x = 111\dots 1 = 2^n - 1$ （ n 个 1）。
- 第 2 步：两边同时加 1： $x + (\sim x + 1) = 2^n - 1 + 1 = 2^n$ 。
- 第 3 步：两边同减 x ： $\sim x + 1 = 2^n - x$ 。■

读法：这可能是你见过的第一个“真正的”证明。它只用了两步：① 取反的意义（每一位互补，所以加起来全是 1，即 $2^n - 1$ ）；② 代数移项。**没有一步说“显然”**。这告诉我们：证明不是魔法，是把“你懂的东西”按规则连起来，直到结论自己冒出来。

定理 3（补码还原自治）：取反加一再取反加一，回到原数

对 $0 \leq x < 2^n$ ，令 $f(x) = (\sim x + 1) \bmod 2^n$ （取反加一后取 n 位）。则 $f(f(x)) = x$ 。

证明：

- 第 1 步：由定理 2， $f(x) \equiv 2^n - x \pmod{2^n}$ ，即 $f(x) \equiv -x \pmod{2^n}$ 。
- 第 2 步：再取一次： $f(f(x)) \equiv -f(x) \equiv -(-x) = x \pmod{2^n}$ 。
- 第 3 步： $f(f(x))$ 和 x 都是 $[0, 2^n)$ 内的数，且同余，故相等。■

读法：定理 3 就是对话第 5 轮“造也是它、还原也是它”的严格版。它把“转一圈回来”这个直觉，翻译成了“取两次相反数（模 2^n ）等于自己”。**注意第 3 步那一下“同余且在范围内所以相等”**——这是模算术里常用的收尾，先记住这个套路。

反例/边界：补码的四个边界

- **最高位是 1 一定是负数吗？** 在补码语义下是（因为 $2^{n-1} \leq \text{值} < 2^n$ 对应负数区间）；但同一串比特按无符号读就是正数。“**是正还是负**”不是比特自带的，是你选的解读语义。例：`11111001` 按无符号读是 249，按补码读是 -7。
- **溢出会静默错：** n 位补码能表示的范围是 $[-2^{n-1}, 2^{n-1}-1]$ 。超出这个范围，加法结果会“绕回”——`127 + 1` 在 8 位里得 `10000000`，按补码读是 -128，不是 128。**溢出不报错，是补码的代价**（这也是为什么工程里要检查溢出）。
- **负数比正数多一个：** 8 位补码能表示 -128 到 127，负数的绝对值比正数多 1（因为 -128 的补码 `10000000` 没有对应的正数）。这不是 bug，是“模 2^n 恰好有个半开半闭区间”的必然。
- **补码的加法对“负数+负数”同样成立：** 例 8 位里 $(-5)+(-3)$ ：`11111011 + 11111101 = 11111000`，溢出丢弃得 `11111000` = -8 ✓（定理 1 保证，不靠猜）。

知识点总结

知识点（本章推出的核心概念）

- **二进制不是数学的必然，是工程的便宜**：两态比十态容错、便宜、好做，机器选它是因为物理上两态更可靠。
- **整数除二取余**：不断除以 2、记余数、从下往上拼；最低位是奇偶，往左每一位翻倍。
- **补码 (two's complement)**：把负数重编码进正数区间 ($-x = 2^n - x$)，让减法变成加法，省掉减法器；工程上等价动作为“取反加一”。
- **编码**：比特本身无意义，正数、负数都是人和机器约好的标签。
- **“凭什么”= 证明的种子**：你问“最低位是奇偶凭什么一直对”“取反加一凭什么等于 256 减它”——这些不安就是证明的起点。**证明不是魔法，是把“你懂的东西”按规则连起来。**

历史结论（本章涉及的史料）

- **二进制**：莱布尼茨 1679 年完整描述二进制系统，当时只是数学玩具，三百年后才统治计算机。
- **补码**：EDSAC (1949) 用符号位法， ± 0 并存、加减判符号；IBM System/360 (1964) 定补码为整数标准，沿用至今。

工程实践结论（本章知识在真实工程里怎么用）

- **ALU 只需一个加法器**：因为补码把减法重写为加法，CPU 少一大块减法逻辑。
- **补码取反加一**：硬件只靠 NOT 门 + 加法器，就能既造负数又还原负数。
- **溢出要检查**：8 位补码 $127+1$ 会绕回 -128 ，不报错——工程里必须显式检测溢出。

计算训练场

位置说明：对话负责“为什么”，这里负责“怎么算”。每题动笔 10 分钟以上，先自己写再对答案。

题 1（中）：批量进制换算 + 补码加减验证

题干：老谟给了造物主一串十进制数：13、42、100、255。请：(a) 把它们全部转成二进制（写出除二取余过程）。(b) 把 13 和 42 用 8 位补码相加，验证结果是否等于 55 的 8 位补码。(c) 讨论：若用 8 位无符号数， $100 + 200$ 会怎样？用 8 位补码， $100 + 200$ 又会怎样？两个结果说明什么？

完整分步解答：

(a) 进制换算：

- 13: $13 \div 2 = 6$ 余 1; $6 \div 2 = 3$ 余 0; $3 \div 2 = 1$ 余 1; $1 \div 2 = 0$ 余 1。从下往上: **1101**。
- 42: $42 \div 2 = 21$ 余 0; $21 \div 2 = 10$ 余 1; $10 \div 2 = 5$ 余 0; $5 \div 2 = 2$ 余 1; $2 \div 2 = 1$ 余 0; $1 \div 2 = 0$ 余 1。**101010**。
- 100: $100 \div 2 = 50$ 余 0; $50 \div 2 = 25$ 余 0; $25 \div 2 = 12$ 余 1; $12 \div 2 = 6$ 余 0; $6 \div 2 = 3$ 余 0; $3 \div 2 = 1$ 余 1; $1 \div 2 = 0$ 余 1。**1100100**。
- 255: $255 \div 2 = 127$ 余 1; $127 \div 2 = 63$ 余 1; $63 \div 2 = 31$ 余 1; $31 \div 2 = 15$ 余 1; $15 \div 2 = 7$ 余 1; $7 \div 2 = 3$ 余 1; $3 \div 2 = 1$ 余 1; $1 \div 2 = 0$ 余 1。**11111111**。

(b) 补码加法:

- 13 的 8 位补码 = 00001101; 42 的 8 位补码 = 00101010。
- 相加: $00001101 + 00101010$ (从位 0 起逐位, 进位向右传递):

```

      位7 6 5 4 3 2 1 0
13   = 0000 1101
42   = 0010 1010
-----
位0: 1+0 = 1, 进 0    → 1
位1: 0+1 = 1, 进 0    → 1
位2: 1+0 = 1, 进 0    → 1
位3: 1+1 = 2, 写 0 进 1 → 0
位4: 0+0+1 = 1, 进 0   → 1
位5: 0+1 = 1, 进 0     → 1
位6: 0+0 = 0           → 0
位7: 0+0 = 0           → 0
结果(位7..位0): 0011 0111

```

- 得 **00110111** = $32+16+4+2+1 = 55$ ✓
- 55 的 8 位补码 = 00110111 ✓ 一致。
- **关键在进位传递**: 位 3 的 $1+1$ 产生进位 1, 必须带到位 4 ($0+0+1=1$); 位 4 没有新进位, 后面位 5 的 $0+1$ 就是 1。逐位加法最易错的就是漏掉这个跨位的进位。

(b 备选·直接用补码验证溢出归零): 也可验证 $13 + (-13)$ 在 8 位里归零: -13 的补码 = $2^8 - 13 = 243 = 11110011$; $00001101 + 11110011 = 1\ 00000000$, 取低 8 位 = $00000000 = 0$ ✓ (补码加法的灵魂)。

(c) 溢出:

- 无符号 8 位: $100+200=300$, $300 = 256+44$, 取低 8 位 = **44** ($300 \bmod 256$)。溢出丢失了 256。

- 补码 8 位：范围 $[-128, 127]$ 。100+200=300 超出， $300 \bmod 256 = 44$ ，但 44 在正数范围内，按补码读还是 44——**注意：这里恰好没绕回负数，但结果已不是 300 了。**
- 结论：**8 位不管无符号还是补码，都装不下 300，结果都被“模 256”截断了。** 溢出不报错，这是定长编码的固有代价——工程里要检查结果是否超出范围。

答：(a) $13=1101$ 、 $42=101010$ 、 $100=1100100$ 、 $255=11111111$ ；(b) $00110111=55 \checkmark$ ；(c) 都变成 44 ($\bmod 256$ 截断)，说明定长编码溢出会静默错。

常见错误：① 除二取余把余数从上往下读（必须先算出来的是最低位）；② 补码加法忘了取低 8 位（把进位 1 也当结果）；③ 以为“补码负数相加会出负数、正数相加会出正数”——**溢出时不一定，必须检查范围。**

题 2（中→难）：取反加一的验证与“还原”循环

题干：给定 +7 的 8 位补码 00000111。(a) 用“取反加一”求 -7 的补码。(b) 对 (a) 的结果再“取反加一”，验证回到 00000111（还原自洽）。(c) 用“256 减”的方法独立求 -7 的补码，与 (a) 对照。(d) 证明：取反加一与“256 减”在 8 位内永远等价（用严格化走廊定理 2 的话说： $\sim x + 1 \equiv 2^8 - x \bmod 2^8$ ）。

完整分步解答：

(a) 取反加一：

- $+7 = 00000111$ 。取反：11111000。加 1： $11111000 + 1 = \mathbf{11111001}$ 。
- 验证： $11111001 + 00000111 = 1\ 00000000$ ，取低 8 位 = $00000000 = 0 \checkmark$ （5 加 -5 同款）。

(b) 还原：

- 对 11111001 取反：00000110。加 1： $00000110 + 1 = \mathbf{00000111} \checkmark$ 回到原数。

(c) 256 减：

- -7 的补码 $= 2^8 - 7 = 256 - 7 = 249$ 。
- 249 的 8 位二进制： $249 = 128 + 64 + 32 + 16 + 8 + 1 = \mathbf{11111001} \checkmark$ 与 (a) 一致。

(d) 等价性（写证明）：

- 由定理 2： $x + \sim x = 2^8 - 1$ （每一位互补）。
- 两边加 1： $x + (\sim x + 1) = 2^8$ 。
- 移项： $\sim x + 1 = 2^8 - x$ 。■ 即“取反加一”恒等于“256 减 x”（模 2^8 意义下）。■

答：(a) 11111001；(b) 回到 00000111；(c) $249 = 11111001$ 一致；(d) 证明见上。

常见错误：① 取反时把符号位也翻了——**取反是“每一位”都翻，没有例外位**（这正是补码和符号位法的本质区别）；② 加 1 时忘了最低位的进位会一路传（如 $11111000+1$ 时，最低位 $0+1=1$ 不进位，但如果最低位是 1 就会连续进位）；③ 混淆“249 的二进制”和“-7 的补码”——它们恰好是同一串比特，但**同一串比特读成无符号是 249，读成补码是 -7**。

题 3（难）：二进制加法器——把“进位”写成人话

题干：设计一个 4 位二进制加法器的手算流程。老谟给了造物主两个 4 位二进制数： $A = 1011$ (11)， $B = 0110$ (6)。要求：(a) 从最低位开始逐位相加，完整写出每一步的“本位和 + 进位”。(b) 验证结果是否等于 $11 + 6 = 17$ 的 4 位表示。(c) 讨论：17 在 4 位里能表示吗？如果给的是补码语义， $1011 + 0110$ 又等于多少？两个结果说明了什么？

完整分步解答：

(a) 逐位加法（从右往左，第 0 位起，进位逐位传递）：

```

1011    (11)
+ 0110   (6)
-----
第0位: 1+0 = 1, 进位 0      → 本位 1
第1位: 1+1 = 2, 写 0 进 1    → 本位 0, 进位 1
第2位: 0+1+1 = 2, 写 0 进 1  → 本位 0, 进位 1
第3位: 1+0+1 = 2, 写 0 进 1  → 本位 0, 进位 1 (最高位进位丢弃)
4 位结果: 0001

```

(b) 验证： $11 + 6 = 17$ 。17 的 4 位表示： $17 = 16 + 1$ ，而 $16 = 2^4$ 超出 4 位，所以 $17 \bmod 16 = 1 = 0001$ ✓ 与逐位加法一致。

(c) 讨论：

- 17 在 4 位无符号里**不能表示**（最大 15），溢出丢掉了最高位。
- 若按补码语义： $1011 = -5$ （最高位 1）， $0110 = 6$ ， $-5+6 = 1$ ，4 位结果 $0001 = 1$ ✓ 同样成立。
- 结论：**同一串比特，无符号读是“17 溢出成 1”，补码读是“-5+6=1”——解读不同，但硬件做的加法一模一样。** 这就是“编码是约定”的活例子。

答：(a) 逐位 1,0,0,0，结果 0001；(b) $17 \bmod 16 = 1$ ✓；(c) 无符号读溢出、补码读合法，同一电路两种语义。

常见错误：① 忘记进位要在第 2 位之后逐位传递（0+1 也要加进位）；② 把丢弃的进位 1 当结果的一部分（4 位加法器只有 4 位输出）；③ 混用无符号/补码语义——同一比特串，读法不同答案不同，但加法电路是同一个。

向导便签

这一章咱们干了什么：从拆开的那台电子表，一路逼到补码。你亲手重走了"怎么用开关装下一个数字"的路程——二进制、进制转换、负数补码，一路由最底层构筑。但比样书多的是：你开始问"凭什么"了。

钉死一句话：**比特本身是个哑巴，编码才是它开口说话的语法；而约定是松的，证明让它站住。** 同一个 101，按无符号读是 5，按补码读是 -3，按符号位法（原码）读是 -1——谁对？都"对"，因为"对"的意思是"照同一张表在读"。

记住：二进制不是天选，是工程将就；补码不是魔法，是把减法藏进加法的编码巧思。而**"取反加一和 256 减它凭什么一样"**——这个"凭什么"，是你证明之路的第一块台阶。

造物主手记

我曾踩的坑：

我以为"计算机用二进制"是因为二进制在数学上更优雅、更"底层正确"。被老谟一句话戳穿——是物理上两态比十态容错、便宜，才退而求其次。我还会把"整数除二取余"当成一个要背的口诀，忘了它其实是从"最低位是奇偶"一步步踩出来的。最丢人的是符号位法：我给 -5 贴了个负号标签，就以为搞定负数了，结果 $5+(-5)$ 在机器里算出 -10。但我这章最大的收获不是学会补码，是学会了不安——我两次问出"凭什么"，老谟说这就是证明的种子。

顿悟时刻：

真正开窍是老谟逼我想"把 -5 藏进正数区间"那一刻。 $256-5=251$ ，251 加 5 溢出成 0——减法凭空消失了。我突然懂了：负数从来不是"带负号的数"，而是"被重新编码、好让加法不挑肥拣瘦的数"。而更震撼的是严格化走廊里那个证明： $\sim x + 1 = 2^n - x$ ，只用了"每一位互补所以加起来全是 1"这一件事。原来证明不是天书，是把已经懂的东西按规则连起来——我这个"第一次写证明"的人，居然看懂了。

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A (计算, 易→中) : 批量换算 + 补码求值

(a) 把 29、64、131 转成二进制 (写出过程)。 (b) 写出 29、-29、-1、-128 的 8 位补码。 (c) 用 8 位补码计算 $(-29) + (-1)$, 验证结果。

完整解答: (a) $29=11101$; $64=1000000$; $131=10000011$ 。 (b) 29 的补码 = 00011101 ; $-29 = 2^8-29 = 227 = 11100011$; $-1 = 2^8-1 = 255 = 11111111$; $-128 = 2^8-128 = 128 = 10000000$ 。 (c) $11100011 + 11111111 = 1\ 11100010$, 取低 8 位 = 11100010 。 -30 的补码: $2^8-30 = 226 = 11100010$ ✓ 一致。

题 B (计算, 中→难) : 补码还原循环

给定 8 位补码 11101011 (它是某个负数 x 的补码)。 (a) 用"取反加一"还原出 $|x|$ 。 (b) 说出 x 是多少。 (c) 用"256 减"独立验证。

完整解答: (a) 取反: 00010100 ; 加 1: $00010101 = 21$ 。 (b) 所以 $x = -21$ 。 (c) $2^8 - 21 = 235 = 11101011$ ✓ (与给定一致, 确认 -21)。

题 C (概念, 检查理解) : 为什么补码必须证明

老谟说"取反加一和 256 减它在 8 位里永远一样", 造物主问"凭什么"。请用本章严格化走廊定理 2 的证明思路, 写一个简短的证明 ($x + \sim x = 2^n - 1$ 起手), 说明"取反加一 $\equiv 2^n - x$ "。

完整解答: 设 $0 \leq x < 2^n$ 。 x 的每一位非 0 即 1, $\sim x$ 是每一位取反, 故 $x + \sim x$ 每一位都是 1, 即 $x + \sim x = 111\dots 1 = 2^n - 1$ (n 个 1)。 两边加 1: $x + (\sim x + 1) = 2^n$ 。 移项: $\sim x + 1 = 2^n - x$ 。 ■

预告

比特哑着, 编码让它说话。这一章它开口报了数——0 和 1 能表示任何量, 而你也第一次尝到了"凭什么"的味道。

可光会"报数"还不够: 机器总得对这几个数做出判断, 比如"这个数字是不是负数"这两个条件成不成立。而"判断"这件事, 老谟说藏着证明之路的第一块正式台阶——**一个判断, 要么成立要么不成立, 可"凭什么成立"?**

下一章, 我们让比特从"会报数"升级到"会判断"——而判断的规则 (布尔代数) 和判断的依据 (逻辑证明), 是同一枚硬币的两面。

第 02 章 · 逻辑——开关怎么做判断

本章从哪个问题出发：上一章你学会了用开关“报数”——一串比特能表示任何量。可一台机器总不能只会报数：它得对数字**做判断**，“这个数是不是负数”“这两个条件成不成立”。**开关要怎么“决定”一件事成不成立？** 推导到哪个结论：判断的规则叫**布尔代数（Boolean algebra）**——AND/OR/NOT 三种基本开关，外加一套“开关的算术”（公理与定律）。最反直觉的结论是：**这么多逻辑运算，其实只需要一种叫 NAND 的开关，就能全部拼出来**——这叫 NAND 的完备性，而它的成立，靠的是一个“构造性证明”。

核心认知：**逻辑判断是一套人为约定的算术，不是天经地义；而“任意逻辑函数都能用 NAND 拼出来”这件事，是能证明的——证明的方法是“构造”：我把 NOT、AND、OR 一个个用 NAND 盖出来给你看。** 这一章你第一次见识“构造性证明”的威力：要证明“能做出来”，就真的动手做出来。

引子

周五傍晚，老谟在我桌上摆了三只小盒子。每只盒子有一根输入线、一根输出线。第一只盒子标着“AND”，第二只标着“OR”，第三只标着“NOT”。

“你上一章学会让开关报数了，”他说，“可报完数，机器得拿这些数去‘做决定’。你看——我给你两个开关，一开一关。你要不要决定‘两个都开’才放行，还是‘开一个就放行’？”

“那得看是干嘛用的啊。”我说。

“对喽。‘两个都得满足’、‘满足一个就行’、‘取反’——这三种判断，就是开关世界里的三种基本动作。”他把三只盒子排成一排，“可我现在要问你一个狠的：**如果我只给你一种盒子，你能不能用它，把另外两种也拼出来？**”

“只给一种，拼出另外两种？”我盯着那三只盒子，“这……三种动作明明不一样，一种怎么够？”

“‘够不够’这件事，”老谟把其中一只盒子拿起来，“在数学里不靠感觉，靠**证明**。而且证明的方法，是‘动手造’——我要把你逼到‘只给一种，你自己拼出全部’的地步。你先把这三种动作是什么搞明白，再来想‘够不够’。”

我盯着那三只盒子，总觉得“一种拼出全部”像是变戏法。可老谟说这事能证明——而且靠的是“动手造”。直觉和可证之间，好像隔着一道我还没踩上去的台阶。

对话引擎

第 1 轮：三种判断——“都要”、“任一”、“反过来”

老谟：“先定义动作。给你两个输入 A、B，各自能是 0 或 1。**AND (与)**：A、B 都等于 1，结果才是 1；**OR (或)**：A、B 只要有一个是 1，结果就是 1；**NOT (非)**：只有一个输入，结果把它反过来（1 变 0，0 变 1）。你把这三种动作，用一张“输入→输出”的表列出来——能列吗？”

造物主：“……能。就是列出所有输入组合和对应输出。A、B 各有 0、1 两种，AND 和 OR 都有 4 行，NOT 有 2 行。”

老谟：“你列出来了。这张表有个名字，叫**真值表 (truth table)**——它把一个逻辑函数的“全部行为”一览无余地摊开。你先别急着往下走，看着这张表，回答我：**AND 和 OR 的区别在哪一行？**”

造物主：“……在 A=1、B=1 那一行！AND 给 1，OR 也给 1……不对，区别在 A、B 都 1 时，两者都 1；但在“一个 1 一个 0”时，OR 是 1，AND 是 0。哦，我看出来了：**AND 是“全都满足”，OR 是“至少一个满足”。**”

老谟：“你说得对。这三个动作，就是你判断一件事成不成立的最基本原料。**可它们不是从天上掉下来的——它们是约定。**你说‘AND 是都满足’，凭什么？凭什么不是反过来？”

造物主：“……这不就是定义吗？AND 就是这么定义的。”

老谟：“对，就是定义——**是人和机器约好的。**这一章最重要的认知之一，就是：**逻辑运算不是天经地义，是约定；可约定一旦立下，它自己会滋生出许多‘推得出来’的规律——那些规律，就要靠证明来钉。**你先把这三个约定记牢，下一轮我们看这些约定能长出什么。”

知识锚点：与、或、非——逻辑的三种基本动作

【历史溯源】**布尔代数 (Boolean algebra) 是 1854 年乔治·布尔 (George Boole) **在《思维规律的研究》里提出的——他把逻辑推理写成“代数”，用 0 和 1 表示“假”和“真”，用 $\wedge$ (与)、 $\vee$ (或)、 $\neg$ (非) 表示“并且”“或者”“非”。**布尔当时完全没想过计算机——他把这当成“人类思维规律的代数”。**真正把它接到机器上的是 **1938 年香农 (Claude Shannon) **的硕士论文《继电器与开关电路的符号分析》：**他证明继电器开关电路可以用布尔代数来分析，从此“逻辑”和“电路”焊在一起。从 1854 到 1938，跨了 84 年，数学和工程各走各的路，最后被香农一句话接通。**

【工程实践】工程师为什么用 AND/OR/NOT 当“积木”？因为**三极管开关天然就能实现这三种行为**：两个开关串联就是 AND (都通才通)，并联就是 OR (一通就通)，一个“反相器”就是 NOT。**用这三种“门”能搭出任意判断电路——大到 CPU 的加法器、比较器，小到一个“密码对不对”的检查。判断电路 = 用这三种门拼出来的布尔表达式。**

【概念深挖】**真值表 (truth table)**：把逻辑函数的所有输入组合和对应输出列成一张表，是逻辑函数的"完全描述"。

A	B	$A \wedge B$ (AND)	$A \vee B$ (OR)
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1

A	$\neg A$ (NOT)
0	1
1	0

记法：AND 写 $A \wedge B$ 或 $A \cdot B$ ，OR 写 $A \vee B$ 或 $A + B$ ，NOT 写 $\neg A$ 或 $\bar{A}$ 。**AND 像乘法（只要一个 0 就归 0），OR 像加法（有一个 1 就归 1）——这个"像"不是偶然，布尔代数刻意模仿了算术，但记住它只是"像"，规则得按布尔自己的来。**

第 2 轮：布尔代数——开关世界的"算术"

老谟："三个动作有了。现在我告诉你：**这三样东西放在一起，能凑出一整套'开关的算术'——一套公理和定律。**你先别急着背，我们先推一条。我问你： **$\neg(A \wedge B)$ 和 $\neg A \vee \neg B$ ，是一回事吗？**"

造物主：".....不知道。看着不像——一个是"把 AND 取反"，一个是"分别取反再 OR"，好像不一样？"

老谟："那我给你最笨、最可靠的办法：**把它们都列真值表，一列一行地比。**A、B 各 0、1 两种，一共 4 行。你把 $\neg(A \wedge B)$ 和 $\neg A \vee \neg B$ 各自的输出算出来，然后逐行比——如果每一行都一样，它俩就是同一个逻辑函数。"

造物主："那我列..... $A=0, B=0$ ： $\neg(0 \wedge 0) = \neg 0 = 1$ ； $\neg 0 \vee \neg 0 = 1 \vee 1 = 1$ 。 $A=0, B=1$ ： $\neg(0 \wedge 1) = \neg 0 = 1$ ； $\neg 0 \vee \neg 1 = 1 \vee 0 = 1$ 。 $A=1, B=0$ ： $\neg(1 \wedge 0) = \neg 0 = 1$ ； $\neg 1 \vee \neg 0 = 0 \vee 1 = 1$ 。 $A=1, B=1$ ： $\neg(1 \wedge 1) = \neg 1 = 0$ ； $\neg 1 \vee \neg 1 = 0 \vee 0 = 0$ 。**四行全一样！它俩真是同一个函数！**"

老谟："你刚用真值表，亲手验证了一条定律——**德摩根律**： $\neg(A \wedge B) = \neg A \vee \neg B$ 。它说的是：'否定一个'与'，等于各自否定再'或'。可你发现没有——你只是'验证了这一组 A、B'，你没证明'对所有 A、B 都对'。真值表只覆盖 2 个输入，能枚举完；可如果有 3 个、4 个、10 个输入，你还能一行行枚举吗？"

造物主：".....2 个输入 4 行，3 个输入 8 行，n 个输入 2^n 行。输入一多，真值表就爆表了。"

老谟："所以真值表是'验证'，不是'证明'。真正的证明，得用'代数'——把已知的定律当工具，一步步变形。这一章你会看到：真值表验证了德摩根律，而严格化走廊里，我们会用更硬的办法把它'钉死'。你先记着这个区别：**验证（枚举给一组看）和证明（说清为什么永远成立），是两回事。**"

知识锚点：布尔代数的定律——开关世界的"乘法口诀"

【概念深挖】**布尔代数**是一套公理加定律，就像算术有"乘法口诀"一样，逻辑也有自己的"口诀"。以下是核心定律（A、B、C 是布尔变量，0 是假、1 是真）：

定律	形式
交换律	$A \wedge B = B \wedge A, A \vee B = B \vee A$
结合律	$(A \wedge B) \wedge C = A \wedge (B \wedge C), (A \vee B) \vee C = A \vee (B \vee C)$
分配律	$A \wedge (B \vee C) = (A \wedge B) \vee (A \wedge C); A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$
同一律	$A \wedge 1 = A, A \vee 0 = A$
零一律	$A \wedge 0 = 0, A \vee 1 = 1$
互补律	$A \wedge \neg A = 0, A \vee \neg A = 1$
双重否定	$\neg(\neg A) = A$
德摩根律	$\neg(A \wedge B) = \neg A \vee \neg B; \neg(A \vee B) = \neg A \wedge \neg B$

注意两条"反直觉"的分配律：布尔代数里，**OR 对 AND 也分配** ($A \vee (B \wedge C) = (A \vee B) \wedge (A \vee C)$) ——这在普通算术里没有对应物 ($a + (b \times c) \neq (a + b) \times (a + c)$)。这是"约定"而不是"从算术搬来的"证据之一：**布尔代数有自己的规则，跟算术只是像，不是一回事。**

【历史溯源】德摩根律以 **奥古斯都·德摩根（Augustus De Morgan，19 世纪英国数学家）** 命名，他是布尔的同事、好友，两人共同推动了逻辑代数化。**德摩根律在工程里极其常用：**它是"把一个复杂判断取反"时的万能工具——你想判断"不是(A 且 B)"，直接拆成"不是A 或 不是B"就行。**电路设计师靠它把 NAND 门翻译成"负逻辑下的 OR"，一句话省下一整片电路。**

【工程实践】布尔代数的定律在工程里的实际价值是**化简**：同一个逻辑功能，可以写成许多等价的表达式，有的表达式用的门多、有的门少。**工程师的目标是把表达式化到最少门**（省面积、省功耗、省延迟）——化简靠的就是这些定律（尤其分配律、德摩根律）。所以"布尔代数化简"**不是数学游戏，是芯片里每一纳秒和每一平方毫米都在争的东西。**

第 3 轮：逻辑等价——"同一个函数"到底凭什么

老谟："你验证了 $\neg(A \wedge B) = \neg A \vee \neg B$ 。那我想问你：你说的'同一个逻辑函数'，凭什么？你怎么知道两个长得完全不一样的表达式，其实是同一个东西？"

造物主：".....真值表一样啊。每个输入组合，输出都一样。"

老谟："对。两个逻辑表达式等价 (equivalent)，精确的定义就是：对每一组输入，它们的输出都相同——换句话说，它们的真值表一模一样。这定义跟'长得像不像'无关，只跟'行为一不一样'有关。你以后化简表达式，抓的就是这个：换个长得完全不同的写法，但只要行为一样，就还是同一个函数。"

造物主："那.....那我化简的时候，怎么知道改完还是同一个？难道每次都列真值表？"

老谟："列真值表是'验证'，代数变形（用定律一步步推）是'证明'。工程师两头都用：变量少时列真值表，变量多时用代数定律变形，并记得每一步变形的依据。你现在先抓住'等价=真值表相同'这个精确定义，等严格化走廊里，我们用代数把它钉成更硬的证明。下一轮，我们玩个大的——"他只给一种盒子"那个问题，该揭晓了。"

第 4 轮：NAND——一种开关，凭什么能拼出全部

老谟："回到开头那只盒子。现在我只给你一种：**NAND**。它的动作是'先 AND 再 NOT'—— $A \text{ NAND } B = \neg(A \wedge B)$ 。就这一种盒子，一堆，管够。我问你：你能不能用 NAND，拼出 NOT？"

造物主："NAND 是两个输入的.....我拿一个输入接到两个头上？A 和 A 都接同一个？那 $A \text{ NAND } A = \neg(A \wedge A) = \neg A$ ！**就一个 NAND，两个输入都接同一个 A，就是 NOT！**"

老谟："漂亮。你把两个输入短路成一个，NAND 就退化成 NOT 了。现在——**拼出 AND**。你有 NOT 了，能拼 AND 吗？"

造物主：".....NAND 是 $\neg(A \wedge B)$ ，我把它再 NOT 一下不就是 $A \wedge B$ 了？ $A \wedge B = \neg(\neg(A \wedge B)) = \neg(A \text{ NAND } B)$ 。而我刚学会用 NAND 造 NOT，所以拿一个 NAND 的输出，再接一个'短路 NAND'当 NOT，就是 $A \wedge B$ ！"

老谟："你用两个 NAND：第一个算出 $A \text{ NAND } B$ ，第二个把它取反，就得到 $A \wedge B$ 。**AND 也被你拼出来了**。现在最后一个——**OR**。你手上有 NAND 和 NOT（还有刚拼出的 AND），能拼 OR 吗？"

造物主：".....OR 是'至少一个为 1'。我记得德摩根律： $A \vee B = \neg(\neg A \wedge \neg B)$ 。那就是把 A、B 分别 NOT 一下，再 AND，再 NOT。 $A \vee B = \neg(\neg A \wedge \neg B)$ ——我可以先短路 NAND 造出 $\neg A$ 、 $\neg B$ ，再 NAND 它俩（等于 $\neg(\neg A \wedge \neg B)$ ），再短路 NAND 取反....."

老谟："你链起来了。到这里，你已经用 NAND 拼出了 NOT、AND、OR 全部三种基本动作。现在我问你那个最狠的问题——**NAND 凭什么'够'？**你别说'因为我把三种都拼出来了'——你想想：为什么拼出这三种，就等于拼出了'一切'？"

造物主：".....因为一切逻辑函数，不就是用这三种动作搭出来的吗？只要 NOT、AND、OR 都能用 NAND 造，那任何用这三种搭出来的东西，也都能换成 NAND！"

老谟："你摸到了那个'为什么'的边。但这还不是'证明'——你只是'相信'一切函数都由 AND/OR/NOT 构成。下一轮，我们把这个'凭什么'，写成严格的构造性证明：先证明'一切布尔函数能用 AND/OR/NOT 表达'，再证明'AND/OR/NOT 各能用 NAND 表达'，两头一接，就证完 NAND 完备。你先把这三种拼法记牢，尤其是那句'短路 NAND 就是 NOT'。"

知识锚点：NAND 与它的三种拼法

【概念深挖】NAND (与非门)： $A \text{ NAND } B = \neg(A \wedge B)$ ，真值表在"两个都是 1"时输出 0，其余输出 1（就是 AND 的输出取反）。它只有一个"0"的位置，其余全是 1。

用 NAND 拼出三种基本动作（这几条是"构造性证明"的核心材料，务必亲手验）：

- **NOT**： $A \uparrow A = \neg(A \wedge A) = \neg A$ 。（把两个输入短路。）
- **AND**： $A \wedge B = (A \uparrow B) \uparrow (A \uparrow B)$ 。（先 NAND，再对结果短路 NAND 取反。）
- **OR**： $A \vee B = (A \uparrow A) \uparrow (B \uparrow B)$ 。（先各自短路 NOT，再 NAND——用德摩根律 $A \vee B = \neg(\neg A \wedge \neg B)$ 。）

为什么 NAND 这么"全"：因为它天然"否定 + 与"一肩挑。否定（NOT）是逻辑里最"基础"的反转动作，与（AND）是"全都满足"，两者一组合，就足以"织"出任何函数。这是布尔代数的一条深刻性质：**完全集 (functionally complete)**——一小撮运算（甚至一种，如 NAND）就能表达所有布尔函数。

【历史溯源】NAND 的"一神论"地位，是工程反向逼出来的。1960-70 年代，TTL（晶体管-晶体管逻辑）芯片厂家发现：只要生产一种 NAND 门，就能搭出所有其他门（把 NAND 拼成 AND/OR/NOT），库存和工艺都省到极致。于是"NAND-only"成了早期数字设计的默认打法。数学上"完全集"的概念早就有，但让它大红大紫的是工程——成本结构逼着工程师只认一种门。到 FPGA（可编程逻辑门阵列）时代，查表（LUT）和 NAND 式的"少量基础资源撑起全部逻辑"思想，依然活着。

【工程实践】NAND 在工程里是"最小万能积木"：逻辑门芯片（74 系列）里 NAND 门数量最多、最常用；任何布尔函数，工程师都可以只用 NAND 门实现，这在"只有一个门类型的库存"或"FPGA 的查找表化简"时极其实用。你会在第 07 册（可计算性）看到：'一小撮运算能表达一切'这个思想，会从'逻辑函数'升级到'程序'——NAND 完备性，是那条路的第一个脚印。

第 5 轮：构造性证明——“能拼出来”怎么被钉死

老谟：“你说‘一切逻辑函数都能用 NAND 拼’——这话现在还是你的直觉。我要你把它变成证明。证明‘能做出’这件事，最硬的办法是什么？”

造物主：“.....真的做出？可一切函数那么多，我总不能一个个拼。”

老谟：“对，所以不能一个个拼，要构造一个通用方法。我问你：给你任意一个布尔函数——不管它多复杂、几个输入——你怎么保证能用 AND/OR/NOT 表达它？你手里唯一知道的，就是它的真值表。你拿真值表能干嘛？”

造物主：“.....真值表？我把输出是 1 的那些行挑出来？每一行代表一组输入组合，比如 $A=1, B=0$ 输出 1.....”

老谟：“对！把输出为 1 的每一行，写成‘这一行的条件’——比如 $A=1$ 且 $B=0$ 写成 $(A \wedge \neg B)$ ；然后把所有这样的行用 OR 连起来。因为‘函数输出 1’，等价于‘命中了这些行中的某一行’。你想想，这个法子对‘任何’函数都行得通吗？”

造物主：“.....行得通！只要函数的真值表能写出来，我就能把‘输出为 1 的行’一个个用 AND 描述、再用 OR 串起来——这就是一个通用的‘从真值表造 AND/OR/NOT 表达式’的流水线！”

老谟：“你刚构造出了一个**万能方法**：任何布尔函数，都能用 AND/OR/NOT 写出来（这叫**析取范式，DNF**）。这一步是‘构造’——你给了具体的做法，而不只是说‘应该能’。现在把两段拼起来：第一段，任何函数都能用 AND/OR/NOT 表达（你刚构造的）；第二段，AND/OR/NOT 各能用 NAND 拼（你上一轮拼的）。两段一接——任何函数都能用 NAND 表达。这就是 NAND 完备性的**构造性证明**。你说说，这个证明为什么‘硬’？”

造物主：“.....因为它不是空口说‘应该能’，它给了‘具体怎么做’——真值表→DNF→换成 NAND，每一步都是明确的操作。谁能照着走，谁就能真拼出来。”

老谟：“这句话，就是‘构造性证明’的定义：要证明‘存在’，就把它‘造’出来给你看。你这一章，亲手走完了一个构造性证明的全过程。把它记住——第 05 章我们系统讲证明方法时，你是全册第一个见过‘构造’招式的读者。”

知识锚点：构造性证明——“存在”靠“造出来”来证

【概念深挖】构造性证明 (constructive proof)：要证明“存在某个东西”（比如“存在一种用 NAND 拼出函数的方法”），不是抽象地论证它“必然存在”，而是真的构造出一个具体的东西，证明它满足要求。它的口号是：“你说能做，做给我看。”

本章的构造性证明分两段（完整版在严格化走廊）：

1. **任何布尔函数能用 AND/OR/NOT 表达**（构造析取范式 DNF）：把真值表中输出为 1 的每一行写成“该行输入条件的 AND”，再用 OR 串起来。
2. **AND/OR/NOT 各能用 NAND 表达**： $\text{NOT} = A \uparrow A$ ， $\text{AND} = (A \uparrow B) \uparrow (A \uparrow B)$ ， $\text{OR} = (A \uparrow A) \uparrow (B \uparrow B)$ 。
3. **两段复合**：把第 1 段得到的 AND/OR/NOT 表达式里的每个 AND/OR/NOT 依次换成 NAND 拼法，得到纯 NAND 表达式。

这为什么是“证明”：因为它给出了从“任意函数”到“NAND 表达式”的**明确、可执行、永不失败**的步骤，而不是“感觉应该行”。**能照着做，就是证明**。对比：反证法（第 05 章）是“假设不对，推出矛盾”；构造性证明是“直接做出对的”——两种招式，各有适用。

【历史溯源】“构造性证明”在数学史上是**有立场**的：****直觉主义数学（布劳威尔，20 世纪初）****坚持“证明存在必须造出来，不能只靠反证”，和希尔伯特的形式主义吵了很久。但**“构造”本身作为证明技巧，无人反对**——从欧几里得的几何作图（“尺规作正五边形”就是构造）到现代算法设计（“写个程序生成它”就是构造），构造性证明一直是数学和工程的共通语言。**对你来说，最贴身的例子是编程：你要证明“能实现某个功能”，最好的证明就是把它写出来跑通。**

【工程实践】工程里“构造性证明”无处不在，只是换了个名字——**构造算法**：要证明“存在一种算法能解决这个问题”，就真的写出来、证明它终止、证明它输出正确。**NAND 完备性在工程里最直接的应用**：FPGA 的查找表、最小逻辑门库设计，都依赖“一小撮基础运算能表达一切”这条定理。而**“真值表→DNF”那个构造，就是今天编译器里“把真值表转成逻辑表达式”的原型。**

第 6 轮：落点——判断的算术，也是证明的练兵场

老谋：“把这一章收一遍。你从‘开关怎么做判断’出发，最后拿到什么？”

造物主：“我拿到三种基本判断 AND/OR/NOT，还有一套布尔代数的‘算术’——公理和定律。我发现‘等价’的精确意思是‘真值表相同’。最狠的是 NAND：我亲眼看着‘短路 NAND 就是 NOT’，然后一步步用 NAND 拼出 AND、OR，最后用‘真值表→DNF→换 NAND’这条流水线，证明任何函数都能用 NAND 表达。”

老谋：“你收得齐。这一章你其实经历了一次完整的‘证明之旅’——虽然你没意识到。你从‘一种盒子不够吧’的怀疑，走到‘用构造证明它够’。**你第一次见到了‘构造性证明’这个招式。**你说说，这一章和上一章比，你对‘证明’的感觉变了没？”

造物主：“变了。上一章我说‘取反加一凭什么一样’，这一章我看到了一个完整的证明是怎么长出来的——它不玄，它给步骤，步骤能走通，就成了。**而且我意识到：逻辑判断的规则，全是我们约定的；可约定一旦立下，能推出什么，就是必然的了——那部分不需要商量，只需要证明。**”

老谋：“这话记好了。**判断的规则是约定（AND 凭什么这样，是我们说的）；约定之下的推论是必然（ $\neg(A \wedge B) = \neg A \vee \neg B$ ，不是我们说的，是推出来的）。**约定和推论的分界线，正是‘证明’工作的地方。

下一章，我们离开'开关'，走进一个更抽象的地方——**把东西'圈起来'的集合**。而那里，你会第一次看到'证明'另一种更常用的形态。"

严格化走廊

位置说明：对话负责"直觉"——判断为什么是算术、NAND 为什么够。本走廊负责"严谨"——把"任意函数能用 NAND 表达"落成可复写的构造性证明。下面是读者接触"构造性证明"的第一次正式亮相：**每一步写清楚用哪条定义/定律，绝不说"显然"**。

证明类型标注：本章的证明有两种招式：

- **定理 1（德摩根律）：直接证明（代数型）**——由真值表/定义直接推出，逐项核对；
- **定理 2（NAND 完备性）：构造性证明**——要证"存在一种 NAND 表达"，就真的构造出"从任意函数到 NAND 表达式"的步骤。完整的证明方法分类（直接/反证/归纳/构造）会在第 05 章系统展开，这里你先认识"直接"和"构造"这两招。

定义：布尔函数、NAND

布尔变量：取 0（假）或 1（真）的变量。

n 元布尔函数（Boolean function） $f: \{0,1\}^n \rightarrow \{0,1\}$ ，即对每组 0/1 输入，给一个 0/1 输出。每个布尔函数由它的**真值表**唯一确定（ 2^n 行）。

三种基本运算：AND ($A \wedge B$ ，都 1 才 1)、OR ($A \vee B$ ，任一 1 即 1)、NOT ($\neg A$ ，取反)。

NAND（与非）： $A \uparrow B = \neg(A \wedge B)$ 。真值表：除" $A=B=1$ "输出 0，其余输出 1。

定理 1（德摩根律）： $\neg(A \wedge B) = \neg A \vee \neg B$ 与 $\neg(A \vee B) = \neg A \wedge \neg B$

前提：A、B 是布尔变量。

证明类型：直接证明（真值表逐项核对）。

证明（以第一条为例，第二条同理）：列出 A、B 的 4 组取值，逐项计算 $\neg(A \wedge B)$ 与 $\neg A \vee \neg B$ ：

A	B	$A \wedge B$	$\neg(A \wedge B)$	$\neg A$	$\neg B$	$\neg A \vee \neg B$
0	0	0	1	1	1	1
0	1	0	1	1	0	1
1	0	0	1	0	1	1
1	1	1	0	0	0	0

逐行比较第 5 列与第 8 列：4 行完全相同，故 $\neg(A \wedge B) = \neg A \vee \neg B$ 。■

读法：这是"直接证明"最简单的一种——**穷举所有情况**。 n 个变量共有 2^n 种情况， n 很小时（如 2、3）穷举可行，这就是为什么真值表在变量少时是"证明"；变量多时（ $n \geq 5$ ） 2^n 爆炸，真值表降级为"验证"，得用代数变形来证（第 05 章会展开）。**判定标准：**穷举完备即为证明；枚举不完就只是验证。

定理 2 (NAND 完备性)：任意 n 元布尔函数都能用 NAND 表达

前提： f 是任意 n 元布尔函数（由真值表给定）。

证明类型：构造性证明（分两段，每段给出可执行的构造步骤）。

第 1 段：任何布尔函数能用 AND/OR/NOT 表达（构造析取范式 DNF）

- 第 1 步：取 f 的真值表，只看输出为 1 的那些行。
- 第 2 步：对每一行，构造一个"该行条件"的 AND 项：若第 i 个变量在该行为 1 就写 A_i ，为 0 就写 $\neg A_i$ ，把这些文字用 $\wedge$ 连起来。（该 AND 项只在"这一行"取值时为 1，其余行为 0。）
- 第 3 步：把所有这样的 AND 项用 $\vee$ 连起来，得到 DNF 表达式 g 。
- 第 4 步：对任意输入， g 为 1 $\Leftrightarrow$ 至少一个 AND 项为 1 $\Leftrightarrow$ 输入命中了某个"输出为 1 的行" $\Leftrightarrow f$ 为 1。故 $g = f$ 。■（第 1 段完成：任何函数可表为 AND/OR/NOT）

第 2 段：AND/OR/NOT 各能用 NAND 表达

- NOT: $A \uparrow A = \neg(A \wedge A) = \neg A$ 。（NAND 定义 + $A \wedge A = A$ 的幂等律。）
- AND: $A \wedge B = \neg(A \uparrow B) = (A \uparrow B) \uparrow (A \uparrow B)$ 。（NAND 定义 + 双重否定律。）
- OR: $A \vee B = \neg(\neg A \wedge \neg B) = (A \uparrow A) \uparrow (B \uparrow B)$ 。（德摩根律 $\neg(\neg A \wedge \neg B) = A \vee B$ ，加 NOT 的 NAND 拼法。）
- 故 AND/OR/NOT 各自存在纯 NAND 的替换式。■（第 2 段完成）

第 3 段：两段复合

- 对任意布尔函数 f ，先按第 1 段构造出它的 DNF 表达式 g （只含 AND/OR/NOT）。
- 把 g 中每个 AND、OR、NOT 依次替换成第 2 段的纯 NAND 拼法。
- 替换不改变函数行为（每步都用了等价定律），故得到纯 NAND 表达式，且 $= f$ 。■

读法：这是"构造性证明"的完整示范——它不是证明"存在"，而是**给出从任意函数到 NAND 表达式的明确流水线**。三个入口：①"输出为 1 的行 → AND 项 → OR 串起来" (DNF)；②"短路 NAND 是 NOT"；③"德摩根律把 OR 变成 NOT+AND"。**你能照着做出来，就是证明。**这正是第 05 章"构造性证明"定义的模样。

反例/边界：逻辑与 NAND 的边界情况

- **只用 AND、OR 不完备：**AND 和 OR 没有"取反"能力，拼不出 NOT。例：恒为 0 的函数、恒为 1 的函数，AND/OR 都造不出"翻转"。**要"否定"才能齐全——所以 NAND (自带否定) 才完备，而 AND/OR 组合不完备。**
- **变量多时真值表不可行：** $n=10$ 时真值表 $2^{10}=1024$ 行， $n=30$ 时 $2^{30}\approx 10$ 亿行——**穷举不再是证明手段，必须用代数变形/其他证明方法。**这是"验证"和"证明"在规模上的分水岭。
- **德摩根律的"反直觉"在于 OR 也分配：** $A\vee(B\wedge C)=(A\vee B)\wedge(A\vee C)$ ，普通算术没有对应物——**这是约定的产物，不是算术搬来的，别用算术直觉去套。**
- **"等价"只看行为：**两个表达式长得完全不同（如 $\neg(A\wedge B)$ 和 $\neg A\vee\neg B$ ），只要真值表相同就是同一个函数。**判断逻辑等价，别凭长相，看行为。**

知识点总结

知识点（本章推出的核心概念）

- **布尔代数 (Boolean algebra)：**逻辑判断的"算术"，1854 布尔提出；AND/OR/NOT 三种基本动作 + 一套公理定律。
- **真值表 (truth table)：**逻辑函数的完全描述；变量少时是证明，变量多时降级为验证 (2^n 爆炸)。
- **德摩根律：** $\neg(A\wedge B)=\neg A\vee\neg B$ ， $\neg(A\vee B)=\neg A\wedge\neg B$ ——"取反一个与/或"拆开。
- **逻辑等价：**真值表完全相同 = 同一个函数，与"长得像不像"无关。
- **NAND 完备性：**任意布尔函数都能用 NAND 表达（构造性证明：真值表 $\rightarrow$ DNF $\rightarrow$ 换 NAND）。
- **约定 vs 推论：**AND/OR/NOT 是约定；约定之下的定律（德摩根等）是必然，靠证明钉死。
- **构造性证明：**证明"存在"靠"造出来"——给明确可执行的步骤。

历史结论（本章涉及的史料）

- **布尔 (1854)** 《思维规律的研究》：逻辑代数化，用 0/1 表示假/真。
- ****香农 (1938) **** 硕士论文：继电器开关电路可用布尔代数分析——逻辑和电路接通，84 年后。

- **德摩根 (19 世纪)**：德摩根律，布尔的同事，共同推动逻辑代数化。
- **TTL 芯片时代 (1960-70)**：厂商只生产 NAND 一种门搭全部，工程反向逼出"NAND-only"打法。

工程实践结论（本章知识在真实工程里怎么用）

- **判断电路 = 布尔表达式**：AND/OR/NOT 三极管门搭出任意判断（加法器、比较器、密码检查）。
- **化简省门**：布尔定律（分配律、德摩根律）把表达式化到最少门，省面积、功耗、延迟。
- **NAND 是万能积木**：74 系列 NAND 门最常用；FPGA 查找表、最小逻辑门库依赖"一小撮运算表达一切"。
- **跨册**：NAND 完备性"一小撮运算表达一切"的思想，会在第 07 册（可计算性）从"逻辑函数"升级到"程序"。

计算训练场

位置说明：对话负责"为什么"，这里负责"怎么算"。每题动笔 10 分钟以上，先自己写再对答案。

题 1（易→中）：化简二变量表达式并用真值表验证

题干：老谟给了一个逻辑表达式 $F = (A \wedge B) \vee (A \wedge \neg B) \vee (\neg A \wedge B)$ 。(a) 用布尔代数定律化简它（提示：对前两项用分配律提公因式 A ）。(b) 写出化简后的最简形式。(c) 列 4 行真值表，逐行对比化简前后的 F 与 $A \vee B$ ，验证二者等价。

完整分步解答：

(a) 化简：

- 第 1 步：前两项 $(A \wedge B) \vee (A \wedge \neg B)$ ，提公因式 A ： $= A \wedge (B \vee \neg B)$ 。
- 第 2 步： $B \vee \neg B = 1$ （互补律），故前两项 $= A \wedge 1 = A$ （同一律）。
- 第 3 步：整个 $F = A \vee (\neg A \wedge B)$ 。
- 第 4 步： $A \vee (\neg A \wedge B)$ 用分配律 $A \vee (\neg A \wedge B) = (A \vee \neg A) \wedge (A \vee B) = 1 \wedge (A \vee B) = A \vee B$ 。
- 所以 $F = A \vee B$ 。

(c) 真值表验证：

A	B	$A \wedge B$	$A \wedge \neg B$	$\neg A \wedge B$	F(原)	$A \vee B$
0	0	0	0	0	0	0
0	1	0	0	1	1	1
1	0	0	1	0	1	1
1	1	1	0	0	1	1

逐行对比第 6 列与第 7 列：完全相同。**F 化简为 $A \vee B$ 正确。**

答：F = $A \vee B$ 。

常见错误：① 第 4 步直接用"吸收律" $A \vee (\neg A \wedge B) = A \vee B$ 却没意识到它其实是分配律+互补律的组合，步骤要写全；② 化简后忘了列真值表验证，光靠代数怕算错——**代数化简和真值表验证双保险。**

题 2 (中→难)：枚举真值表证明两个表达式逻辑等价

题干：老谟说两个表达式 $G = \neg(A \wedge B) \vee C$ 与 $H = (\neg A \vee \neg B) \vee C$ 是等价的。请：(a) 写出 G 与 H 的完整真值表 (A、B、C 三变量，8 行)。(b) 逐行比较，证明二者等价。(c) 说明这个等价背后用的是哪条定律。

完整分步解答：

(a)(b) 完整真值表 (8 行)：

A	B	C	$A \wedge B$	$\neg(A \wedge B)$	$\neg A \vee \neg B$	$G = \neg(A \wedge B) \vee C$	$H = (\neg A \vee \neg B) \vee C$
0	0	0	0	1	1	1	1
0	0	1	0	1	1	1	1
0	1	0	0	1	1	1	1
0	1	1	0	1	1	1	1
1	0	0	0	1	1	1	1
1	0	1	0	1	1	1	1
1	1	0	1	0	0	0	0
1	1	1	1	0	0	1	1

- 第 7 列与第 8 列逐行对比：8 行完全相同 $\Rightarrow G = H$ 。
- 注意 $A=1, B=1, C=0$ 那行 (第 7 行)： $A \wedge B=1$ ， $\neg(A \wedge B)=0$ ，且 $\neg A \vee \neg B=0$ (德摩根)，所以 $G=H=0$ ——这是最容易算错的一行。

(c) 背后定律：G 中的 $\neg(A \wedge B)$ 由**德摩根律** $\neg(A \wedge B) = \neg A \vee \neg B$ 替换，即得 H。**真值表是“验证”，德摩根律是“证明”**——本题用真值表枚举完备了（8 行），故作为证明成立；同时它印证了德摩根律。

答：二者等价（8 行真值表全同）；背后是德摩根律。

常见错误：① 三变量真值表漏行（必须 8 行，顺序按二进制 000→111）；② 算 $\neg A \vee \neg B$ 时忘了“任一 $\neg$ 为 1 结果就是 1”（OR 规则）；③ 只在 $A=1, B=1$ 那一行比对，忽略其他行——**等价必须全部行都相同**。

题 3（中→难）：只用 NAND 表达一个复杂函数

题干：老谟给了一个逻辑函数 $f(A, B, C) = (A \wedge B) \vee \neg C$ 。(a) 先写出它的真值表，确认它确实是个合法的布尔函数。(b) 用“短路 NAND 是 NOT”等拼法，把 f 改写成只含 NAND 的表达式（写出每一步替换）。(c) 任取一行（如 $A=1, B=1, C=0$ ）代入 NAND 表达式，验证与原函数输出一致。

完整分步解答：

(a) 真值表：

A	B	C	$A \wedge B$	$\neg C$	$f = (A \wedge B) \vee \neg C$
0	0	0	0	1	1
0	0	1	0	0	0
0	1	0	0	1	1
0	1	1	0	0	0
1	0	0	0	1	1
1	0	1	0	0	0
1	1	0	1	1	1
1	1	1	1	0	1

(b) 换 NAND：

- $f = (A \wedge B) \vee \neg C$ 。
- 第 1 步： $A \wedge B = (A \uparrow B) \uparrow (A \uparrow B)$ （AND 的 NAND 拼法）。
- 第 2 步： $\neg C = C \uparrow C$ （NOT 的 NAND 拼法）。
- 第 3 步：设 $P = (A \uparrow B) \uparrow (A \uparrow B)$ （表示 $A \wedge B$ ）， $Q = C \uparrow C$ （表示 $\neg C$ ）。则 $f = P \vee Q$ 。
- 第 4 步：OR 的 NAND 拼法： $P \vee Q = (P \uparrow P) \uparrow (Q \uparrow Q)$ 。
- 第 5 步：代入， $f = [((A \uparrow B) \uparrow (A \uparrow B)) \uparrow ((A \uparrow B) \uparrow (A \uparrow B))] \uparrow [(C \uparrow C) \uparrow (C \uparrow C)]$ 。
- 更简洁： $P = A \wedge B$ 记作中间量， $f = (P \uparrow P) \uparrow (Q \uparrow Q)$ ，其中 $P = (A \uparrow B) \uparrow (A \uparrow B)$ ， $Q = C \uparrow C$ 。

(c) 验证一行 ($A=1, B=1, C=0$) : 原函数 $f=(1\wedge 1)\vee \neg 0=1\vee 1=1$ 。

- NAND 式: $A\uparrow B = 1\uparrow 1 = 0$; $P=(0)\uparrow(0)=1$ ($=A\wedge B$ ✓) 。 $C\uparrow C=0\uparrow 0=1$ ($=\neg C$ ✓) 。 $f=(P\uparrow P)\uparrow(Q\uparrow Q)=(1\uparrow 1)\uparrow(1\uparrow 1)=0\uparrow 0=1$ ✓ 与原函数一致。

答: (a) 真值表见上; (b) $f = (P\uparrow P)\uparrow(Q\uparrow Q)$, 其中 $P=(A\uparrow B)\uparrow(A\uparrow B)$ 、 $Q=C\uparrow C$; (c) 该行 NAND 式也输出 1 ✓。

常见错误: ① 把 OR 的 NAND 拼法记错 (正确是 $(P\uparrow P)\uparrow(Q\uparrow Q)$, 即"各自短路 NOT 再 NAND", 用的是德摩根 $A\vee B=\neg(\neg A\wedge \neg B)$) ; ② 代入验证时把 P、Q 算错 ($P=(A\uparrow B)\uparrow(A\uparrow B)$ 里外两层短路 NAND 都要算) ; ③ 忘了 NAND 是"先与后非", $A\uparrow B=\neg(A\wedge B)$, 别算成 $A\wedge B$ 。

向导便签

这一章咱们干了什么: 让比特从"会报数"升级到"会判断"。你拿到了三种基本判断 AND/OR/NOT, 和一套布尔代数"算术"; 学会了用真值表"验证"、用代数定律"化简"; 最重要的是——你亲眼见证了 NAND 完备性的**构造性证明**: 先构造"真值表 $\rightarrow$ DNF"把任何函数写成 AND/OR/NOT, 再构造"短路 NAND 是 NOT"等拼法把 AND/OR/NOT 换成 NAND, 两段一接, 证明完成。

钉死一句话: **逻辑判断的规则是约定 (AND 凭什么这样, 是我们说的); 约定之下的定律是必然 (德摩根律不是我们说的, 是推出来的)**。而"任意函数都能用 NAND 表达"这个"能", 靠的不是感觉, 是**构造性证明**——你把它造出来, 它就算数。

记住: **验证 (枚举给一组看) 和证明 (说清为什么永远成立) 是两回事**——变量少真值表够用, 变量多就必须代数变形。而"等价"只看行为 (真值表), 不看长相。

造物主手记

我曾踩的坑:

我第一坑是把"布尔代数"当成"算术搬过来的"——被老谟一句"OR 对 AND 也分配? 普通算术可没这条"噎住, 才明白逻辑有它自己的约定, 跟算术只是像。第二坑是以为"逻辑等价=长得像"——其实 $\neg(A\wedge B)$ 和 $\neg A\vee \neg B$ 长得完全不像, 可真值表四行全同, 它俩就是同一个函数。第三坑是化简时想用"吸收律"却忘了它是分配律+互补律推出来的, 老谟让我把步骤写全——**化简要每一步标定律, 不能跳**。最震撼的是 NAND: 我说"一种盒子怎么够", 结果亲眼看着短路 NAND 变成 NOT, 再一步步拼出 AND、OR, 最后整条"真值表 $\rightarrow$ DNF $\rightarrow$ 换 NAND"的流水线走完——**原来"能造出来"本身就是证明**。

顿悟时刻:

真正开窍是那句"构造性证明：你说能做，做给我看"。以前我以为证明都是"写一堆抽象符号"，可 NAND 完备性根本不是——它就是"我把 NOT、AND、OR 一个个用 NAND 造出来给你看"。**我突然明白，证明的'硬'，不在符号多玄，而在步骤能不能走通。**而"约定 vs 推论"这条分界线，让我看清了整章：AND/OR/NOT 是人定的，可德摩根律、NAND 完备性，是人定完之后"推出来"的必然——**约定的松，推论的硬，这一章全尝到了。**

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A (计算, 易→中)：化简 + 真值表验证

化简 $F = (A \vee B) \wedge (A \vee \neg B)$ 并列 4 行真值表验证化简前后等价。

完整解答：

- 化简： $F = (A \vee B) \wedge (A \vee \neg B)$ 。提公因式 A (分配律)： $= A \vee (B \wedge \neg B)$ 。 $B \wedge \neg B = 0$ (互补律)，所以 $F = A \vee 0 = A$ (同一律)。
- 真值表： $A=0, B=0$: $F = (0 \vee 0) \wedge (0 \vee 1) = 0 \wedge 1 = 0$, $A=0 \checkmark$; $A=0, B=1$: $(0 \vee 1) \wedge (0 \vee 0) = 1 \wedge 0 = 0$, $A=0 \checkmark$; $A=1, B=0$: $(1 \vee 0) \wedge (1 \vee 1) = 1 \wedge 1 = 1$, $A=1 \checkmark$; $A=1, B=1$: $1 \wedge 1 = 1$, $A=1 \checkmark$ 。四行全等， $F=A$ 。

题 B (计算, 中→难)：NAND 完备性实操

老谟要求只用 NAND 表达 $g = \neg(A \vee B)$ 。(a) 用短路 NAND 与德摩根律，写出 g 的纯 NAND 表达式。(b) 任取一行 ($A=1, B=0$) 验证。

完整解答：(a) $g = \neg(A \vee B)$ 。由德摩根律 $\neg(A \vee B) = \neg A \wedge \neg B$ 。 $\neg A = A \uparrow A$, $\neg B = B \uparrow B$ 。 $\neg A \wedge \neg B = (\neg A \uparrow \neg B) \uparrow (\neg A \uparrow \neg B)$ 。代入： $g = [((A \uparrow A) \uparrow (B \uparrow B))] \uparrow [((A \uparrow A) \uparrow (B \uparrow B))]$ $= (P \uparrow P) \uparrow (P \uparrow P)$ 其中 $P = (A \uparrow A) \uparrow (B \uparrow B)$ 。整理： $g = (Q \uparrow Q)$ 其中 $Q = (A \uparrow A) \uparrow (B \uparrow B)$ 。(用中间量： $g = Q \uparrow Q$, $Q = (A \uparrow A) \uparrow (B \uparrow B)$ 。)(b) 验证 $A=1, B=0$ ：原函数 $\neg(1 \vee 0) = \neg 1 = 0$ 。 $Q = (1 \uparrow 1) \uparrow (0 \uparrow 0) = (\neg(1 \wedge 1)) \uparrow (\neg(0 \wedge 0)) = (0) \uparrow (1) = \neg(0 \wedge 1) = \neg 0 = 1$ 。 $g = Q \uparrow Q = 1 \uparrow 1 = 0 \checkmark$ 与原函数一致。

题 C (概念, 检查理解)：构造性证明 vs 验证

(a) "列真值表"为什么变量少时是证明、变量多时只是验证？(b) 构造性证明和"感觉应该能"的本质区别是什么？(c) 为什么 AND、OR 组合不完备，而 NAND 完备？

完整解答：(a) 真值表穷举所有 2^n 种输入。 n 小时 (2、3) 枚举完备，每行都核对过，故是证明； n 大时 (如 30) $2^{30} \approx 10$ 亿行，无法枚举完，只能抽查验证，不构成证明。(b) 构造性证明给出明确、可执行、永不失败的步骤 (如"真值表 $\rightarrow$ DNF $\rightarrow$ 换 NAND")，能照着造出来；"感觉应该能"没有具体做

法，只停留在直觉。(c) AND、OR 都不带"取反" (NOT)，拼不出恒为 0/1 的翻转，故不完备；NAND 自带"与+非"，用短路 NAND 能造 NOT，再由 NOT+AND 造 OR，从而覆盖全部基本动作，故完备。

预告

你让比特"会判断"了——AND、OR、NOT 三种动作，加上布尔代数这套"开关的算术"，你甚至证明了 NAND 一种开关就能拼出全部。你第一次走完了一个"构造性证明"。

可你发现没有——这一章我们从始至终都在跟"开关、真值表、表达式"打交道，还是"机器的语言"。**接下来我们要往更抽象的地方走一步：数学世界的第一块地基，不是开关，是"把东西圈起来"。**

我圈一个圈，把"这些"放进圈里，把"那些"留在圈外——"一个东西在不在圈里"，老谟说，这就是数学的起点。而你会第一次见到"证明"的另一种、也是更常用的形态：**不是构造，而是"双包含"——你猜，"两边互相包含"能证明什么？**下一章，集合。

第 03 章 · 集合——把东西圈起来

本章从哪个问题出发：前两章你在开关和逻辑里打转。可数学世界真正的地基，不是开关，是"把东西圈起来"——**我画一个圈，把"这些"放进圈里，把"那些"留在圈外**。一个东西在不在圈里，凭什么判断？怎么给"圈"本身也造一套算术？推导到哪个结论：答案是**集合 (set)**——一套"圈"的语言。本章你会拿到：元素/属于/包含/相等的精确定义，**幂集**（集合的集合）、**笛卡尔积**（把两个圈配对成对子）这些"造新圈"的工具，并用一种新证明招式——**双包含**——亲手证明集合版的**德摩根律**。

核心认知：**集合的"相等"，不是"长得像"，是"互相包含" ($A \subseteq B$ 且 $B \subseteq A$)**；**证明两个集合相等的最常用武器，就是"双包含"**——**先把一边含进另一边，再把另一边含回来**。上一章你见了"构造性证明"，这一章你见第二种常用招式"双包含"，它会把集合的运算定律（德摩根律）钉死。

引子

周六中午，老谟在餐桌上用筷子划了个圈，圈住几粒花生米。

"你说，"他指指圈里的花生，"这个圈，是什么？"

"一堆花生？"我说。

"你说'一堆'。"老谟又把圈外的一粒花生夹进来，"我加了一粒进去，它还是那'一堆'吗？"

".....还是啊，就多一粒。"

"那我把圈里一粒夹走呢？"

"那也还是那堆花生。"

"你发现了没——"老谟把筷子放下，"你嘴里说着'一堆'，可它忽大忽小，你根本说不清它到底是什么。**'一堆花生'这个说法，是个陷阱：它既不是这粒、也不是那粒，可你硬说它是'什么'**。数学家被这种含糊折磨了几百年，最后他们决定：**不定义'这堆东西'，只定义'哪些东西在圈里'**。"

"只定义'在不在圈里'？"我盯着那个圈，"那.....圈本身就是由'在不在'决定的？"

"对。一个集合，**不靠'它长什么样'定义，靠'它的成员是谁'定义——两个集合一样，不是长得像，是'圈里的东西一模一样'**。这一章，我们把'圈'这件最朴素的工具，打磨成整个数学的地基。你先记住**'集合靠成员定义'**这句话，明天我们用它盖楼。"

我盯着那圈花生，忽然觉得"圈"这个动作朴素得过分，可"靠成员定义"这五个字，好像藏着数学所有"严格"的源头。

对话引擎

第 1 轮：集合不是“一堆东西”，是“一条规定”

老谟：“我画一个圈，圈里放 1、2、3 三个数。这个圈，我说它是‘一个集合’。但我要你严格说：这个圈‘是什么’？不许说‘是一堆数’，不许说‘看起来怎样’。”

造物主：“……它是‘由 1、2、3 组成的’？可我还是觉得我在描述它‘长什么样’……”

老谟：“你再绕。你上一章学逻辑，‘等价’只看真值表、不看长相。这里同理——**集合不看长相，看成员**。我问你：一个圈里放着 {1,2,3}，另一个圈里放着 {3,1,2}，这俩圈是一个集合还是两个？”

造物主：“……按你说的‘不看长相’，那俩圈里都是 1、2、3，成员一模一样，应该是一个集合！”

老谟：“对。集合的顺序无所谓 ($\{1,2,3\}=\{3,1,2\}$)，重复也无所谓 ($\{1,1,2\}=\{1,2\}$)——因为它不是‘一串东西’，是‘一个成员资格的规定’。你判断‘x 在不在这个集合里’，答案唯一，别的都不重要。这就是集合的**外延性 (extensionality)**：一个集合，由‘谁在圈里’完全决定，别的都不算数。”

造物主：“……那我怎么判断‘谁在圈里’？总不能一个个问。”

老谟：“这正是集合的价值——你定一条‘规则’（叫它集合的构造方式），能明确回答‘任意一个东西 x，它在不在里面’，你就定义了一个集合。比如‘所有偶数’、‘小于 5 的正整数’，都是规则。你手里现在有了两个东西：**元素 (element, 圈里的个体)** 和 **集合**，以及‘属于 ($\in$)’这个关系—— $x \in S$ 表示 x 是 S 的成员。先把这三个字钉死，下一轮我们用它描述‘圈和圈’的关系。”

知识锚点：集合、元素、属于——“圈”的语言

【概念深挖】**集合 (set)**：由明确指定的对象组成的整体。**外延性原理 (extensionality)**：一个集合由它的元素完全决定——两个集合相等，当且仅当它们含有的元素完全相同。因此集合没有顺序、没有重复 ($\{1,2,3\}=\{3,2,1\}=\{1,1,2,3\}$)。

元素 (element) 与属于 ($\in$)： $x \in S$ 表示“x 是集合 S 的元素”。这是集合论的原子关系——一切集合语言都建立在 $\in$ 之上。

集合的两种定义方式：

- **列举法 (roster)**： $\{1,2,3\}$ ，直接列元素。
- **描述法 (set-builder)**： $\{x : P(x)\}$ ，表示“所有满足性质 P 的 x”。如 $\{x : x \text{ 是偶数}\}$ 、 $\{x \in \mathbb{N} : x < 5\}$ 。

记号： $\mathbb{N}$ (自然数)、 $\mathbb{Z}$ (整数)、 $\mathbb{Q}$ (有理数)、 $\mathbb{R}$ (实数)。这些本身就是集合。

注意"集合"和"属于"是两码事： $\{1,2\}$ 是一个集合；1 是一个元素； $1 \in \{1,2\}$ 是"属于"关系。**别把"属于"和后面的"包含"搞混——属于是"元素在集合里"，包含是"集合在集合里"。**这个区分是集合论第一道坎。

【历史溯源】"集合"这么朴素的词，数学家却为它吵了一百多年。****康托尔 (Georg Cantor, 1874 起) ****创立朴素集合论，把集合定义为"我们直觉能想到的对象的总体"——可他很快发现这定义有漏洞。**1901 年罗素 (Russell) 给出"罗素悖论"：**设 $S = \{x : x \notin x\}$ (所有"不包含自己"的集合的集合)，那么 $S \in S$ 当且仅当 $S \notin S$ ——矛盾！这动摇了整个数学地基。****策梅洛-弗兰克尔 (ZFC) 公理体系 (20 世纪初) ****用严格的公理约束"能造出什么集合"，才堵住悖论。**这一章你不碰悖论，但要知道：'集合'今天的严格面貌，是从一场差点掀翻数学的危机里熬出来的。**

【工程实践】工程里"集合"是数据去重的默认模型：**数据库里的"主键集合"、代码里的 Set/哈希表**——它的价值正在于"去重、查在不在"这两个动作都是 $O(1)$ (哈希) 或 $O(\log n)$ (树)。**Python 的 `set`、Java 的 `HashSet`、SQL 的 `IN` 查询，全在做同一件事：判断'x 在不在这个圈里'。**你后面学数据库、学算法，集合是最常被当成"圈"用的数据结构。

第 2 轮：子集与相等——"双包含"这颗种子

老谟："现在让圈和圈发生关系。我说 A 是 B 的**子集** ($A \subseteq B$) ——它的意思是：**A 的每一个元素，都是 B 的元素。**你给我用'属于'这个词，把' $A \subseteq B$ '严格写出来。"

造物主：" $A \subseteq B$ 就是：对任意 x ，如果 $x \in A$ ，那么 $x \in B$ 。"

老谟："你写对了。这就是子集的精确定义——一个'**对任意 x 都成立**'的蕴含。现在关键问题来了：**两个集合 A 和 B '相等'，用子集怎么说？**你上一章学了'集合靠成员定义'，把这两句话接起来。"

造物主：".....如果 A、B 成员一模一样，那 A 里每个元素都在 B 里 ($A \subseteq B$)，B 里每个元素也都在 A 里 ($B \subseteq A$)。**所以 $A=B$ 就等价于 ' $A \subseteq B$ 且 $B \subseteq A$ '！**"

老谟："你摸到本章最锋利的一把刀了。**要证明两个集合相等，你不用把它们'摊开对比'，你只要证明两件事： $A \subseteq B$ ，和 $B \subseteq A$ 。**这个办法，数学家叫**双包含 (double inclusion)** ——证明 A 含在 B 里、B 又含在 A 里，两个方向一夹，A 和 B 就被证明相等了。**记住它，这一章我们要用它证德摩根律。**但先别急，你先用这个定义看看——如果 A 是空集呢？空集 $\subseteq$ 任何集合吗？"

造物主："空集 $\emptyset$ 是什么都没有的集合。' $A \subseteq B$ '要求'A 里每个元素都在 B 里'——空集里一个元素都没有，那这句话.....是不是'空真'地成立？没有元素需要检查，所以空集 $\subseteq$ 任何集合都成立？"

老谟："对，这就是数学里著名的**空真 (vacuously true)** ——'空集 $\subseteq$ 一切'不是约定出来的，是定义推出来的： $A \subseteq B$ 要求'对任意 $x \in A$ ， $x \in B$ '，而空集里没有 x ，所以条件无人违反，自动成立。**这个'空真'的道理很反直觉，但它让集合理论保持一致。**下一轮，我们用这些定义造'新圈'。"

知识锚点：子集、相等与空集——集合的关系

【概念深挖】

- **子集 (subset)** : $A \subseteq B \Leftrightarrow$ 对任意 $x, x \in A \Rightarrow x \in B$ 。
- **集合相等 (equality)** : $A=B \Leftrightarrow A \subseteq B$ 且 $B \subseteq A$ (**双包含**)。这是证明集合相等的标准方法。
- **真子集 (proper subset)** : $A \subset B \Leftrightarrow A \subseteq B$ 且 $A \neq B$ (B 里有 A 没有的元素)。
- **空集 (empty set) $\emptyset$** : 不含任何元素的集合。 $\emptyset \subseteq A$ **对一切 A 成立 (空真)** ; $\emptyset$ 只有一个 (外延性: 都空当然相等)。

双包含 (double inclusion) ——证明集合相等的最常用武器:

1. 先证 $A \subseteq B$: 任取 $x \in A$, 推得 $x \in B$ 。
2. 再证 $B \subseteq A$: 任取 $x \in B$, 推得 $x \in A$ 。
3. 由两者得 $A=B$ 。

为什么这是"证明"而不是"摊开看": 集合可能无限大 (如"所有偶数"), 摊不开。但"任取一个元素、推出它属于另一边"这个论证, 对无穷集合也成立——**这就是集合语言比"数一数"强大的地方。**

【历史溯源】"空真"和"空集"看似琐碎, 却是集合论精细化的结果。**空集 $\emptyset$ 的记号是 1939 年由布尔巴基学派 (Bourbaki, 法国数学家小组) 标准化推广的** (此前有各种记号)。而**"空真蕴含"这件事, 是现代逻辑学的严谨产物**——它保证了对任意集合 $S, \emptyset \subseteq S$ 恒成立, 让" $\subseteq$ "这条关系在包括空集的所有集合上都有良好行为。**没有空集做'零元', 集合理论会在边界处处别扭。**

【工程实践】工程里空集/空值的语义到处是坑: **数据库里"空表"、代码里"空集合" (`set()`)、API 返回空列表**——它们都对应数学空集, 而"空集 $\subseteq$ 一切"对应到代码就是"空集合满足所有'子集检查'的命题"。但**工程里空值也最容易踩坑**: SQL 的 `NULL` 不等于空集、不等于空字符串, 语义要分清。**判断集合是否为空, 是算法里最常用的边界检查之一。**

第 3 轮：幂集——集合的集合

老谟: "现在造新圈。给我一个集合 $A=\{1,2\}$ 。我想把'A 的所有子集'收集起来, 装进一个大圈里。这个'大圈', 你猜该长什么样?"

造物主: ".....A 的子集? A 的子集有..... $\emptyset, \{1\}, \{2\}$, 还有 $\{1,2\}$ 。四个。把它们装进一个大圈: $\{\emptyset, \{1\}, \{2\}, \{1,2\}\}$?"

老谟："对。这个'由 A 的所有子集构成的集合'，叫 A 的**幂集 (power set)**，记作 $P(A)$ 或 2^A 。**注意：幂集的元素是'集合'，不是'数'——它是'集合的集合'。**你先别急着算，先回答我： $A=\{1,2\}$ 有 2 个元素，幂集有 4 个元素；那 A 有 n 个元素时，幂集该有几个？"

造物主：".....2 个元素→4 个幂集元素。那我猜 n 个元素→ 2^n 个？每个元素'在不在子集里'是个 0/1 选择，n 个元素就是 2^n 种选法。"

老谟："你猜对了，而且你其实已经给出证明了——**每个子集由'每个元素选进不选进'决定，n 个 0/1 选择，共 2^n 个子集。**这就是幂集记作 2^A 的原因： $|P(A)|=2^{|A|}$ 。**注意空集的幂集： $P(\emptyset)=\{\emptyset\}$ ，是含一个元素的集合（含空集），不是空集！**你想想，这俩为啥不一样？"

造物主："..... $P(\emptyset)=\{\emptyset\}$ ，它的元素是空集 $\emptyset$ ，所以它有 1 个元素。空集 $\emptyset$ 本身有 0 个元素。 **$P(\emptyset)$ 和 $\emptyset$ 是两个不同的集合！ $2^0=1$ ，所以 $|P(\emptyset)|=1$ ，没错。"**

老谟："你抓得很准。 **$P(\emptyset)=\{\emptyset\} \neq \emptyset$ ——这是个经典陷阱。**下一轮，我们造最后一种'新圈'，把两个集合'配对'。"

▢ 知识锚点：幂集——集合的集合

【概念深挖】**幂集 (power set)**：集合 A 的所有子集构成的集合，记 $P(A)$ 或 2^A 。

$$P(A) = \{S : S \subseteq A\}$$

大小：若 $|A|=n$ ，则 $|P(A)|=2^n$ 。理由：构造任意子集 S，就是对每个元素做一个"选/不选" (0/1) 决策，n 个决策共 2^n 种。

- $A=\{1,2\}$ ： $P(A)=\{\emptyset, \{1\}, \{2\}, \{1,2\}\}$ ，4 个元素。
- $P(\emptyset)=\{\emptyset\}$ ，含 1 个元素（就是空集本身）， **$P(\emptyset) \neq \emptyset$ 。**

【历史溯源】幂集的记法 2^A 不是随便写的——它把"幂集的大小"和"指数"直接挂钩 ($|2^A|=2^{|A|}$)。**康托尔用幂集证明了集合论最震撼的结论之一：一个集合的幂集，永远比它本身"大"（哪怕是无穷集合）——这叫康托尔定理，它引出了"无穷也有大小之分"（可数无穷 vs 不可数无穷）。**这一章你只用到有限幂集的大小，但要知道：**幂集是通往"无穷的比较"的第一级台阶，它会在第 02 册《实数极限与连续》讨论实数为什么不可数时回来。**

【工程实践】工程里"幂集"最常见的化身是**子集枚举 / 位掩码 (bitmask)**：n 个物品的所有取法正好是 2^n 种，用一个 n 位二进制数就能表示一个子集（第 i 位 1 表示选第 i 个）。**动态规划里的"状态压缩"、穷举所有组合都用它。** $|P(A)|=2^n$ 的指数爆炸，也是工程里必须警惕的复杂度—— $n=30$ 时 $2^{30} \approx 10$ 亿，暴力枚举幂集直接崩。

第4轮：笛卡尔积——把两个圈配对成"对子"

老谟："最后一种造圈法：**笛卡尔积**。我有两个集合 $A=\{1,2\}$ ， $B=\{x,y\}$ 。我想把'A 的每个元素'和'B 的每个元素'两两配对，配成'对子'——比如 $(1,x)$ 、 $(1,y)$ 、 $(2,x)$ 、 $(2,y)$ 。把这所有对子收进一个大圈。这个圈叫什么、多大？"

造物主：".....所有'有序对'的集合？ $(1,x)$ 和 $(x,1)$ 还不一样，得讲顺序。A 有 2 个、B 有 2 个，两两配对 $2 \times 2 = 4$ 个对子。"

老谟："对。 $A \times B = \{(a,b) : a \in A, b \in B\}$ ，叫笛卡尔积。**注意它是有序对：** $(a,b) \neq (b,a)$ （除非 $a=b$ ）。大小 $|A \times B| = |A| \times |B|$ 。**你先告诉我：为什么要'有序'？如果我只要'哪些数碰过哪些数'，不用管顺序，行吗？"**

造物主：".....如果不管顺序，那 $(1,2)$ 和 $(2,1)$ 就成同一个了，可它俩在'从 A 取一个、B 取一个'的场景里是两种不同结果。**比如坐标 (经度,纬度)、(行,列)，顺序错了就差远了。"**

老谟："你抓到了'有序'的意义。**笛卡尔积是'把两个圈组合成一个对子空间'——它是坐标、是表格、是'多种选择叠加'的数学模型。**你后面学实数时，平面就是 $\mathbb{R} \times \mathbb{R}$ （所有实数对）；学函数时，函数本身就是定义域 $\times$ 值域里的一类"特殊子集"。**笛卡尔积是集合论的"乘法"，而幂集是"次方"。**你把这两个"造圈法"记住，下一轮我们干本章的大事：用双包含证德摩根律。"

知识锚点：笛卡尔积——集合的"乘法"

【概念深挖】**笛卡尔积 (Cartesian product)：**

$$A \times B = \{(a,b) : a \in A, b \in B\}$$

- **有序对 (ordered pair)：** $(a,b)=(c,d) \Leftrightarrow a=c$ 且 $b=d$ 。顺序重要。
- **大小：** $|A \times B| = |A| \cdot |B|$ 。
- **n 个集合的积：** $A_1 \times A_2 \times \dots \times A_n$ 是所有 n 元组 $(a_1, \dots, a_n)$ 的集合，大小 $|A_1| \dots |A_n|$ 。
- **例子：** $\mathbb{R} \times \mathbb{R}$ 是所有平面点的集合（坐标）； $\{0,1\}^3$ 是所有 3 位 0/1 串的集合。

【历史溯源】笛卡尔积以**笛卡尔 (René Descartes, 17 世纪)**命名，因为正是他发明了坐标几何——用"两个数的有序对"表示平面上的点。**这个看似平凡的'把集合配对'，是解析几何的数学底座：**平面 $= \mathbb{R} \times \mathbb{R}$ ，空间 $= \mathbb{R} \times \mathbb{R} \times \mathbb{R}$ 。**从集合论看，'有序对'本身也可以还原成集合**（库拉托夫斯基定义 $(a,b) = \{\{a\}, \{a,b\}\}$ ），这让"对子"也成为纯集合的构造——**数学里没有'新的东西'，只有'用集合造出来的东西'，笛卡尔积就是例证。**

【工程实践】笛卡尔积在工程里的化身是**关系型数据库的表**（每一行就是一个元组，列就是各个集合）和**多因素组合**（"所有城市 $\times$ 所有日期"的排列组合）。SQL 的 **CROSS JOIN** 就是笛卡尔积；**两层嵌套循**

环本质上在遍历一个笛卡尔积。但要注意笛卡尔积的规模爆炸：几个大集合一乘，结果数飞速增长，这是"组合爆炸"的数学来源。

第 5 轮：双包含——证明集合相等的第一把刀

老谟："现在用你手里的刀。我要你证一条集合的定律： $(A \cup B)^c = A^c \cap B^c$ ——'并集的补'等于'补的交'。你上一章见过布尔版的德摩根律 $(\neg(A \wedge B) = \neg A \vee \neg B)$ ，这一章是它在集合里的孪生兄弟。你别急着套结论，我要你用刚学的'双包含'证它。先说：怎么用双包含证这个相等？"

造物主：".....双包含是证 $A \subseteq B$ 且 $B \subseteq A$ 。所以我要证两件事： $(A \cup B)^c \subseteq A^c \cap B^c$ ，和 $A^c \cap B^c \subseteq (A \cup B)^c$ 。"

老谟："对。现在证第一方向： $(A \cup B)^c \subseteq A^c \cap B^c$ 。双包含的每一方向都是'任取一个元素，用定义一步步翻译'。你告诉我：取 $x \in (A \cup B)^c$ ，你第一步打算把它'翻译'成什么？"

造物主：".....第一步？ $x \in (A \cup B)^c$ ，意思是 x 不在 $A \cup B$ 里。而'不在并里'.....我猜就是'既不在 A 、也不在 B '——因为并集是'至少一个'嘛。这一步一翻译开，后面应该就能一路用定义推下去了。"

老谟："你抓到了双包含的灵魂：第一步永远是把' $\in$ 左边'翻译成'不属于某某'或'属于某某'，然后一路用定义换词。至于从' x 不在 $A \cup B$ '到' $x \in A^c \cap B^c$ '的具体两步、以及反方向的对称推演——完整可复写的证明在严格化走廊的定理 2 里，你翻过去对着空白纸补全。我们先把双包含这套'招式'记住：任取一个元素，用定义翻译 $\in$ ，推出它在右边；反过来再推一遍。"

造物主：".....原来如此。我不用把元素一个个数，只要'任取一个'，这个论证对无穷集合也成立？"

老谟："对。双包含是'任取一个元素'的证明，它借用了'对任意 x '这种放之四海皆准的力量，所以无穷集合也扛得住。这一章，你拿到了证明的第二种常用招式（双包含）——它的正式亮相在走廊定理 2，完整书写格式也在那里。"

知识锚点：集合运算与德摩根律（集合版）

【概念深挖】集合的三类基本运算，和逻辑三兄弟——对应：

- **并集 (union)** : $A \cup B = \{x : x \in A \text{ 或 } x \in B\}$ (对应逻辑 $\vee$) 。
- **交集 (intersection)** : $A \cap B = \{x : x \in A \text{ 且 } x \in B\}$ (对应逻辑 $\wedge$) 。
- **补集 (complement)** : $A^c = \{x : x \notin A\}$ (对应逻辑 $\neg$ ，需在某个全集 U 内取) 。

集合版德摩根律：

$$(A \cup B)^c = A^c \cap B^c, \quad (A \cap B)^c = A^c \cup B^c$$

同构关系：集合运算 ($\cup, \cap, ^c$) 与逻辑运算 ($\vee, \wedge, \neg$) 是同构的——" $x \in A$ "对应"命题 A 为真"，集合的并/交/补就是逻辑的或/与/非。所以上一章布尔德摩根的一切结论，在集合章原样成立，只是把"真/假"换成" $\in/\notin$ "。这正是集合论"统一"的力量：逻辑和集合，是同一枚硬币的两面。

【历史溯源】集合德摩根律的严格证明方法——双包含——本身就是现代集合论的标准证明范式。康托尔的集合论让"集合相等"有了可证的形式；后来的公理化 (ZFC) 让"双包含"这类证明在严格框架下无懈可击。德摩根律的历史源流与逻辑章相同 (德摩根 19 世纪)，集合版只是同一思想的另一件外衣。而"逻辑与集合同构"这个洞见，是 20 世纪数学的基石之一——它让数学家相信：逻辑推理本身可以还原成集合运算，整个数学可以"砌"在集合论这块地基上。

【工程实践】集合运算在工程里的直接应用：SQL 的 UNION (并)、INTERSECT (交)、EXCEPT (差)；代码里对两个 Set 做并/交/差；过滤器的组合 ("满足 A 且满足 B"就是交集)。德摩根律在工程里最实用的是"取反时的分解"：要筛选"不属于 A 也不属于 B"，直接写" $\text{NOT}(A \cup B)$ "或用德摩根拆成" $\text{NOT } A \text{ 且 } \text{NOT } B$ "——两个写法等价，看哪个好优化。理解双包含，也帮你理解为什么数据库的集合操作可以"任取一行验证"——跟"任取一个元素"是同一回事。

第 6 轮：落点——圈起来的，就是数学的起点

老谟："把这一章收一遍。你从'这圈花生是什么'出发，最后拿到什么？"

造物主："我拿到'圈'的严格语言：集合靠成员定义 (外延性)，属于 $\in$ 、子集 $\subseteq$ 、相等=双包含。然后我用这些造新圈：幂集 $P(A)$ (子集的集合， $|P(A)|=2^n$)、笛卡尔积 $A \times B$ (有序对的集合， $|A \times B|=|A||B|$)。最后我用双包含，亲手证了集合版德摩根律 $(A \cup B)^c = A^c \cap B^c$ 。"

老谟："你收得齐。这一章你最该记住的，不是德摩根律本身，是双包含这把刀——它是证明集合相等最常用、也最万能的武器。你上一章见了构造，这一章见了双包含。你说说，为什么数学要费劲定义集合、证明这些'显然'的定律？"

造物主：".....因为'显然'会骗人？像那个 $P(\emptyset) = \{\emptyset\} \neq \emptyset$ ，还有空集 $\subseteq$ 一切，不严格推，光靠直觉很容易想错。"

老谟："对。数学敢建那么高的楼，正因为每一层都'证过'而不是'觉得显然'。集合是地基——第 02 册的实数、第 04 册的线性代数，全都要在这块'圈'的地基上盖。你先把这块地基踩实。下一章，我们让圈和圈'发生关系'——一个圈的东西，怎么变成另一个圈的东西？那是最常见的数学对象：函数。"

严格化走廊

位置说明：对话负责"直觉"——集合为什么靠成员定义、双包含为什么万能。本走廊负责"严谨"——把集合的定律落成可复写的证明。下面是读者接触"双包含"招式的正式亮相：**每一步写清楚用哪条定义，绝不说"显然"**。

证明类型标注：本章的证明属于**直接证明**，但有两个"子招式"：

- **定理 1 (双包含判等)：**直接证明（等价性推演）；
- **定理 2 (集合德摩根律)：****直接证明·双包含**——把"集合相等"拆成"双向包含"，每一方向都是"任取一个元素、用定义逐步推"。完整的证明方法分类（直接/反证/归纳/构造）会在第 05 章系统展开，这里你认识了"双包含"这一种特别常用的直接证明形态。

定义：集合、子集、相等、幂集、笛卡尔积、集合运算

- **集合 (set)：**由明确指定的对象组成的整体；由外延性决定——两个集合相等当且仅当元素完全相同。
- **属于 ($\in$)：** $x \in S$ 表示 x 是 S 的元素。
- **子集 ($\subseteq$)：** $A \subseteq B \Leftrightarrow$ 对任意 x , $x \in A \Rightarrow x \in B$ 。
- **相等 ($=$)：** $A = B \Leftrightarrow A \subseteq B$ 且 $B \subseteq A$ 。
- **并集/交集/补集：** $A \cup B = \{x : x \in A \vee x \in B\}$; $A \cap B = \{x : x \in A \wedge x \in B\}$; $A^c = \{x \in U : x \notin A\}$ (U 为全集)。
- **幂集：** $P(A) = \{S : S \subseteq A\}$ 。
- **笛卡尔积：** $A \times B = \{(a, b) : a \in A, b \in B\}$ 。

定理 1 (双包含判等)： $A = B \Leftrightarrow (A \subseteq B \text{ 且 } B \subseteq A)$

前提： A 、 B 是任意集合。

证明类型：直接证明（等价性推演）。

证明：

- ($\Rightarrow$) 若 $A = B$ ，由集合外延性， A 、 B 元素完全相同。任取 $x \in A$ ，则 $x \in B$ （元素相同），故 $A \subseteq B$ ；任取 $x \in B$ ，则 $x \in A$ ，故 $B \subseteq A$ 。所以 $A \subseteq B$ 且 $B \subseteq A$ 。
- ($\Leftarrow$) 若 $A \subseteq B$ 且 $B \subseteq A$ ：任取 $x \in A$ ，由 $A \subseteq B$ 得 $x \in B$ ；任取 $x \in B$ ，由 $B \subseteq A$ 得 $x \in A$ 。故 A 、 B 的元素集合完全相同，由外延性 $A = B$ 。■

定理 2 (集合德摩根律) : $(A \cup B)^c = A^c \cap B^c$ 与 $(A \cap B)^c = A^c \cup B^c$

前提: A, B 是全集 U 的子集。

证明类型: 直接证明·双包含。(注: 方向 1 第 3 步"若 $x \in A$ 则矛盾"只是局部用了一句反证式推理来否定 $x \in A$ ——整体仍是直接证明的骨架; 严格的反证法招式在第 05 章才正式登场。)

证明 (以第一条为例, 第二条同法) :

- **方向 1: 证 $(A \cup B)^c \subseteq A^c \cap B^c$ 。**
 - 第 1 步: 任取 $x \in (A \cup B)^c$ 。(开始"任取一个元素"。)
 - 第 2 步: 由补集定义, $x \notin (A \cup B)$ 。
 - 第 3 步: 由并集定义, $x \notin A \cup B$ 意味着 $x \notin A$ 且 $x \notin B$ (若 $x \in A$ 或 $x \in B$, 则 $x \in A \cup B$, 矛盾)。
 - 第 4 步: 由补集定义, $x \notin A \Rightarrow x \in A^c$; $x \notin B \Rightarrow x \in B^c$ 。
 - 第 5 步: 由交集定义, $x \in A^c$ 且 $x \in B^c \Rightarrow x \in A^c \cap B^c$ 。
 - 第 6 步: 对任意 $x \in (A \cup B)^c$ 都推出 $x \in A^c \cap B^c$, 故 $(A \cup B)^c \subseteq A^c \cap B^c$ 。■
- **方向 2: 证 $A^c \cap B^c \subseteq (A \cup B)^c$ 。**
 - 第 1 步: 任取 $x \in A^c \cap B^c$ 。
 - 第 2 步: 由交集定义, $x \in A^c$ 且 $x \in B^c$ 。
 - 第 3 步: 由补集定义, $x \notin A$ 且 $x \notin B$ 。
 - 第 4 步: 由并集定义, $x \notin A$ 且 $x \notin B \Rightarrow x \notin A \cup B$ (并集要至少一个)。
 - 第 5 步: 由补集定义, $x \notin A \cup B \Rightarrow x \in (A \cup B)^c$ 。
 - 第 6 步: 对任意 $x \in A^c \cap B^c$ 都推出 $x \in (A \cup B)^c$, 故 $A^c \cap B^c \subseteq (A \cup B)^c$ 。■
- **结论:** 两个方向成立, 由定理 1 (双包含判等), $(A \cup B)^c = A^c \cap B^c$ 。■

读法: 双包含证明的"套路"是固定的——**第一步永远'任取 x 属于左边'**, 然后每一步只用定义 (补/并/交的定义) 把一个 ' $\in$ ' 翻译成 '且/或/非', 最后推出 x 属于右边。反向同理。它没有任何一步需要 '摊开整个集合', 全靠 '任意 x ' 这个普遍量化——这就是它能处理无穷集合的原因。

反例/边界: 集合与集合运算的边界情况

- $P(\emptyset) = \{\emptyset\} \neq \emptyset$: 空集的幂集是"含空集"的集合 (1 个元素), 不是空集 (0 个元素)。同理 $P(P(\emptyset)) = \{\emptyset, \{\emptyset\}\}$ (2 个元素)。**"集合的集合"和"集合本身"是两个层级, 别混。**
- **空集 $\subseteq$ 一切 (空真) :** $\emptyset \subseteq A$ 对任意 A 成立, 因为子集定义"任取 $x \in \emptyset \Rightarrow x \in A$ "中 $\emptyset$ 没有元素, 条件空真。**这不是约定, 是定义推出来的。**

- **罗素悖论的警示**："所有不包含自己的集合" ($S=\{x:x\notin x\}$) 不可定义为一个集合——否则 $S\in S \Leftrightarrow S\notin S$ 矛盾。所以**"描述法定义集合"不能太随意，公理化 (ZFC) 才安全** (本章不深究，但要知道"集合靠描述法"有边界)。
- **补集必须有全集**： A^c 只有在明确"全集 U "时才有意义 ($A^c=\{x\in U:x\notin A\}$)。不指定 U 谈补集，是没定义的。德摩根律 $(A\cup B)^c=A^c\cap B^c$ 里的补集，都默认在同一个 U 内取。

知识点总结

知识点 (本章推出的核心概念)

- **集合 (set)**：由明确对象组成的整体；**外延性**——集合由成员完全决定，无顺序、无重复。
- **属于 $\in$ / 子集 $\subseteq$ / 相等 $=$** ： $\in$ 是"元素在集合里"， $\subseteq$ 是"集合在集合里"， $A=B \Leftrightarrow A\subseteq B$ 且 $B\subseteq A$ 。
- **空集 $\emptyset$** ：不含元素； $\emptyset\subseteq$ 一切 (空真)； $P(\emptyset)=\{\emptyset\}\neq\emptyset$ 。
- **幂集 $P(A)$** ：所有子集的集合， $|P(A)|=2^{|A|}$ (每个元素做"选/不选"决策)。
- **笛卡尔积 $A\times B$** ：所有有序对的集合， $|A\times B|=|A||B|$ ，顺序重要。
- **集合运算与德摩根律**： $\cup, \cap, ^c$ 对应逻辑 $\vee, \wedge, \neg$ ，同构； $(A\cup B)^c=A^c\cap B^c$ ， $(A\cap B)^c=A^c\cup B^c$ 。
- **双包含 (double inclusion)**：证明集合相等的万能武器——证 $A\subseteq B$ 且 $B\subseteq A$ 。

 **证明格式模板：双包含 (本册第一张模板卡，后面会反复用)**

要证： $A = B$ (两个集合相等)

方向 1 ($A\subseteq B$)：

- ① 任取 $x \in A$ 。
- ② 用" $\in$ /子集/并/交/补"的定义，把 $x\in A$ 一步步"翻译" (换词)。
- ③ 翻译到 $x \in B$ ，方向 1 完成。

方向 2 ($B\subseteq A$)：对称地，任取 $x \in B$ ，用定义翻译到 $x \in A$ 。

结论：两个方向都成立 $\Rightarrow A = B$ (由"相等=双向包含")。

要点：①每一步只能"用定义换词"，不许跳步；②"任取 x "让它对无穷集合也成立；③两方向缺一不可。这跟第 04 章的"直接证明·任取输入"是同一套招式，只是换了个对象 (元素 vs 输入)。

历史结论 (本章涉及的史料)

- **康托尔 (1874 起)**：创立集合论；用幂集证明"无穷也有大小" (康托尔定理)。

- **罗素悖论 (1901)** : $S=\{x : x \notin x\}$ 导致矛盾, 动摇数学地基; ZFC 公理化 (20 世纪初) 堵住悖论。
- **笛卡尔 (17 世纪)** : 坐标几何, 平面 $= \mathbb{R} \times \mathbb{R}$, 笛卡尔积因此得名。
- **布尔巴基 (1939)** : 标准化空集 $\emptyset$ 记号。
- **德摩根 (19 世纪)** : 德摩根律; 集合版与逻辑版同构。

工程实践结论 (本章知识在真实工程里怎么用)

- **集合=去重+查在不在**: Set/哈希表、SQL 主键, $O(1)/O(\log n)$ 判断 $x \in S$ 。
- **幂集=位掩码**: n 位二进制数表示子集, 状态压缩 DP 用它; 2^n 指数爆炸要警惕。
- **笛卡尔积=数据库表/嵌套循环**: CROSS JOIN、两层循环都是笛卡尔积, 组合爆炸的数学来源。
- **德摩根律用于取反分解**: "不属于 A 也不属于 B" 可写成 $\text{NOT}(A \cup B)$ 或拆成 $\text{NOT } A \text{ 且 } \text{NOT } B$, 选好优化的那个。
- **跨册**: 集合是第 02 册实数、第 04 册线性代数的地基 (平面 $\mathbb{R} \times \mathbb{R}$ 、向量空间都是集合构造)。

计算训练场

位置说明: 对话负责"为什么", 这里负责"怎么算"。每题动笔 10 分钟以上, 先自己写再对答案。

题 1 (易→中) : 求幂集与笛卡尔积大小

题干: 老谟给出集合 $A=\{a,b,c\}$, $B=\{1,2\}$ 。(a) 写出 $P(A)$ (A 的幂集, 8 个元素)。(b) 求 $|P(A)|$ 、 $|P(P(\emptyset))|$ 、 $|A \times B|$ 、 $|P(A \times B)|$ 。(c) 指出 $P(\emptyset)$ 与 $\emptyset$ 的区别, 并写出 $P(\emptyset)$ 和 $P(P(\emptyset))$ 。

完整分步解答:

(a) $P(A)$: $A=\{a,b,c\}$, 子集按"含几个元素"枚举:

- 0 个: $\emptyset$
- 1 个: $\{a\}$ 、 $\{b\}$ 、 $\{c\}$
- 2 个: $\{a,b\}$ 、 $\{a,c\}$ 、 $\{b,c\}$
- 3 个: $\{a,b,c\}$ 共 $1+3+3+1=8$ 个。 $P(A)=\{\emptyset, \{a\}, \{b\}, \{c\}, \{a,b\}, \{a,c\}, \{b,c\}, \{a,b,c\}\}$ 。

(b) 大小:

- $|P(A)|=2^3=8$ 。
- $P(\emptyset)=\{\emptyset\}$, $|P(P(\emptyset))|=2^{|P(\emptyset)|}=2^1=2$ 。

- $|A \times B| = 3 \times 2 = 6$ 。
- $|P(A \times B)| = 2^{|A \times B|} = 2^6 = 64$ 。

(c) $P(\emptyset) = \{\emptyset\}$ (1 个元素, 就是含空集的集合); $\emptyset$ (0 个元素)。 $P(\emptyset) \neq \emptyset$ 。 $P(P(\emptyset)) = \{\emptyset, \{\emptyset\}\}$ (2 个元素)。

答: (a) 见上 8 个; (b) 8、2、6、64; (c) $P(\emptyset) = \{\emptyset\} \neq \emptyset$, $P(P(\emptyset)) = \{\emptyset, \{\emptyset\}\}$ 。

常见错误: ① 忘写空集 $\emptyset$ 和全集 A 本身 (它们都是子集); ② 把 $P(\emptyset)$ 当 $\emptyset$ (漏了一层"集合的集合"); ③ 求 $P(A \times B)$ 忘了先算 $|A \times B| = 6$, 直接把 2 当底数。

题 2 (中→难): 双包含证明 (含空集边界)

题干: 老谟让造物主用双包含证明: $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ (集合版的分配律)。(a) 用双包含法写完整证明 (方向 1: $A \cap (B \cup C) \subseteq (A \cap B) \cup (A \cap C)$)。 (b) 写方向 2。 (c) 说明这个证明在 A 为空集时是否仍然成立。

完整分步解答:

(a) 方向 1: $A \cap (B \cup C) \subseteq (A \cap B) \cup (A \cap C)$

- 第 1 步: 任取 $x \in A \cap (B \cup C)$ 。
- 第 2 步: 由交集定义, $x \in A$ 且 $x \in (B \cup C)$ 。
- 第 3 步: 由并集定义, $x \in (B \cup C)$ 意味着 $x \in B$ 或 $x \in C$ 。
- 第 4 步: 分两种情况:
 - 若 $x \in B$: 则 $x \in A$ 且 $x \in B$, 故 $x \in A \cap B$, 于是 $x \in (A \cap B) \cup (A \cap C)$ 。
 - 若 $x \in C$: 则 $x \in A$ 且 $x \in C$, 故 $x \in A \cap C$, 于是 $x \in (A \cap B) \cup (A \cap C)$ 。
- 第 5 步: 两种情况都有 $x \in (A \cap B) \cup (A \cap C)$, 故 $A \cap (B \cup C) \subseteq (A \cap B) \cup (A \cap C)$ 。 ■

(b) 方向 2: $(A \cap B) \cup (A \cap C) \subseteq A \cap (B \cup C)$

- 第 1 步: 任取 $x \in (A \cap B) \cup (A \cap C)$ 。
- 第 2 步: 由并集定义, $x \in A \cap B$ 或 $x \in A \cap C$ 。
- 第 3 步: 分两种情况:
 - 若 $x \in A \cap B$: 则 $x \in A$ 且 $x \in B$, 故 $x \in B \subseteq B \cup C$, 所以 $x \in A \cap (B \cup C)$ 。
 - 若 $x \in A \cap C$: 则 $x \in A$ 且 $x \in C$, 故 $x \in C \subseteq B \cup C$, 所以 $x \in A \cap (B \cup C)$ 。
- 第 4 步: 两种情况都有 $x \in A \cap (B \cup C)$, 故 $(A \cap B) \cup (A \cap C) \subseteq A \cap (B \cup C)$ 。 ■
- 双包含完成: $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ 。 ■

(c) 空集边界：证明依然成立。当 $A=\emptyset$ 时，左边 $A\cap(B\cup C)=\emptyset\cap\ldots=\emptyset$ ，右边 $(\emptyset\cap B)\cup(\emptyset\cap C)=\emptyset\cup\emptyset=\emptyset$ ，两边都空，相等。**在 (a) 第 4 步"任取 $x\in A\cap(B\cup C)$ "时，若 A 空则没有这样的 x，方向 1 空真成立——这正是"空真"的用处。**

答：双包含证明见上； $A=\emptyset$ 时分配律仍成立（空真）。

常见错误：① 双包含忘了证反方向（只证一边不算证明集合相等）；② 方向 1 第 4 步没分情况（ $x\in B$ 或 $x\in C$ ），直接把 $x\in A\cap B$ 当结论；③ 以为空集时证明"失效"——其实空集时用"空真"恰好成立。

题 3（中→难）：幂集大小 + 笛卡尔积的综合

题干：老谟让造物主研究一个四元素的集合 $X=\{1,2,3,4\}$ 的"所有子集"。(a) 求 $|P(X)|$ ，并说明为什么是 2^4 。(b) 用"每个元素选/不选"的视角，写出 $P(X)$ 中"恰好含 2 个元素"的子集有几个，并列出。(c) 若 $Y=\{a,b\}$ ，求 $|P(X\times Y)|$ ，并说明笛卡尔积与幂集组合时的计算顺序。

完整分步解答：

(a) $|P(X)|=2^4=16$ 。理由：构造任意子集 $S\subseteq X$ ，对 X 中每个元素（4 个）做"选/不选"决策，每元素 2 种，共 $2^4=16$ 种，每种恰好对应一个子集。

(b) 恰好 2 个元素：从 4 个元素里选 2 个，组合数 $C(4,2)=6$ 个： $\{1,2\}$ 、 $\{1,3\}$ 、 $\{1,4\}$ 、 $\{2,3\}$ 、 $\{2,4\}$ 、 $\{3,4\}$ 。

(c) $|P(X\times Y)|$ ：先算 $|X\times Y|=|X|\cdot|Y|=4\times 2=8$ ；再算 $|P(X\times Y)|=2^{|X\times Y|}=2^8=256$ 。

- **计算顺序：**必须先把笛卡尔积的大小算出来（8），再当作幂集的"元素个数"做次方（ 2^8 ）。**顺序不能反——** $P(X)\times P(Y)$ 是另一回事（那是 $16\times 4=64$ ），和 $P(X\times Y)$ （256）不同。

答：(a) 16；(b) 6 个，列在下方；(c) $|P(X\times Y)|=256$ ，先算积再算幂。

常见错误：① 把 $P(X\times Y)$ 和 $P(X)\times P(Y)$ 混淆——前者是对"8 个有序对"取幂集（256），后者是"16 个子集 $\times$ 4 个子集"的笛卡尔积（64），不是一回事；② 幂集计数漏掉"选 0 个"（空集）和"选全部"（ X 本身）。

向导便签

这一章咱们干了什么：从餐桌上一圈花生，盖起了集合论的地基。你学会"集合靠成员定义"（外延性）、属于 $\in$ 、子集 $\subseteq$ 、相等=双包含；用幂集（子集的集合）、笛卡尔积（有序对的集合）造新圈；最后用**双包含**这把新刀，亲手证了集合版德摩根律 $(A \cup B)^c = A^c \cap B^c$ 。

钉死一句话：**集合的相等，不是"长得像"，是"互相包含"**；证明两个集合相等的最常用武器，就是**双包含**——任取左边一个元素，一步步推到它属于右边，再反着来一遍。

记住： $P(\emptyset) = \{\emptyset\} \neq \emptyset$ （"集合的集合"和"集合本身"两个层级）； $\emptyset \subseteq$ 一切（空真）；**补集必须有全集才有意义**。而逻辑（第 02 章）和集合（本章）是同一枚硬币的两面——德摩根律换件衣服，还能再证一遍。

造物主手记

我曾踩的坑：

我第一坑是把"属于"和"包含"搞混—— $1 \in \{1, 2\}$ （元素在集合里）和 $\{1\} \subseteq \{1, 2\}$ （集合在集合里）是两回事，我一度以为 $1 \subseteq \{1, 2\}$ 也对。第二坑更经典：我写 $P(\emptyset) = \emptyset$ ，被老谟一句" $P(\emptyset)$ 是含空集的集合，它有几个元素？”噎住—— $P(\emptyset) = \{\emptyset\}$ ，1 个元素，不是空集！第三坑是双包含只证一边就以为证完，老谟一句"你证了 $A \subseteq B$ ，可 $B \subseteq A$ 呢？”提醒我——**集合相等必须双向都夹住**。最有用的是发现"空真"： $\emptyset \subseteq$ 一切不是约定，是定义推出来的， $A = \emptyset$ 时分配律照样成立。

顿悟时刻：

真正开窍是"集合靠成员定义，不看长相"这句话。以前我以为集合就是"一堆东西"，被老谟一句" $\{1, 2, 3\}$ 和 $\{3, 2, 1\}$ 是一个集合"点醒——**集合不是一串，是一条"成员资格的规定"**。而双包含更是让我看到证明的另一种形态：不是上一章那种"构造给你看"，而是"任取一个元素、用定义推它属于另一边"——它平凡，却对无穷集合也管用。**我突然懂了：数学敢把"无穷"当对象，靠的正是这种"对任意 x "的普遍论证，而不是把元素一个个数出来**。这一章，我摸到了"严格"为什么能撑起那么高的楼。

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A（计算，易→中）：幂集与笛卡尔积

设 $A = \{x, y\}$ ， $B = \{1, 2, 3\}$ 。(a) 写出 $P(A)$ ，求 $|P(A)|$ 。(b) 求 $|A \times B|$ 、 $|P(B)|$ 。(c) 判断下列真伪并说明理由： $\emptyset \in P(A)$ ； $\{x\} \in P(A)$ ； $x \in P(A)$ ； $\{x\} \subseteq A$ 。

完整解答： (a) $P(A) = \{\emptyset, \{x\}, \{y\}, \{x, y\}\}$, $|P(A)| = 4$ 。 (b) $|A \times B| = 2 \times 3 = 6$; $|P(B)| = 2^3 = 8$ 。 (c) ① $\emptyset \in P(A)$: 真 ($\emptyset$ 是 A 的子集, 所以在幂集里); ② $\{x\} \in P(A)$: 真 ($\{x\}$ 是 A 的子集); ③ $x \in P(A)$: 假 (x 是元素不是子集, $P(A)$ 的元素是子集 $\{x\}$, 不是 x); ④ $\{x\} \subseteq A$: 真 ($\{x\}$ 是 A 的子集)。**区分 $\in$ 与 $\subseteq$ 的关键:** $\in$ 的右边是"圈", $\subseteq$ 的两边是"圈"。

题 B (计算, 中一难): 双包含证明德摩根律

用双包含完整证明 $(A \cap B)^c = A^c \cup B^c$ (集合版德摩根律第二条)。

完整解答：方向 1: 证 $(A \cap B)^c \subseteq A^c \cup B^c$ 。任取 $x \in (A \cap B)^c$, 则 $x \notin A \cap B$ (补集定义)。若 $x \in A$ 且 $x \in B$, 则 $x \in A \cap B$ (交集定义), 矛盾。故 " $x \in A$ 且 $x \in B$ " 为假, 即 $x \notin A$ 或 $x \notin B$ (德摩根逻辑版)。 $x \notin A \Rightarrow x \in A^c$, $x \notin B \Rightarrow x \in B^c$, 故 $x \in A^c \cup B^c$ (并集定义)。所以 $(A \cap B)^c \subseteq A^c \cup B^c$ 。**方向 2: 证 $A^c \cup B^c \subseteq (A \cap B)^c$ 。**任取 $x \in A^c \cup B^c$, 则 $x \in A^c$ 或 $x \in B^c$ (并集定义), 即 $x \notin A$ 或 $x \notin B$ (补集定义)。若 $x \in A \cap B$, 则 $x \in A$ 且 $x \in B$, 与 " $x \notin A$ 或 $x \notin B$ " 矛盾, 故 $x \notin A \cap B$, 即 $x \in (A \cap B)^c$ 。所以 $A^c \cup B^c \subseteq (A \cap B)^c$ 。**双包含完成:** $(A \cap B)^c = A^c \cup B^c$ 。■

题 C (概念, 检查理解): 双包含与空真

(a) 为什么证明集合相等要用"双包含", 而不是"摊开对比"? (b) 为什么 $\emptyset \subseteq A$ 对任意集合 A 成立? 这在 A 为空集时意味着什么?

完整解答： (a) 因为集合可能无穷大 (如"所有偶数"), 摊不开、数不完; 而"任取一个元素、证明它属于另一边"的普遍论证, 对无穷集合也成立。双包含把"集合相等"翻译成两个可证的"蕴含"。(b) $\emptyset \subseteq A$ 要求"任取 $x \in \emptyset \Rightarrow x \in A$ "。 $\emptyset$ 没有元素, 所以没有 x 需要检查, 条件"空真"成立。 A 为空集时 $\emptyset \subseteq \emptyset$ 仍成立 (任取 $x \in \emptyset$ 无元素, 空真), 故空集是自己的子集。

预告

你学会了"把东西圈起来"——集合靠成员定义, 相等靠双包含, 幂集和笛卡尔积能造新圈。你第一次见识了双包含这种"任取一个元素、推来推去"的证明招式。

可你发现没有——这一章我们从始至终都在谈"圈"和"圈"的静态关系 (在不在、谁包含谁、谁和谁的积)。**数学里还有一件更基本、更动态的事: 一个圈里的东西, 怎么变成另一个圈里的东西?**

我把左边圈里的每个数, 按一条规则对应到右边圈里——"加一"、"取平方"、"反过来"。**这种'从一个到另一个'的对应, 老谟说, 是整个数学最常用的对象, 没有之一。**而它身上, 也藏着"证明"的又一种形态。下一章, 函数。

第 04 章 · 函数——从一个到另一个

本章从哪个问题出发：前几章你学会了把东西圈起来（集合），判断成不成立（逻辑），报数（二进制）。可数学里还有一件更基本、更动态的事：**一个圈里的东西，怎么变成另一个圈里的东西？** "加一"、"取平方"、"反过来"——这种"从一个到另一个"的对应，凭什么算数？推导到哪个结论：答案是**函数 (function)** ——一条"每个输入恰好对应一个输出"的规则。本章你会拿到：定义域/值域/像（函数的三件套）、单射/满射/双射（一对一/全满/一一对应），以及两种"接起来"的动作：**复合**（把函数接成链）和**反函数**（双射才能倒着走）。并用"直接证明"亲手证**复合的结合律**。

核心认知：**函数不是"会变的数"，是一条'每个输入对应恰好一个输出'的规则；复合结合律不是背的，是"两边的括号剥开都是同一个 $f(g(h(x)))$ "推出来的——直接证明的真面目，就是'两边都算到同一个东西'。** 这一章你见识证明的第三种常用形态：直接证明（对任意输入，两边算出同一结果）。

引子

周日傍晚，老谟在桌上画了两个圈。左边圈里写着 1、2、3，右边圈里写着 2、4、6。他画了三根箭头：1→2, 2→4, 3→6。

"你看这三根箭，"他说，"左边每个数，都射向右边一个数。我问你：这是一条'规则'吗？"

"是.....吧？每个数都有个去处。"

"那我把右边改一下：1→2, 2→4, 3→4。'1 射 2'、'2 射 4'、'3 也射 4'——两个数射向同一个目标。这还是一条'规则'吗？"

".....也是啊，反正每个数都射出去了。"

"那我再改：1→2, 2 不射, 3→6。'2 没箭头'。"老谟看着我，"这还算一条'规则'吗？"

"这.....2 没有去处，那就不完整了吧？"

"你摸到关键了。"老谟把三根箭擦掉，"**函数这条'规则'，有两个死规矩：第一，每个输入都必须有去处（不能有数没箭头）；第二，每个输入只能有一个去处（不能一数射两箭）。** 左边任何数不能射偏、不能射漏、不能射两箭。至于右边，两个数射向同一个地方，**可以**。你记住这两条死规矩，我们这一章就在它们的基础上盖楼。"

我盯着那两圈和几根箭，忽然觉得"规则"这个朴素的词，被这两条死规矩钉得死死的。而"射过去再射过去"，好像还能接成更长的链。

对话引擎

第 1 轮：函数的死规矩——每个输入，恰好一个输出

老谟："把两条死规矩钉成定义。一个函数 f 从集合 A 到集合 B (记 $f:A \rightarrow B$)，是给 A 里每个元素 a 指派 B 里恰好一个元素 $f(a)$ 。你要注意三个限定词：'每个' (A 里没有漏网的)、'恰好一个' (没有射两箭的)、'B 里的元素' (输出必须在 B 里，但不要求 B 每个元素都被射到)。你先把我这三句话，用例子复述一遍。"

造物主：".....比如 $f:\{1,2,3\} \rightarrow \{1,2,3,4\}$, $f(1)=2$ 、 $f(2)=2$ 、 $f(3)=4$ 。'每个'——1、2、3 都有定义；'恰好一个'——每个都只有一个输出；'B 里的'——输出 2、2、4 都在 $B=\{1,2,3,4\}$ 里。但 B 里的 1、3 没被射到，没关系。"

老谟："你复述得很清楚。那我要你反过来想——如果我用一句'规则'来描述 f ，比如 ' $f(x)=x^2$ '，那它跟'一张射箭表' (枚举每个输入对应谁) 是什么关系？"

造物主：".....' $f(x)=x^2$ ' 是一条规则，可以算出每个输入对应谁；'射箭表' 是把所有对应列出来。它俩是同一件事的两种写法？规则是'压缩版'，表是'摊开版'。"

老谟："对。同一个函数，可以用'公式规则'写，也可以用'枚举表'写——只要两者对每个输入给同样的输出，就是同一个函数。这跟上一章'集合靠成员定义'一样：函数靠'每个输入对应谁'定义，不靠'写法像不像'。你上一章刚学的集合，这一章就成了函数的'容器'——定义域和值域都是集合。你说说，这一章和上一章接上没？"

造物主："接上了！函数就是'从定义域集合到值域集合'的一种特殊对应，是建立在集合之上的。"

老谟："你接得对。函数不是凭空的东西，它是'集合之间的映射规则'。那你还记得前两章的约定观吗？——02 章'逻辑运算不是天经地义，是约定'、03 章'集合靠成员定义'。这两条死规矩也一样：'每个输入恰好一个输出'，是我们给'函数'立下的约定，不是天上掉下来的。你想，要是我们约定'一个输入可以有多个输出'，那它就不叫函数，叫'多值对应'了——正因为我们这样约定，函数才有'能倒着走'能复合'这些漂亮性质。约定是起点，推论是必然。下一轮，我们给这个'映射'配上'三件套'——它随身带的三样东西。"

知识锚点：函数、定义域、值域、像——"映射规则"三件套

【概念深挖】函数 (function) $f:A \rightarrow B$ ：对 A 中每个元素 a ，恰好指派 B 中一个元素 $f(a)$ 。

- 定义域 (domain) A ：所有允许的输入。
- 陪域/值域 (codomain) B ：所有可能的输出所在 (不要求全被射到)。
- 像/值域 (image, range) $f(A)$ ：真正被射到的那些 B 中元素， $f(A)=\{f(a): a \in A\}$ 。

注意"陪域 B"和"像 f(A)"的区别： $f:\mathbb{R}\rightarrow\mathbb{R}, f(x)=x^2$ 的陪域是 $\mathbb{R}$ ，但像只有非负数 $[0,\infty)$ 。像 $\subseteq$ 陪域，不一定相等。这个区分是单射/满射/双射（下一轮）的基础。

函数的两条死规矩（复习）：

1. 定义域里每个元素都有定义（没有漏网的）；
2. 每个输入恰好一个输出（没有射两箭的）。输出端可以"两个射一个"（允许）。

同一函数的不同写法： $f(x)=x^2$ 与"表 $x\rightarrow x^2$ "是同一个函数，只要对每个输入输出一致。**函数靠"输入输出对"定义，不靠公式长相。**

【历史溯源】"函数"这个词和概念，是数学史上演变得最久的概念之一。"function"一词由莱布尼茨（17 世纪末）提出，最初仅指"随曲线变化的量"；**欧拉（18 世纪）**把它定义为"能用解析式表达的对应"；直到 19 世纪，狄利克雷（Dirichlet）、康托尔把函数抽象成"集合之间的对应"，不再要求有解析式——**狄利克雷函数（有理数取 1、无理数取 0）就是"无解析式却合法"的著名例子。**这一抽象让函数从"能写公式的"扩展到"任意对应规则"，彻底解放了数学。

【工程实践】函数在工程里是"输入→输出"的默认模型：**一个 API（输入请求→输出响应）、一个纯函数（同样的输入必得同样输出）、一个软件模块（输入数据→输出结果）。**工程师用"定义域/值域"想清楚"这个函数接受什么、返回什么"——**类型签名（type signature）就是函数定义域的工程版。**而"每个输入恰好一个输出"这条死规矩，对应到编程就是"纯函数"（无副作用、确定性）——**这是可测试、可推理代码的基础。**

第 2 轮：单射、满射、双射——函数的三种"性格"

老谟："函数的'像'和'陪域'不一定相等，你上一轮抓到了。现在我给你三个'性格测试'，每种函数一种性格：**第一种：单射（injective）**——不同的输入，绝不能射向同一个输出（不许'两个射一个'）。**第二种：满射（surjective）**——陪域里每个元素，都至少被一个输入射到（'像=陪域'，一个不漏）。**第三种：双射（bijective）**——又单射又满射，也就是'每个输出恰被一个输入射到'。"他顿了顿，"我先问你： $f:\mathbb{R}\rightarrow\mathbb{R}, f(x)=x^2$ ，是单射吗？是满射吗？"

造物主：" $f(x)=x^2$ $f(1)=1, f(-1)=1$ ，两个不同输入射向同一个输出——**不是单射**。像只有非负数，射不到负数——**不是满射**。"

老谟："那它俩都不是。 $f:\mathbb{R}\rightarrow\mathbb{R}, f(x)=2x$ 呢？"

造物主：" $2x$不同的 x 得不同的 $2x$ ，是单射；任何实数 y ，都有 $x=y/2$ 使 $2x=y$ ，是满射。**又单射又满射，是双射！**"

老谟："对。这三种'性格'，决定了这个函数能不能'倒着走'。你先别急着想'倒着走'，先记住三件事：单射保证'每个输出至多一个来源'（能反着认）；满射保证'每个输出至少一个来源'（没漏掉）；**双射**

(单+满) 保证'每个输出恰好一个来源'——只有双射，才能倒着走成'反函数'。 下一轮，我们把函数'接起来'，然后看'接'和'倒着走'怎么配合。"

知识锚点：单射、满射、双射——函数的三种性格

【概念深挖】设 $f:A \rightarrow B$ 。

- **单射 (injective, 一对一)**：若 $a_1 \neq a_2$ ，则 $f(a_1) \neq f(a_2)$ 。等价表述：若 $f(a_1) = f(a_2)$ ，则 $a_1 = a_2$ 。**不同的输入给不同输出 (不许两个射一个)。**
- **满射 (surjective, onto)**：对每个 $b \in B$ ，都存在 $a \in A$ 使 $f(a) = b$ 。即 $f(A) = B$ (像=陪域，一个不漏)。
- **双射 (bijective, 一一对应)**：既单射又满射。**每个输出恰好被一个输入射到。**

直觉图景：

- 单射：从 A 射到 B，箭头"不打岔"，但可能没射满 B。
- 满射：射满整个 B，但可能"多对一"。
- 双射：不多不少，每支箭——对上——这是"最完美"的对应，A、B 元素个数相同。

判定方法：单射用"若 $f(a_1) = f(a_2)$ 则 $a_1 = a_2$ "; 满射用"任给 $b \in B$ 都能解出 a "。

【历史溯源】"单射/满射/双射"这套术语 (injection/surjection/bijection) 是 **20 世纪法国布尔巴基学派 (Bourbaki) **在重塑数学语言时系统化的 ("surjective" 的 sur 来自法语"在上"，指射满；"injective" 来自"射入"，指不打岔)。**但思想古已有之**：康托尔比较无穷集合大小时，正是用"能否建立双射"来判断两个无穷集合是否"一样大"——**双射是"两个集合一样大"的现代定义** (这也是第 02 册讨论实数不可数时的工具)。

【工程实践】三种性格在工程里的对应：

- **单射**：哈希函数最好接近单射 (不同键尽量不同哈希，减少碰撞)；主键要保证不重复。
- **满射**：一个函数"能不能覆盖所有可能输出" (如编码器能不能编码所有值)。
- **双射**：**可逆操作**——加解密 (加密是双射才能解密)、压缩 (无损压缩是双射)、序列化/反序列化。一个操作要"能倒着走"，它必须是双射。

第 3 轮：复合——把函数接成链

老谟："现在把函数'接起来'。我有 $f(x) = x + 1$ ， $g(x) = x^2$ 。我想先平方、再加一，也就是'先 g 后 f'。你把这个'接起来的函数'写出来。"

造物主："先平方 g ，再加一 f ——那就是 $f(g(x)) = g(x)+1 = x^2+1$ 。整个是 $x \rightarrow x^2 \rightarrow x^2+1$ 。"

老谋："你写对了。这个'先 g 后 f '的接法，叫**复合 (composition)**，记作 $f \circ g$ ： $(f \circ g)(x) = f(g(x))$ 。

注意顺序陷阱： $f \circ g$ 是"先算 g ，再算 f "——**右边的先算**。你把'加一再平方'写成 $g \circ f$ ，看看是什么。"

造物主：" $g \circ f$ 是先 f 再 g ： $g(f(x)) = (x+1)^2 = x^2+2x+1$ 。和 $f \circ g = x^2+1$ 不一样！顺序反了，结果就变。"

老谋："对。复合不满足交换律—— $f \circ g \neq g \circ f$ 一般。这个'顺序'陷阱，工程里天天踩。那我要问你一个更关键的：复合这个'接法'，有什么性质是'不管怎么括'都成立的？我给你三个函数 f 、 g 、 h ，你算算 $(f \circ g) \circ h$ 和 $f \circ (g \circ h)$ 是不是一样——先别急着算，猜。"

造物主：".....我猜一样？毕竟都是'先 h ，再 g ，再 f '，只是括号位置不同。"

老谋："你猜的方向对。这个'不管怎么括都一样'的性质，叫**结合律 (associativity)**——它比交换律更'硬'，是复合最重要的性质。可'猜'不算数，你得证。下一轮我们专门证它——你会发现，它的证明极其直接：两边都算到同一个东西。现在你先记住'复合'的定义和顺序陷阱。"

知识锚点：复合函数——把函数接成链

【概念深挖】**复合 (composition)**：设 $f:B \rightarrow C$ ， $g:A \rightarrow B$ ，则 $f \circ g:A \rightarrow C$ 定义为

$$(f \circ g)(x) = f(g(x))$$

- **顺序陷阱：** $f \circ g$ 是"先 g 后 f "（右边的先算）。一般 $f \circ g \neq g \circ f$ （不满足交换律）。
- **定义域/值域衔接：** $f \circ g$ 要求 g 的像 $\subseteq f$ 的定义域，才能接上（ g 的输出要被 f 接收）。
- **结合律：** $(f \circ g) \circ h = f \circ (g \circ h)$ ——不管怎么加括号，结果一样（严格证明在走廊）。

为什么复合是"最常用的接法"：几乎一切"流水线"都是复合——先做 A 再做 B ；神经网络的层间传递就是一层层复合；函数式编程里 `f ∘ g` 表示"把 g 的输出喂给 f "。

【历史溯源】复合函数的思想贯穿整个微积分——**链式法则 (chain rule)** 就是"复合函数的导数"，它是微积分最重要的公式之一。"复合"的抽象表述 ($f \circ g$) 在现代数学里成为标准记号，因为函数复合的结构（结合律、单位元恒等函数）让它天然构成一个"半群/群"结构——这会在更高级的代数里发光。对你现在来说，抓住'复合=先右后左的接链'和'结合律成立'就够。

【工程实践】复合在工程里遍地都是：**流水线处理**（数据经过 `transform`→`validate`→`persist` 一道道工序）、**函数式编程**（`compose` / `pipe` 把多个函数接成一个）、**管道命令**（Unix 的 `a | b | c`）。'复合顺序影响结果'是最常见的 **bug 来源**——同一个功能，先过滤再映射和先映射再过滤，可能差之千里。所以工程师写复合链时，必须明确每一步的顺序。

第 4 轮：结合律——“不管怎么括，结果一样”

老谟：“现在证结合律。我要证 $(f \circ g) \circ h = f \circ (g \circ h)$ 。你说，怎么证两个函数相等？你上一章学了集合相等的双包含——那函数相等呢？”

造物主：“.....函数靠‘每个输入对应谁’定义。那两个函数相等，应该是‘对每个输入 x ，输出都一样’？”

老谟：“对。函数相等 = 对每个输入 x ，两个函数的输出相等（这其实就是‘函数靠输入输出对定义’的直接推论）。所以证结合律，就是任取一个输入 x ，证 $(f \circ g) \circ h$ 和 $f \circ (g \circ h)$ 在 x 处输出相等。你先别急着算——你告诉我，证‘两个复合在 x 处相等’，你打算从哪头下手？”

造物主：“.....两个复合都套着括号。我猜，得先把括号一层层剥开—— $(f \circ g) \circ h$ 作用在 x 上，就是先算 $h(x)$ ，再把 $(f \circ g)$ 作用上去； $f \circ (g \circ h)$ 呢，就是先算 $(g \circ h)(x)$ ，再让 f 作用。两边都往‘最里面的 x ’剥，最后应该剥到同一个东西？”

老谟：“你摸到方法了：把括号一层层剥开，看两边是不是剥出同一个结果。至于‘左边具体怎么剥到 $f(g(h(x)))$ 、右边怎么剥到同一个’——这两步的完整推演在严格化走廊的定理 1 里，你翻过去对着空白纸补全。我们先把这一章的‘招式’认下来：直接证明，最常见形态就是‘任取一个输入/元素，把两边都算到同一个值’。你上一章的双包含，其实是直接证明的一种特例；这一章的复合结合律，是直接证明的另一副面孔。下一轮，我们把函数‘倒着走’——而你会看到，‘能倒着走’这件事，跟这一章的性格测试（单射/满射/双射）严丝合缝地扣上了。”

知识锚点：结合律与直接证明

【概念深挖】复合的结合律 (associativity)：

$$(f \circ g) \circ h = f \circ (g \circ h)$$

证明思路（完整版在走廊）：任取输入 x ，

- 左边： $(f \circ g) \circ h$ 在 $x = (f \circ g)(h(x)) = f(g(h(x)))$ 。
- 右边： $f \circ (g \circ h)$ 在 $x = f((g \circ h)(x)) = f(g(h(x)))$ 。
- 两边都 $= f(g(h(x)))$ ，故对任意 x 相等 $\Rightarrow$ 两函数相等。

为什么结合律重要：因为有了它，写复合时可以放心不加括号——“ $f \circ g \circ h$ ”不会产生歧义（怎么括都一样）。对照：复合不满足交换律（ $f \circ g \neq g \circ f$ ），但满足结合律（ $(f \circ g) \circ h = f \circ (g \circ h)$ ）——交换律管‘顺序能不能换’，结合律管‘括号能不能去’，是两码事。

直接证明 (direct proof)：从已知假设出发，按规则一步步推出结论，中途不绕弯（不反证、不归纳、不构造）。复合结合律是最典型的直接证明——任取 x ，两边各算一步，发现到同一处。

【历史溯源】结合律是代数结构的基石性质：半群、群、环、域这些结构都要求运算满足结合律（加法结合、乘法结合）。复合函数满足结合律，这一件事让“函数的复合”成为一个合法的代数运算——它是“变换群”（如置换群）的理论基础。欧拉、伽罗瓦等人在研究对称性时，正是靠“复合满足结合律”才建立起群论。你现在证的这个结合律，是整座群论大厦的第一块砖。

【工程实践】结合律在工程里的价值是“可重构”：因为复合可结合，一段处理链可以重新分组而不改变结果——并行化、缓存、断点重排都依赖“结合律允许重新括”。例如矩阵乘法满足结合律（虽不满足交换律），所以可以分组并行计算。工程师判断“这个运算能不能安全重组”时，问的第一句话往往是“它满足结合律吗”。

第 5 轮：反函数——双射才有资格“倒着走”

老谟：“现在把函数‘倒着走’。你说 $f(x)=2x$ 是双射。那我要一个‘逆操作’，能把我输出的 $2x$ 变回 x ——也就是‘除以 2’。你猜，这个‘逆操作’叫什么、什么时候才存在？”

造物主：“叫……反函数？可它什么时候存在？得能‘无歧义地倒着走’才行——得知道每个输出是从哪个输入来的。”

老谟：“对，你抓到要害了。反函数要存在，每个输出必须‘恰好对应一个输入’——也就是双射。你看：如果不是单射，两个输入射向同一输出，你倒着走时不知道回到哪个；如果不是满射，有的输出根本没被射到，你的逆操作在那些输出上‘无定义’。只有双射（单+满），每个输出恰有一个来源，反函数才存在。你先把这个条件记住，下一轮严格化走廊里，我们把它证成‘反函数存在的充要条件’。”

造物主：“……那我明白了。反函数不是‘随便倒着走’，是只有双射才配有的‘特权’。 $f(x)=2x$ 能倒着走成 $f^{-1}(y)=y/2$ ，因为它是双射； $f(x)=x^2$ 倒着走会‘两个输入抢一个输出’，所以没有（真正的）反函数。”

老谟：“你总结得准。你现在手里有四个东西：函数的定义、三种性格（单/满/双射）、复合（接链）、反函数（倒着走）。下一轮，我们把这一章收个尾，看看这四个东西怎么拧成一股绳。”

知识锚点：反函数——双射的特权

【概念深挖】反函数 (inverse function)：设 $f:A\rightarrow B$ 。若存在 $g:B\rightarrow A$ 使 $f\circ g = \text{id}_B$ 且 $g\circ f = \text{id}_A$ (id 是恒等函数 $\text{id}(x)=x$)，则 g 是 f 的反函数，记 f^{-1} 。

反函数存在的充要条件： f 是双射。

- 单射：保证每个输出至多一个来源（能唯一地倒着走）。
- 满射：保证每个输出至少一个来源（倒着走时没有“空缺”）。

- **双射**：每个输出恰有一个来源，反函数定义无歧义。

记法： $f^{-1}(y)$ 表示“满足 $f(x)=y$ 的那个唯一 x ”。注意 f^{-1} 不是 $1/f$ （倒数）——**这是最常见的符号坑**。

【历史溯源】反函数思想源于“解方程”——**给定 y ，求满足 $f(x)=y$ 的 x** 。这是数学最古老的动作之一。但“反函数何时存在”这个严格问题，要等函数概念完全抽象化（19 世纪）之后才彻底说清——狄利克雷等把函数从“解析式”解放后，反函数的存在性就清晰指向“双射”这个条件。**单射/满射/双射与反函数的关系，是现代数学里“可逆性”的精确表达。**

【工程实践】反函数=可逆操作：**加密算法必须是双射（加密后能解密）；无损压缩必须是双射（压缩后能还原）；数据库主键映射、UID 编解码都要“能倒着走”**。工程里判断一个操作能不能逆，就问它**是不是双射**——比如“MD5 是单射吗？不是（有碰撞），所以不能当加密，只能当指纹”。而 $f^{-1} \neq 1/f$ **这个坑**：工程里“逆函数”和“倒数”是两个概念，别混。

第 6 轮：落点——从“一个到另一个”，数学最常用的一句话

老谟：“把这一章收一遍。你从‘两圈几根箭’出发，最后拿到什么？”

造物主：“我拿到函数的定义——每个输入恰好一个输出的规则。它的三件套：定义域、陪域、像。三种性格：单射（不打岔）、满射（射满）、双射（一一对应）。两种接法：复合（先右后左接链，满足结合律）、反函数（只有双射才有资格倒着走）。”

老谟：“你收得齐。这一章你见了第三种证明招式——直接证明（任取输入 x ，两边算到同一结果）。你说说，函数这一章，跟前面几章怎么连？”

造物主：“……函数是‘集合之间的映射’，所以它长在集合（第 03 章）上；它的‘每个输入恰好一个输出’、‘等价=行为一致’，又带着逻辑（第 02 章）的味道。**函数把集合、逻辑都串起来了。**”

老谟：“你串得对。前四章——数、逻辑、集合、函数——这就是你整个数学地基的‘四件套’。下一章，我们要把这些地基真正‘用起来’：你前几章一直在‘看证明’（构造、双包含、直接），下一章开始，你要系统学会‘自己证明’——反证法、数学归纳法、构造性证明，全都正式登场。**而函数这个‘从一个到另一个’，会一路陪着你，直到微积分的极限。你准备好了吗？**”

严格化走廊

位置说明：对话负责"直觉"——函数为什么是规则、结合律为什么成立。本走廊负责"严谨"——把函数的结构落成可复写的定义与证明。下面是读者接触"直接证明"的正式亮相：**每一步写清楚用哪条定义，绝不说"显然"**。

证明类型标注：本章的证明属于**直接证明**，且给出两个常用子形态：

- **定理 1 (复合结合律)：**直接证明——任取输入 x ，把两边各算一步，发现到同一处 $f(g(h(x)))$ 。
- **定理 2 (反函数存在充要条件)：**直接证明（双向）—— $(\Rightarrow)$ 反函数存在推出双射， $(\Leftarrow)$ 双射构造出反函数。完整的证明方法分类（直接/反证/归纳/构造）会在第 05 章系统展开，这里你认识了"直接证明"的两种常用做法。

定义：函数、定义域、陪域、像、单射、满射、双射、复合、反函数

- **函数 $f:A \rightarrow B$ ：**给 A 中每个元素 a 指派 B 中恰好一个元素 $f(a)$ 。
- **定义域 A 、陪域 B 、像 $f(A)$ ：** $f(A) = \{f(a) : a \in A\} \subseteq B$ 。
- **单射：** $a_1 \neq a_2 \Rightarrow f(a_1) \neq f(a_2)$ （等价： $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$ ）。
- **满射：**对每个 $b \in B$ ， $\exists a \in A$ 使 $f(a) = b$ （即 $f(A) = B$ ）。
- **双射：**既单射又满射。
- **复合：** $f:B \rightarrow C, g:A \rightarrow B$ ，则 $(f \circ g)(x) = f(g(x))$ 。
- **反函数：** $g:B \rightarrow A$ 满足 $f \circ g = \text{id}_B$ 且 $g \circ f = \text{id}_A$ ，则 $g = f^{-1}$ 。
- **恒等函数：** $\text{id}(x) = x$ 。

定理 1 (复合结合律)： $(f \circ g) \circ h = f \circ (g \circ h)$

前提： $h:A \rightarrow B, g:B \rightarrow C, f:C \rightarrow D$ （各函数定义域/陪域能衔接）。

证明类型：直接证明。

证明：任取 $x \in A$ 。

- 第 1 步（左边剥开）： $((f \circ g) \circ h)(x) = (f \circ g)(h(x))$ （复合定义， $(f \circ g) \circ h$ 先作用于 x 得 $h(x)$ ，再作用于 $h(x)$ ）。
- 第 2 步： $= f(g(h(x)))$ （复合定义，对 $h(x)$ 先 g 后 f ）。
- 第 3 步（右边剥开）： $(f \circ (g \circ h))(x) = f((g \circ h)(x))$ （复合定义， f 后接 $(g \circ h)$ ）。

- 第 4 步：= $f(g(h(x)))$ (复合定义, $(g \circ h)(x) = g(h(x))$) 。
- 第 5 步：对任意 $x \in A$, 左边=右边= $f(g(h(x)))$ 。由"函数相等=每个输入输出相等", 故 $(f \circ g) \circ h = f \circ (g \circ h)$ 。■

读法：这个证明的全部内容，就是"把括号一层层剥开，两边剥出同一个 $f(g(h(x)))$ "。它没有用任何高深工具，只用了'复合的定义'两遍，加上'任取 x '。这就是直接证明的真面目：把'要证的'翻译成'定义的展开'，两边殊途同归。你可以对着空白纸，自己把 x 换成任意值，验证两边确实相等。

定理 2 (反函数存在的充要条件)： $f:A \rightarrow B$ 有反函数 $\Leftrightarrow f$ 是双射

前提： $f:A \rightarrow B$ 是函数。

证明类型：直接证明 (双向) 。

证明：

- **($\Rightarrow$) 若 f 有反函数 g , 则 f 是双射。**
 - 单射：设 $f(a_1) = f(a_2)$ 。两边同时用 g 作用： $g(f(a_1)) = g(f(a_2))$ 。由 $g \circ f = \text{id}_A$, 得 $a_1 = \text{id}_A(a_1) = a_2$ 。故单射。
 - 满射：任取 $b \in B$ 。令 $a = g(b) \in A$ 。则 $f(a) = f(g(b)) = \text{id}_B(b) = b$ (由 $f \circ g = \text{id}_B$)。故对每个 b 都存在 a 使 $f(a) = b$, 满射。
 - 所以 f 双射。■
- **($\Leftarrow$) 若 f 是双射, 则 f 有反函数。**
 - 构造 g ：对每个 $b \in B$, 因 f 满射, 存在 $a \in A$ 使 $f(a) = b$; 因 f 单射, 这样的 a 唯一。定义 $g(b) = \text{这个唯一的 } a$ 。
 - 则 $f \circ g = \text{id}_B$ ($g(b)$ 是 f 的原像, $f(g(b)) = b$) ; $g \circ f = \text{id}_A$ (对 a , $g(f(a))$ 是 $f(a)$ 的原像, 由单射性等于 a) 。
 - 故 g 是 f 的反函数。■

读法：($\Rightarrow$) 方向用的是"反函数的定义"——把 $f \circ g = \text{id}$ 、 $g \circ f = \text{id}$ 这两个等式当成工具，推出单射和满射。($\Leftarrow$) 方向是"构造"——明确给出 $g(b)$ 的取法 (利用满射保证存在、单射保证唯一)。**整个定理说明：**'能倒着走'不是玄学，就是'每个输出恰有一个来源' (双射)。这也是"可逆性"的精确数学表达。

反例/边界：函数与复合的边界情况

- **不是双射就没有反函数**： $f: \mathbb{R} \rightarrow \mathbb{R}, f(x)=x^2$ 不是单射 ($f(1)=f(-1)$)，也不是满射（射不到负数），故无反函数。若强行定义“倒着走”，会得到“一个输出对应两个输入”——那不是函数（违反“每个输入恰好一个输出”）。
- **复合顺序不可交换**： $f(x)=x+1, g(x)=x^2$ ，则 $f \circ g = x^2+1$ ， $g \circ f = (x+1)^2 = x^2+2x+1$ ，不相等。**复合满足结合律但不满足交换律——两个性质是两回事。**
- **复合的定义域衔接**： $f \circ g$ 需要 g 的像 $\subseteq f$ 的定义域。若 g 的输出超出 f 的定义域，复合无定义。
例： $g: \mathbb{R} \rightarrow \mathbb{R}$ 输出负值，而 $f: \mathbb{R}^+ \rightarrow \mathbb{R}$ 只接受正数，则 $f \circ g$ 在“ g 输出负数”处无定义。
- **f^{-1} 不是 $1/f$** ： $f^{-1}(y)$ 表示“满足 $f(x)=y$ 的 x ”（反函数），绝不是 $1/f(y)$ （倒数）。**这是最常见的符号陷阱**： x^2 的反函数是 $\sqrt{x}$ （正负要看定义域），不是 $1/x^2$ 。
- **狄利克雷函数（无解析式也是函数）**： $f(x)=1$ (x 有理数)、 $f(x)=0$ (x 无理数)——它是合法函数，却没有解析式。**函数靠“输入输出对应”定义，不靠“有没有公式”**。这提醒你：别把“函数=公式”当信条。

知识点总结

知识点（本章推出的核心概念）

- **函数 $f: A \rightarrow B$** ：给 A 中每个元素恰好指派 B 中一个元素的规则。两条死规矩：每个输入都有定义、恰好一个输出。
- **定义域/陪域/像**： $A, B, f(A)=\{f(a)\}$ 。**像 $\subseteq$ 陪域，不一定相等。**
- **单射/满射/双射**：单射=不同输入给不同输出；满射=射满整个陪域（像=陪域）；双射=单+满=每个输出恰一个来源。
- **复合 $f \circ g$** ： $(f \circ g)(x)=f(g(x))$ ，先右后左；**满足结合律，不满足交换律。**
- **反函数**：只有双射才有反函数（每个输出恰一个来源才能无歧义倒着走）； **$f^{-1} \neq 1/f$ 。**
- **函数相等**：每个输入输出相等（靠“输入输出对”定义，不靠公式长相）。

历史结论（本章涉及的史料）

- **"function"由莱布尼茨（17 世纪末）提出**，最初仅指“随曲线变化的量”。
- **欧拉（18 世纪）**：定义函数为“能用解析式表达的对应”。
- **狄利克雷、康托尔（19 世纪）**：把函数抽象成“集合之间的对应”，不再要求解析式（狄利克雷函数是反例）。
- **布尔巴基（20 世纪）**：系统化单射/满射/双射术语。

- **复合结合律** → **群论**：结合律是半群/群/环/域的基石，复合满足结合律是变换群的基础（欧拉、伽罗瓦）。

工程实践结论（本章知识在真实工程里怎么用）

- **函数=API/纯函数**：类型签名是定义域的工程版；"每个输入恰好一个输出"对应"纯函数"（无副作用、确定性）。
- **双射=可逆操作**：加解密、无损压缩、编解码都要双射；MD5 不是单射（有碰撞）只能当指纹不能当加密。
- **复合=流水线/管道**：Unix 管道 `a|b|c`、函数式 `compose / pipe`、神经网络的层间传递；**顺序影响结果，结合律允许重新分组**（并行化/缓存依赖结合律）。
- **跨册**：函数是第 02 册极限、第 03 册微积分的核心对象（极限是函数行为，在第 02 册；链式法则复合的导数，在第 03 册）。

计算训练场

位置说明：对话负责"为什么"，这里负责"怎么算"。每题动笔 10 分钟以上，先自己写再对答案。

题 1（易→中）：判定单射/满射/双射并求反函数

题干：老谟给三个函数，请判定每个是单射、满射还是双射，并对双射的求反函数：(a) $f: \mathbb{R} \rightarrow \mathbb{R}$, $f(x)=3x-2$ 。(b) $g: \mathbb{R} \rightarrow \mathbb{R}$, $g(x)=x^2+1$ 。(c) $h: \mathbb{R} \rightarrow \mathbb{R}$, $h(x)=e^x$ （指数函数，像为正数）。

完整分步解答：

(a) $f(x)=3x-2$ ：

- 单射：设 $f(a)=f(b)$ ，即 $3a-2=3b-2$ ，得 $3a=3b$, $a=b$ 。 **单射**。
- 满射：任给 $y \in \mathbb{R}$ ，解 $3x-2=y$ ，得 $x=(y+2)/3 \in \mathbb{R}$, $f(x)=y$ 。 **满射**。
- **双射**。反函数： $f^{-1}(y)=(y+2)/3$ ，即 $f^{-1}(x)=(x+2)/3$ 。验证： $f(f^{-1}(x))=3 \cdot (x+2)/3 - 2 = x \checkmark$ ； $f^{-1}(f(x))=(3x-2+2)/3=x \checkmark$ 。

(b) $g(x)=x^2+1$ ：

- 单射： $g(1)=2$, $g(-1)=2$ ，两个不同输入同输出 → **不是单射**。
- 满射：像只有 ≥ 1 的数 ($x^2+1 \geq 1$)，射不到 0 等 → **不是满射**。
- **既非单射也非满射，不是双射，无反函数**。

(c) $h(x)=e^x$ (注意陪域是 $\mathbb{R}$) :

- 单射: $e^a=e^b \Rightarrow a=b$ (指数函数严格增) 。 **单射**。
- 满射 (对 $\mathbb{R}$) : 像只有正数 $(0,\infty)$, 射不到 0 和负数 $\rightarrow$ **不是满射** (对 $\mathbb{R}$ 而言) 。
- **单射但非满射** (对陪域 $\mathbb{R}$) 。 **注意**: 若把陪域改成 $(0,\infty)$ (正实数) , 则 $h:\mathbb{R}\rightarrow(0,\infty)$ 是双射, 反函数 $h^{-1}(y)=\ln y$ 。

答: (a) 双射, $f^{-1}=(x+2)/3$; (b) 非单射非满射; (c) 对 $\mathbb{R}$ 单射非满射; 对 $(0,\infty)$ 是双射, 反函数 $\ln$ 。

常见错误: ① 判单射只用"看不出两个输入同输出"就下结论——必须写"设 $f(a)=f(b)$ 推出 $a=b$ "才算证; ② 判满射忘检查"像是否=陪域", 如 (c) 对 $\mathbb{R}$ 不是满射但若陪域改小就是; ③ 求反函数后不验证 $f\circ f^{-1}=\text{id}$ 。

题 2 (中 $\rightarrow$ 难) : 复合顺序影响 + 结合律验证

题干: 设 $f(x)=x+1$, $g(x)=2x$, $h(x)=x^2$ 。(a) 求 $f\circ g$ 、 $g\circ f$, 并说明为何不同。(b) 求 $(f\circ g)\circ h$ 与 $f\circ(g\circ h)$, 验证结合律成立。(c) 求 $f\circ(g\circ h)$ 在 $x=3$ 处的值。

完整分步解答:

(a) **复合顺序**:

- $f\circ g$: 先 g 后 f , $f(g(x))=f(2x)=2x+1$ 。
- $g\circ f$: 先 f 后 g , $g(f(x))=g(x+1)=2(x+1)=2x+2$ 。
- $2x+1 \neq 2x+2$, 顺序不同结果不同 $\rightarrow$ 复合不满足交换律。

(b) **结合律验证**:

- $(f\circ g)\circ h$: 先算 $f\circ g=2x+1$, 再复合 h : $(f\circ g)(h(x))=(f\circ g)(x^2)=2x^2+1$ 。
- $f\circ(g\circ h)$: 先算 $g\circ h$, $g(h(x))=g(x^2)=2x^2$, 再复合 f : $f(g(h(x)))=f(2x^2)=2x^2+1$ 。
- **两者都 = $2x^2+1$ $\checkmark$ 结合律成立。**

(c) $x=3$: $f\circ(g\circ h)$ 在 3 = $2\cdot 3^2+1=2\cdot 9+1=19$ 。逐步: $h(3)=9$, $g(9)=18$, $f(18)=19 \checkmark$ 。

答: (a) $f\circ g=2x+1$, $g\circ f=2x+2$; (b) 都是 $2x^2+1$; (c) 19。

常见错误: ① 复合方向搞反 (以为 $f\circ g$ 是先 f 后 g) —— **$f\circ g$ 是"先 g 后 f ", 右边的先算**; ② 结合律验证只算一边就下结论, 必须两边都算到同一式; ③ 忘了先算内层 ($h(x)$ 要先求出来才能复合) 。

题 3 (中 $\rightarrow$ 难) : 反函数构造 + 分段函数判定

题干: 老谟给了一个分段函数

$$f(x) = \begin{cases} 2x & x \geq 0 \\ x-1 & x < 0 \end{cases}$$

定义域、陪域都是 $\mathbb{R}$ 。(a) 判定 f 是否单射。(b) 判定 f 是否满射。(c) 若是双射, 构造反函数 f^{-1} (分段形式), 并验证 $f^{-1}(f(x))=x$ 。

完整分步解答:

(a) 单射: 设 $f(a)=f(b)$ 。分情况:

- 若 $a, b \geq 0$: $2a=2b \Rightarrow a=b$ 。
- 若 $a, b < 0$: $(a-1)=(b-1) \Rightarrow a=b$ 。
- 若 $a \geq 0$ 、 $b < 0$: $f(a)=2a \geq 0$, $f(b)=b-1 < 0$ (因为 $b < 0 \Rightarrow b-1 < -1 < 0$), 两者不可能相等 (一非负一负)。
- 同理 $a < 0$ 、 $b \geq 0$ 不可能相等。故所有情况 $a=b$, **单射**。

(b) 满射: 逐支算像, 再合并看是否覆盖整个 $\mathbb{R}$ 。

- 第 1 步: $x \geq 0$ 支 $f(x)=2x$, 随 x 扫过 $[0, \infty)$, 像为 $[0, \infty)$ 。
- 第 2 步: $x < 0$ 支 $f(x)=x-1$, 随 x 扫过 $(-\infty, 0)$, $x-1$ 扫过 $(-\infty, -1)$, 像为 $(-\infty, -1)$ 。
- 第 3 步: 两支像合并: $([0, \infty)) \cup ((-\infty, -1)) = \mathbb{R} \setminus [-1, 0)$ 。
- 第 4 步: 合并结果**不等于** $\mathbb{R}$ ——像漏掉了 $[-1, 0)$ 这个区间 (例如 $y=-0.5$ 落在缝里, 任何 x 都给不出 $f(x)=-0.5$)。故 **f 不是满射**。

(c) 因不是满射, f 不是双射, 无反函数。

答: (a) 单射; (b) **不是满射** (像漏掉 $[-1, 0)$) ; (c) 无全定义反函数。

常见错误: ① 判满射时只检查"每一支都能解出 x ", 却忘了**两支的像要拼起来覆盖整个陪域**——本例两支像 $[0, \infty)$ 与 $(-\infty, -1)$ 之间漏了 $[-1, 0)$ 的缝; ② 用"取 $x=y+1$ "时没检查 x 是否落在对应分支 ($y=-0.5$ 时 $y+1=0.5$ 落错分支); ③ 分段函数判定必须**逐支算像、再合并**, 不能只看某一支。

向导便签

这一章咱们干了什么：从“两圈几根箭”，盖起了函数的四件套。你拿到函数的定义（每个输入恰好一个输出）、三件套（定义域/陪域/像）、三种性格（单射/满射/双射）、两种接法（复合满足结合律、反函数只在双射时存在）。你亲手用“直接证明”（任取输入 x ，两边算到同一个结果）证了复合结合律。

钉死一句话：**函数不是“会变的数”，是一条“每个输入恰好一个输出的规则”；复合满足结合律、不满足交换律；只有双射才有资格倒着走成反函数。**而证明结合律的真面目，是“两边的括号剥开，都是 $f(g(h(x)))$ ”——直接证明就是“殊途同归”。

记住： **$f^{-1} \neq 1/f$** （反函数不是倒数）；**像 $\subseteq$ 陪域**（满射要求像=陪域）；**复合先右后左**（ $f \circ g$ 先算 g ）。而“函数靠输入输出对定义，不靠公式长相”——狄利克雷函数没有公式，也是合法函数。

造物主手记

我曾踩的坑：

我第一坑是复合方向搞反——我以为 $f \circ g$ 是先 f 后 g ，被老谟一句“ $f \circ g$ 右边的先算，是 g 先进来”纠正，算出来 $2x+1$ 和 $2x+2$ 差之千里。第二坑是以为“看不出两个输入同输出”就判单射——老谟让我必须写“设 $f(a)=f(b)$ 推出 $a=b$ ”才算证，光靠眼力会漏（比如分段函数）。第三坑最狠：求分段函数反函数时，我用“取 $x=y+1$ ”却忘了检查 x 落没落对分支，结果 $y=-0.5$ 时算错——**两支的像拼起来漏了 $[-1,0)$ 那道缝，整个函数根本不是满射，谈不上反函数。**第四坑是符号：我差点把 f^{-1} 当成 $1/f$ ，被老谟提醒反函数是“倒着走”，不是“取倒数”。

顿悟时刻：

真正开窍是复合结合律的证明——我原以为它需要什么高深招数，结果就是把 $(f \circ g) \circ h$ 和 $f \circ (g \circ h)$ 的括号一层层剥开，两边都剥出 $f(g(h(x)))$ 。**“殊途同归”四个字，就是直接证明的全部秘密。**而“反函数只在双射时存在”这句，让我看懂了“可逆性”的本质：不是“想倒着走就能倒着走”，是“每个输出恰有一个来源”才有资格。**这一章我最大的收获，是函数这一根线把前面数、逻辑、集合全串起来了——它是第一件真正“活的”数学对象。**

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A（计算，易→中）：判定 + 反函数

判定下列函数是否双射，双射的求反函数：(a) $f: \mathbb{R} \rightarrow \mathbb{R}$, $f(x)=x^3$ 。(b) $g: \mathbb{R} \rightarrow \mathbb{R}$, $g(x)=x^2-4$ 。(c) $h: \mathbb{R} \rightarrow \mathbb{R}$, $h(x)=2^x$ 。

完整解答：(a) 单射： $a^3=b^3 \Rightarrow a=b$ ✓；满射：任给 y , $x=\sqrt[3]{y}$ 使 $x^3=y$ ✓。**双射**, $f^{-1}(y)=\sqrt[3]{y}$ 。(b) $g(x)=x^2-4$: $g(2)=0=g(-2)$, 非单射；像为 $[-4, \infty)$, 非满射。**非双射**, 无反函数。(c) 单射： $2^a=2^b \Rightarrow a=b$ ✓；对 $\mathbb{R}$ 满射？ 2^x 像为 $(0, \infty)$, 射不到 0/负数 → **对 $\mathbb{R}$ 非满射**。若陪域改 $(0, \infty)$ 则双射，反函数 $\log_2(y)$ 。

题 B (计算, 中一难) : 复合与结合律

设 $f(x)=x-1$, $g(x)=3x$, $h(x)=x^2$ 。(a) 求 $f \circ g$ 与 $g \circ f$ 。(b) 验证 $(f \circ g) \circ h = f \circ (g \circ h)$, 并求它在 $x=2$ 的值。

完整解答：(a) $f \circ g$: 先 g 后 f , $f(g(x))=f(3x)=3x-1$ 。 $g \circ f$: $g(f(x))=g(x-1)=3(x-1)=3x-3$ 。**不相等** (不满足交换律)。(b) 结合律: 先算 $f \circ g=3x-1$, 再复合 h : $(f \circ g)(h(x))=(f \circ g)(x^2)=3x^2-1$ 。另一边: $g \circ h=g(x^2)=3x^2$, $f \circ (g \circ h)=f(3x^2)=3x^2-1$ 。**两者都 = $3x^2-1$** ✓。 $x=2$ 时: $3 \cdot 4-1=11$ 。逐步: $h(2)=4$, $g(4)=12$, $f(12)=11$ ✓。

题 C (概念, 检查理解) : 函数与证明招式

(a) 为什么 $f(x)=x^2$ 没有 (全定义域 $\mathbb{R}$ 上的) 反函数? (b) "函数相等"的严格定义是什么? 为什么它让复合结合律能被直接证明? (c) 直接证明与构造性证明 (第 02 章) 的区别是什么?

完整解答：(a) 因为 x^2 在 $\mathbb{R}$ 上既非单射 ($f(1)=f(-1)$) 也非满射 (射不到负数), 每个输出不满足"恰一个来源", 倒着走会有歧义或无定义, 故无反函数。(b) 函数相等=对每个输入 x , 两函数输出相等 (靠"输入输出对"定义)。正因如此, 证结合律只需任取一个 x , 把两边都算到 $f(g(h(x)))$ ——对任意 x 成立即两函数相等。(c) 直接证明: 从已知出发按规则推到结论 (如结合律两边殊途同归); 构造性证明: 证明"存在"靠"造出来" (如 NAND 完备性给流水线)。直接证明多用于"等式/性质" (两边算到同一结果), 构造性证明多用于"存在性" (真造出来)。

预告

你终于集齐了数学地基的"四件套"——数、逻辑、集合、函数。你手里也有了三种证明的雏形: 构造 (NAND 完备)、双包含 (德摩根)、直接 (复合结合律)。

可你发现没有——前四章你一直在"看证明": 老谟证给你看, 你在旁边跟着走。**你还没系统地学会"自己拿起笔证明"**。而数学真正的门槛, 不是看懂别人的证明, 是自己写得出证明。

下一章, 我们要真正开始"证明方法"了——**反证法 (先假设结论不对, 推出矛盾)**、**数学归纳法 (从第 1 步推到无穷远)**、**构造性证明 (真造出来)**。你前四章种下的"凭什么", 下一章要长成完整的招法。而从"看证明"到"写证明", 是老谟说的、这条路上最陡的一道坡。你准备好了吗?

第 05 章 · 证明方法入门——"显然"在数学里不算数

本章从哪个问题出发：第 01 章埋下一颗种子——你两次问"凭什么"：最低位是奇偶凭什么一直对？取反加一和 256 减它凭什么一样？第 04 章结尾，老谟又逼了你一次。**"显然"这两个字，在数学里到底算不算数？**推导到哪个结论：数学回答"凭什么"的方式，是**证明（proof）**——而且证明是有"招式"的。本章把最常用的四招系统展开：**直接证明**（从定义一路推过去）、**反证法**（假设它错，推出矛盾）、**数学归纳法**（多米诺骨牌）、**构造性证明**（直接造一个给你看）。你学会的不只是"看懂证明"，更是"自己写证明"的格式模板。

核心认知：**证明不是魔法，是把"你懂的东西"按规则连起来，直到结论自己冒出来。**而"按规则连"有四种常用招式：直接证明是"正面推"，反证法是"反着假设"，数学归纳法是"推倒第一块骨牌"，构造性证明是"亲手造出来"。**这套招式，是全系列后续 6 册严格化走廊的通用工具箱——**你会在第 02 册的极限证明、第 03 册的微积分证明里一次次遇到它们。

引子

第 01 章那个"凭什么"一直在我脑子里转。我盯着桌上那排二进制数，从 0 写到 100，越写越不安：我说"最低位是奇偶"，可我数到 100 都对，凭啥？我总不能数到无穷个吧？

我把这个问题甩给老谟。他正把一叠扑克牌按 2、4、6、8.....的偶数排成一排，一张一张往前推。

"你说'最低位是奇偶'，"老谟头也不抬，"那我问你——你要怎么让一个人相信，**这不是你数了 100 个碰巧都对，而是它对从 1 到无穷的每一个数都成立？**"

我手里转着的笔停在半空。".....我怎么让他相信？我总不能真把无穷个数都验一遍吧？"

老谟终于抬眼，把那叠偶数牌一推——第一张倒下，砸倒第二张，第二张倒下砸倒第三张，整排牌哗啦啦全倒了。"你验不完无穷个。可你发现没有——**我只要证明两件事：第一，第一张会倒；第二，只要前面那张倒了，后面那张一定跟着倒。**这两件事都证完了，你还担心最后一张吗？"

我盯着那排倒下的牌，愣是没说话。他推的是牌，可我心里"最低位是奇偶"那个不安，好像正被这一推，松开了什么。

对话引擎

第 1 轮：凭什么相信"对所有的数都成立"——证明要回答的，正是这个问题

老谟："你第 01 章问我'最低位是奇偶凭什么一直对'。我那时候说留到第 05 章。现在到了。你先告诉我——你说'它对所有的数都成立'，可你只验过前 100 个。那你凭什么敢说'所有'？"

造物主：".....我不敢。我心里发虚。我没法把无穷多个数都验一遍。可我要是不验完，又凭什么说它对？我卡在这了。"

老谏："你卡住的地方，正是'证明'要回答的问题。你说'它一直对'，这是个断言（claim）；要让这个断言成立，你得拿出一个'说服任何人、包括最刁钻的怀疑者'的理由——这个理由，就叫证明（proof）。证明不是'我多验了几个'，不是'我感觉它成立'，是'我从已经承认的规则出发，一步一步推到结论'。你先记住这句话：证明不是魔法，是把'你懂的东西'按规则连起来。那具体怎么连？有几种连法？我们一招一招来。"

知识锚点：什么是证明

【概念深挖】证明（proof）：从一个公认的出发点（定义、公理、已证的定理）出发，通过逻辑推理，一步一步推出结论，且每一步都有明确的依据。它的标准是"可复写"——任何人照着你的每一步，都能重走一遍、得到同样的结论。

一个提醒（衔接前面的约定观）：这些"出发点"（定义、公理）本身，也是人为约定——就像第 01 章"比特无意义、意义来自约定"、第 06 章你会看到"1 不是素数"是约定。证明的职责，是在约定之上推出必然；但'约定是什么'，本身是数学里可以商量、也可以改的。约定是地基，证明是盖楼——两者不是一回事，但缺一不可。

证明的类型（本章主线）：证明是有"招式"的，最常用四种——

- **直接证明（direct proof）：**从定义出发，顺着逻辑一步步推到结论（正面推）。
- **反证法（proof by contradiction）：**先假设结论为假，推出一个矛盾，于是假设不成立，结论为真（反着假设）。
- **数学归纳法（mathematical induction）：**先证"第一张骨牌会倒"，再证"只要前面倒了，后面一定倒"，于是整排都倒（多米诺）。
- **构造性证明（constructive proof）：**把要证明的"存在的东西"直接造出来给人看（亲手造）。

如何识别证明类型（本册特别强调）：读一个证明时，先问"它用的是什么招式"——是正面推（直接）、反着假设推矛盾（反证）、推骨牌（归纳）、还是造东西（构造）？看懂"它用的是哪一招"，比看懂"它证了什么"更进一步。这正是第 01 章严格化走廊里埋下的那个概念的系统化展开。

第 2 轮：直接证明——“从定义出发，正面推过去”

老谋：“先看最简单的一招：**直接证明**。它的样子是：从一个你已经承认的**定义**出发，一步一步顺着逻辑推，最后推到你想要的结论。它不绕弯、不假设它假，就是‘正面走’。**我给你个热身：命题‘两个偶数的和是偶数’。你定义里已经有‘偶数’了——你打算怎么证？**”

造物主：“……偶数就是能被 2 整除的数，也就是能写成 $2 \times$ 某个整数的数。那设一个偶数是 $2a$ ，另一个是 $2b$ ，加起来是 $2a+2b=2(a+b)$ ，还是 2 乘以一个整数，所以是偶数。”

老谋：“一字不差，这就是直接证明。你从‘偶数’的**定义**出发，把两个偶数各写成一个整数的 2 倍，相加，再提出 2，发现结果还是‘能写成 2 倍整数’——于是命题成立。**注意你没假设任何反的东西，就是顺着定义推。**它是最朴素、也最常用的一招。那如果碰上‘直接推推不动’的命题呢？比如你想证明‘某个数不是有理数’——你打算从定义怎么‘正面’证它不是？”

造物主：“……从定义正面证‘它不是’？‘有理数’的定义是能写成两个整数之比。要我正面证‘它写不成任何整数之比’，我岂不是要把所有整数之比都检查一遍？这推不动啊。”

老谋：“你撞到直接证明的天花板了。当‘直接证明’推不动——比如要证明一个‘否定型’或‘不可能型’的命题——就该换招。下一招，反证法。”

知识锚点：直接证明——“正面走”与它的适用

【概念深挖】**直接证明 (direct proof)**：要证“若 P 则 Q ”，从 P 出发，利用定义与已证结论，通过逻辑推导直接得到 Q 。**适用场景**：命题的结论能直接从前提的定义推出来，尤其是“正面型”命题（“……是……”“……能被……”）。**书写格式模板**：

命题：若 P ，则 Q 。

证明：

设 P 成立。

（依据某定义）把 P 展开，得到……（如：偶数 = $2 \times$ 整数）

（依据某代数规则）变形，得到……

（依据某定义）这正说明 Q 成立。

■

易错点：① 别把“举几个例子”当证明——例证只能说明“这几个成立”，证明必须覆盖所有情况；② 别跳跃“显然”——每一步都要说清用了哪条定义/规则。

第 3 轮：反证法——“先假设它是假的，看会不会爆炸”

老谟：“直接证明卡住了。现在换招：**反证法 (proof by contradiction)**。它的逻辑很怪但极强：你要证‘结论 Q 成立’，先反着假设‘Q 不成立’，然后从‘Q 不成立’出发推啊推，一直推到推出一个矛盾——矛盾一出现，说明‘Q 不成立’这个假设是错的，于是 Q 成立。你想想，为什么‘推出矛盾’就说明‘Q 不成立’是错的？”

造物主：“.....因为推理过程只要每一步都符合逻辑，那么‘假设成立’就必然导致‘结论成立’。可我推出来的结论和已知事实撞车了——那就只能是那个假设一开始就错了。假设错了，它的反面（Q 成立）就对了。”

老谟：“你这理解就到位了。反证法的灵魂，是‘若假设为真则必出矛盾’——矛盾不会凭空出现，它是假设埋下的雷。什么时候用反证法？正是你刚才卡住的那种——要证‘某个东西不存在’、‘某个数不是某类数’，正面推不动时。比如我告诉你：‘ $\sqrt{2}$ 是个无理数’——你想证明它，正面从定义证‘它写不成任何整数之比’，推不动；可要是反着假设‘ $\sqrt{2}$ 能写成整数之比’，推一推，会不会爆炸？这个完整证明我们放到严格化走廊，你对着空白纸复写。现在你只要先抓住反证法的直觉——‘它要是真的能写成整数之比，我们把它设出来，看看会撞出什么矛盾’。”

造物主：“.....所以反证法是‘把敌人请进自己家，然后关上门看它怎么露馅’。”

老谟：“这比喻不赖。把‘假设它为假’请进来，关上门推，推到矛盾就赢了。这一招，你在第 02 册学极限的时候会反复见到——很多‘不存在/不可能’的极限结论，都是反证法收的尾。”

知识锚点：反证法——“反着假设，推出矛盾”

【概念深挖】**反证法 (proof by contradiction)**：要证结论 Q，先假设 $\neg Q$ (Q 不成立)，从 $\neg Q$ 出发经逻辑推导得到一个**矛盾**（与已知公理、定理、定义，或与 $\neg Q$ 自身冲突），于是 $\neg Q$ 不可能成立，故 Q 成立。**书写格式模板：**

命题：Q 成立。
证明（反证法）：
 假设 $\neg Q$ ，即 Q 不成立。
 （依据某定义/定理）从 $\neg Q$ 推出.....
 （依据某规则）又推出.....
 这与.....矛盾（矛盾点要明确指出）。
 故假设 $\neg Q$ 不成立，所以 Q 成立。■

关键点：① 假设必须是“Q 不成立”的确切否定，别假设成别的；② 推到最后必须**明确说出矛盾在哪**（“这与.....矛盾”），不能含糊；③ 矛盾可以来自任何已承认的真命题，甚至来自假设自身（如“最小分数”的无穷下降）。**易错点：**把反证法用在不该用的地方（直接证明推得动就别绕），以及推不出矛盾就硬说矛盾。

第 4 轮：数学归纳法——“推倒第一张骨牌”

老谟：“现在回到你引子里的不安——‘最低位是奇偶凭什么对所有的数都成立’。我要证明一个关于‘每一个正整数’的命题，可我又不能验无穷多个。你刚才看我推那排扑克牌，**你发现没有，我只需要验/证两件事就够了——哪两件？**”

造物主：“……第一件事，最开头那张骨牌会倒；第二件事，只要前面那张倒了，后面那张就一定跟着倒。这两件都成立，那不管这排牌多长，都会倒。”

老谟：“就是这个。**数学归纳法 (mathematical induction)** 就是把这两件事变成严格证明的招式：**第一，证明当 $n=1$ 时命题成立 (基础步，也叫归纳基)；第二，证明‘若命题对 n 成立，则命题对 $n+1$ 也成立’ (递推步，也叫归纳步)。**这两步都证完，由数学归纳法，命题对一切正整数 n 成立。**你第 01 章那个‘最低位是奇偶’，就是靠这一招，从‘验到 100’升级成‘对一切正整数都成立’。**”

造物主：“……可我还有个疑问：递推步里，我‘假设命题对 n 成立’，这不是耍赖吗？我还没证 n 成立，凭什么敢假设它？”

老谟：“问得好，这恰恰是归纳法最容易被误会的地方。**递推步里的‘假设’不是‘我相信它成立’，而是‘如果它成立，我能推出下一步’——它是一个‘条件句’的证明，不是对 n 的结论。**真正保证‘一切 n 成立’的，是基础步那一个点，加上递推步这个‘链条’：基础步给了 $n=1$ ，递推步从 1 推到 2，再从 2 推到 3……像骨牌一样一路倒下去。**所以两件事缺一不可：只证基础步，盖不住后面的；只证递推步，连第一张都没倒。**”

▢ 知识锚点：数学归纳法——“两件事，倒一整排牌”

【概念深挖】**数学归纳法 (mathematical induction)：**要证关于一切正整数 n 的命题 $P(n)$ ，只需证：

- **基础步 (base case)：** $P(1)$ 成立；
- **递推步 (inductive step)：**对任意 $n \geq 1$ ，若 $P(n)$ 成立，则 $P(n+1)$ 成立。

由这两步， $P(n)$ 对一切正整数 n 成立。**书写格式模板：**

命题：对一切正整数 n , $P(n)$ 成立。

证明（数学归纳法）：

[基础步] 当 $n=1$ 时, $P(1)$: (直接算出两边相等)。

[递推步] 假设 n 时 $P(n)$ 成立 (归纳假设), 即.....。

要证 $P(n+1)$:

(把 $P(n+1)$ 的式子拆开, 用归纳假设替换其中 $P(n)$ 的部分, 再代数化简)

.....得到 $P(n+1)$ 也成立。

由数学归纳法, $P(n)$ 对一切正整数 n 成立。■

关键点：① 归纳假设是“若 $P(n)$ 成立”的条件句，不是“我断言 $P(n)$ 成立”；② 递推步必须真的用到归纳假设，把它代进去化简——不能绕开；③ 代数化简是递推步的重头，别跳步。**易错点：**忘证基础步（只证链条不给起点）；递推步没用归纳假设（那等于没证递推）。**跨册衔接：**第 02 册的实数极限、第 03 册的微积分里，大量“对一切正整数成立”的不等式都是归纳法收的尾。

第 5 轮：构造性证明——“不用吵，我直接造给你看”

老谟：“前三招都是‘推理’。最后一招路子不同——**构造性证明 (constructive proof)**。它要证的往往是‘存在某个东西满足某条件’。这种命题，你不用去吵它存在不存在，你直接把它造出来，往桌上一放：‘喏，这就是。’你想想，第 01 章你有没有干过‘造东西’的事？”

造物主：“.....有！补码！我说‘把 -5 藏进正数区间’，把 -5 定义成 $256-5=251$ ，往那一放， $5+(-5)$ 就归零了。我没证明‘负数有表示法’，我直接造了一个。”

老谟：“正是。补码那套，就是个构造性证明——‘存在一种把负数编码成加法的办法’，你直接用 2^n-x 把它造出来了。构造性证明的优点是**强**：它不止证明‘存在’，还告诉你‘它长什么样’，你随时能照着重造。它的代价是：它只保证‘你造的这个存在’，不保证‘只有这一个’。很多时候，存在性用构造证，唯一性还得另证。具体什么时候用构造性证明，给你个直觉：凡是‘找个例子’‘造个编码’‘拼个集合’这种‘东西存在’的命题，都可以先想能不能直接造。”

知识锚点：构造性证明——“直接造给你看”

【概念深挖】构造性证明 (constructive proof)：要证“存在某物满足某性质”，直接构造出这样一个具体对象，并验证它确实满足性质，从而证明存在。它是四种招式中最“实”的一种——不止证明存在，还给出了例子/构造，读者可照着重造。**书写格式模板：**

命题：存在一个满足性质 x 的对象。

证明（构造性）：

取对象 =（给出具体构造）。

验证：它满足性质 x （逐条检查，依据定义）。

故存在这样的对象。■

易错点：① 构造的对象必须真满足全部性质，不能“看着像”；② 构造性证明只证“存在一个”，若题目要“唯一”，还得另加“唯一性”论证。**跨册衔接：**第 06 章证明实战、以及后续册的“存在性定理”，构造性证明是常用到的“造出来”的招。

第 6 轮：落点——四招齐了，证明的“工具箱”开箱

老谟：“四招都见过了。你现在把它们排成一排，告诉我：**什么时候用哪招？**”

造物主：“.....直接证明是‘正面推得动’就用它——最省事。**反证法**是‘要证不存在/不是某类’、正面推不动时用——反着假设推矛盾。**数学归纳法**是‘对一切正整数都成立’、又没法逐个验时用——推骨牌。**构造性证明**是‘存在某个东西’时用——直接造一个。”

老谟：“这就不赖了。可你注意一件事——**这四招不是互相独立的**：一个复杂证明，常常是几招叠着用（比如反证法里面套直接证明，构造性证明后面跟唯一性证明）。**你现在拿到的不是四个孤立技巧，是一个可组合的工具箱。**下一章，我们把这个工具箱开进‘数论’——用真刀真枪的命题练一遍，尤其是那个两千年都没人敢正面去证的命题：**素数有无穷多个，拿什么证明？**先把这四招的格式模板抄好，那是你以后写证明的脚手架。”

严格化走廊

位置说明：对话负责“直觉”——四招各在什么时候用。本走廊负责“严谨”——把四招落成精确定义，并给出两个**完整可复写**的示范证明（反证法、归纳法各一），每个证明都标注“证明类型 + 依据”。这是全系列“证明工具库”的第一批正式入库件。

定义：四种证明方法（精确表述）

直接证明 (direct proof)：要证“若 P 则 Q ”，从 P 出发，仅用定义、公理、已证定理与逻辑规则，直接推出 Q 。不假设 $\neg Q$ 。

反证法 (proof by contradiction)：要证 Q ，假设 $\neg Q$ 成立，经逻辑推导得到一个矛盾（与公理/定理/定义/ $\neg Q$ 自身冲突），则 $\neg Q$ 不成立，故 Q 成立。

数学归纳法 (mathematical induction)：对关于正整数 n 的命题 $P(n)$ ，若 (1) $P(1)$ 成立，且 (2) 对任意 $n \geq 1$ ， $P(n) \Rightarrow P(n+1)$ ，则 $P(n)$ 对一切正整数 n 成立。

构造性证明 (constructive proof)：要证 " $\exists x$, x 满足性质 X "，给出一个具体的 x 并验证其满足 X ，从而证明存在性。

定理 1 ($\sqrt{2}$ 是无理数) ——证明类型：反证法

前提： $\sqrt{2}$ 定义为满足 $x^2=2$ 的正实数；**引理 (平方为偶则自身为偶)**：若整数 a 满足 a^2 是偶数，则 a 是偶数。（此引理在第 06 章会作为整除性命题正式证明并推广，此处先用最简论证：若 a 是奇数， $a=2k+1$ ，则 $a^2=4k^2+4k+1$ 是奇数，与 " a^2 为偶" 矛盾。）

结论： $\sqrt{2}$ 不是有理数（即无理数）。

证明 (反证法)：

1. **假设** $\sqrt{2}$ 是有理数（即假设 $\neg Q$ ： $\sqrt{2}$ 不是无理数）。
2. 由有理数定义，存在整数 a 、 b ， $b \neq 0$ ，且 **a 、 b 互质**（最大公约数为 1；若可约则先约分，这是欧几里得式论证的常见化简，依据：任何分数可约到最简），使得 $\sqrt{2} = a/b$ 。
3. 两边平方： $2 = a^2/b^2$ ，即 **$a^2 = 2b^2$** 。（依据：等式两边同乘 b^2 ）
4. 由 $a^2=2b^2$ ， a^2 是 2 的倍数，故 a^2 是偶数。
5. 由引理 (a^2 偶 $\Rightarrow a$ 偶)， a 是偶数，故存在整数 k 使 **$a = 2k$** 。
6. 代回 $a^2=2b^2$ ： $4k^2 = 2b^2$ ，两边同除以 2： **$2k^2 = b^2$** ，故 b^2 是偶数。
7. 再由引理 (b^2 偶 $\Rightarrow b$ 偶)， b 是偶数，故存在整数 m 使 **$b = 2m$** 。
8. 于是 a 和 b 都能被 2 整除，即 a 、 b 不互质，**与第 2 步的 " a 、 b 互质" 矛盾**。
9. 故假设不成立， $\sqrt{2}$ 不是有理数。 **$\sqrt{2}$ 是无理数。■**

依据标注：步 1 用反证法设定（假设 $\neg Q$ ）；步 2 用有理数定义 + "约分到最简" 的事实；步 3 用代数等式运算；步 4 用 " $2b^2$ 是偶数" 的整除直观；步 5、7 用引理 (a^2 偶 $\Rightarrow a$ 偶)；步 6 用代数化简；步 8 用 "互质" 定义引出矛盾。**读法**：整个证明的发动机，是引理 "平方为偶则自身为偶"——它让 " a^2 偶" 一步步传导成 " a 偶、 b 偶"，最终撞上 "互质" 这块礁石。**这就是反证法的标准样貌：假设 $\rightarrow$ 传导 $\rightarrow$ 撞礁 $\rightarrow$ 矛盾 $\rightarrow$ 假设错。**

定理 2 (前 n 个自然数之和) ——证明类型：数学归纳法

前提： n 为正整数；求和 $1+2+\dots+n$ 定义为前 n 个正整数相加。

结论：对一切正整数 n ， $1+2+\dots+n = n(n+1)/2$ 。

证明（数学归纳法）：

1. **[基础步]** 当 $n=1$ 时：左边 = 1；右边 = $1 \times (1+1)/2 = 1$ 。左右相等， $P(1)$ 成立。
2. **[递推步·归纳假设]** 假设当 n 时成立（归纳假设 IH），即 $1+2+\dots+n = n(n+1)/2$ 。
3. **[递推步·目标]** 要证当 $n+1$ 时成立： $1+2+\dots+n+(n+1) = (n+1)(n+2)/2$ 。
4. 把左边拆开，前 n 项用 IH 替换： $1+2+\dots+n+(n+1) = [1+2+\dots+n] + (n+1) = n(n+1)/2 + (n+1)$ 。
5. 提取公因式 $(n+1)$ ： $n(n+1)/2 + (n+1) = (n+1)(n/2 + 1) = (n+1)(n+2)/2$ 。
6. 右边正是 $(n+1)(n+2)/2$ ，故 $P(n+1)$ 成立。
7. 由数学归纳法（基础步 + 递推步）， **$1+2+\dots+n = n(n+1)/2$ 对一切正整数 n 成立。** ■

依据标注：步 1 直接计算验证基础步；步 2 用归纳假设（条件句）；步 4 用“归纳假设替换前 n 项”；步 5 用代数提取公因式与通分；步 7 用数学归纳法原理。**读法：**递推步的关键是“把 $P(n+1)$ 拆开，露出 $P(n)$ 那块，用假设替换”。**代数化简（提取公因式、通分）是归纳法递推步的主战场——这块练熟，归纳法就过了一半。**

反例/边界：证明方法失效与使用红线

1. **归纳法缺一不可：**只证基础步（ $n=1$ 成立），不证递推步——盖不住 $n=2, 3\dots$ ；只证递推步，不证基础步——骨牌没第一张可倒。**两个都要，缺一不可。** 两个干净反例：
 - **只证基础步不够：**命题 $P(n)$ ：“ $n = 1$ ”（对所有正整数 n ）。基础步 $P(1)$ ： $1=1$ 成立 ✓；但递推步 $P(n) \Rightarrow P(n+1)$ 失败（ $n=1$ 时推不出 $2=1$ ）。只验基础步就以为证完，错。
 - **只证递推步不够：**命题 $Q(n)$ ：“ $n = n+1$ ”。递推步 $Q(n) \Rightarrow Q(n+1)$ ：“若 $n=n+1$ 则 $n+1=n+2$ ”——假设条件下两边加 1 确实成立 ✓（递推步空洞成立）；但基础步 $Q(1)$ ： $1=2$ 是假的。递推步再“成立”也没用，因为没有第一张骨牌。**两个反例一对照，归纳法“两步缺一不可”就钉死了。**
2. **反证法的“矛盾”必须真矛盾：**反证法最后一步要明确指出“这与 X 矛盾”， X 必须是已承认的真命题。若只是“与我直觉不符”，那不是矛盾，反证失败。**易错点：**推了半天发现没矛盾，就硬说“所以假设错”——不行，必须真撞到礁石。
3. **直接证明别用例证冒充：**“我验了 $n=1, 2, 3$ 都对”不是证明，只是举例。证明必须覆盖所有情况（通常用符号代代数）。例： $1+2+3+4=10$ 、 $n(n+1)/2$ 在 $n=4$ 时 $=10$ ，但“举几个例子”永远不能证明“对所有 n ”。
4. **构造性证明的“唯一性”是另一码事：**构造性证明只保证“存在一个”，不保证“只有这一个”。若要“唯一”，须另行证明唯一性——别把“我造了一个”当成“只有这一个”。

知识点总结

知识点（本章推出的核心概念）

- **证明 (proof)**：从定义/公理/已证定理出发，经逻辑推理逐步推出结论，每一步有依据，标准是"可复写"。
- **直接证明**：从定义正面推到结论。适用"正面型"命题；别用例证冒充。
- **反证法**：假设 $\neg Q$ ，推矛盾，故 Q 成立。适用"不存在/不是某类"的否定型命题；矛盾必须真矛盾。
- **数学归纳法**：基础步 + 递推步，推倒整排骨牌。适用"对一切正整数成立"的命题；两要素缺一不可。
- **构造性证明**：直接造出满足性质的对象。适用"存在某物"命题；只证存在，唯一性另证。
- **证明类型识别**：读证明先问"用的哪一招"，这比看懂结论更进一步。

历史结论（本章涉及的史料）

- **$\sqrt{2}$ 是无理数**：这是数学史上第一个著名的无理数证明，源自古希腊毕达哥拉斯学派，据传发现者希帕索斯因挑战"万物皆数"（一切皆可表示为有理数）而遭排斥。它开了"反证法"的先河——用矛盾证明一个数不属于有理数。
- **数学归纳法的现代形式**：由帕斯卡（Pascal，17 世纪）在组合论中系统使用，后由数学归纳法（"戴德金—皮亚诺"公理化）成为算术公理的推论。它把"无穷多个验证"压缩成"两步"的合法性，是证明理论的一次飞跃。
- **构造性证明传统**：欧几里得《几何原本》大量用"构造"（作图）证明存在性，这是构造性证明的源头；现代数学则用更抽象的"非构造性"存在证明（如选择公理），本章只取最直观的构造派。

工程实践结论（本章知识在真实工程里怎么用）

- **证明类型识别 = 调试直觉**：工程师看一段算法证明，先问"这是直接/反证/归纳/构造"——直接证明对应"推到底"、归纳法对应"递归正确性"（递归算法常归纳法证）、反证法对应"排除不可能"、构造法对应"给出具体算法"。识别招式，等于拿到调试地图。
- **归纳法 = 递归算法正确性的证明工具**：程序里的递归（如 `factorial(n)`、归并排序）正确性，几乎都是数学归纳法证的——基础步=递归出口，递推步=递归调用成立则本层成立。
- **构造性证明 = "能跑起来"的保证**：要证明"存在一种算法解决某问题"，构造性证明直接给算法——"造得出来"比"存在"更工程。

计算训练场

位置说明：本章对话负责“四招什么时候用”，这里负责“完整书写证明”。下面三题重点是**把证明写成可复写的完整格式**，而非纯算数——先自己写，再对照标准格式。**难度递进：易→中→难。**

题 1（易→中）：数学归纳法完整书写——证明前 n 项求和公式

题干：用数学归纳法**完整书写**（含基础步、递推步、每一步依据）证明：对一切正整数 n ， $1+2+\dots+n = n(n+1)/2$ 。

完整分步解答（可复写模板）：

命题：对一切正整数 n ， $P(n): 1+2+\dots+n = n(n+1)/2$ 。

[基础步] 第 1 步：当 $n=1$ 时，左边 $= 1$ ；右边 $= 1 \times (1+1)/2 = 1$ 。第 2 步：左右相等，故 $P(1)$ 成立。

[递推步] 第 3 步：设归纳假设 IH ：对某个 n ， $1+2+\dots+n = n(n+1)/2$ 。第 4 步：要证 $P(n+1)$ ： $1+2+\dots+n+(n+1) = (n+1)(n+2)/2$ 。第 5 步：把左边前 n 项用 IH 替换： $1+2+\dots+n+(n+1) = n(n+1)/2 + (n+1)$ 。第 6 步：提取公因式 $(n+1)$ ： $= (n+1)(n/2 + 1) = (n+1)((n+2)/2) = (n+1)(n+2)/2$ 。第 7 步：右边正是 $(n+1)(n+2)/2$ ，故 $P(n+1)$ 成立。

[收束] 第 8 步：由基础步 + 递推步，数学归纳法得 $1+2+\dots+n = n(n+1)/2$ 对一切正整数 n 成立。■

答：证明如上， $n=1$ 时 $1=1\checkmark$ ，递推步化简得 $(n+1)(n+2)/2\checkmark$ 。

常见错误：① 忘写基础步（只证递推步，骨牌没起点）；② 递推步没把归纳假设代进去，而是重新从 n 算——那等于没证递推；③ 提取公因式时漏了 $(n+1)$ （把 $n/2+1$ 写成 $n/2$ 就错，必须通分成 $(n+2)/2$ ）。

题 2（中→难）：归纳法更难的代数化简——平方和公式

题干：用数学归纳法**完整书写**证明：对一切正整数 n ， $1^2+2^2+\dots+n^2 = n(n+1)(2n+1)/6$ 。

完整分步解答（可复写模板）：

命题： $P(n): 1^2+2^2+\dots+n^2 = n(n+1)(2n+1)/6$ 。

[基础步] 第 1 步： $n=1$ 时，左边 $= 1^2=1$ ；右边 $= 1 \times 2 \times 3/6=1$ 。 $P(1)$ 成立。

[递推步] 第 2 步：设 $IH: 1^2+\dots+n^2 = n(n+1)(2n+1)/6$ 。第 3 步：要证 $P(n+1): 1^2+\dots+n^2+(n+1)^2 = (n+1)(n+2)(2n+3)/6$ 。第 4 步：前 n 项用 IH 替换：左边 $= n(n+1)(2n+1)/6 + (n+1)^2$ 。第 5 步：提取公因式 $(n+1)$ ： $= (n+1)[n(2n+1)/6 + (n+1)]$ 。第 6 步：括号内通分到 $/6$ ： $= (n+1)[(2n^2+n)/6 + (6n+6)/6] = (n+1)[(2n^2+7n+6)/6]$ 。第 7 步：因式分解 $2n^2+7n+6 = (n+2)(2n+3)$

(验证: $n \cdot 2n = 2n^2$, $n \cdot 3 + 2 \cdot 2n = 7n$, $2 \cdot 3 = 6$) : $= (n+1)(n+2)(2n+3)/6$ 。第 8 步: 右边正是 $(n+1)(n+2)(2n+3)/6$, 故 $P(n+1)$ 成立。

[收束] 第 9 步: 由数学归纳法, 命题对一切正整数 n 成立。■

答: 证明如上, 关键代数步骤是"提取 $(n+1)$ + 通分 + 因式分解 $2n^2+7n+6=(n+2)(2n+3)$ "。

常见错误: ① 通分时把 $(n+1)$ 当成整数 1, 写成 $(n+1)[n(2n+1)/6+1]$ 而非 $+(n+1)$; ② 因式分解 $2n^2+7n+6$ 出错 (忘了交叉项 $7n$ 是 $3 \cdot 2n + 2 \cdot n$); ③ 只做到"括号内", 没还原成 $(n+1)(n+2)(2n+3)/6$ 的标准形就收手。

题 3 (难): 反证法完整书写——证明 $\sqrt{2}$ 是无理数

题干: 用反证法**完整书写** (含假设、每一步依据、明确矛盾点) 证明: $\sqrt{2}$ 不是有理数。要求用到引理" a^2 为偶 $\Rightarrow a$ 为偶" (可用最简论证说明此引理成立)。

完整分步解答 (可复写模板):

命题: $\sqrt{2}$ 是无理数。

证明 (反证法): 第 1 步: **假设** $\sqrt{2}$ 是有理数。第 2 步: 由有理数定义, 存在整数 a, b , $b \neq 0$, 且 a, b 互质, 使 $\sqrt{2} = a/b$ 。(依据: 任何有理数可写成互质的整数比。) 第 3 步: 两边平方: $2 = a^2/b^2$, 即 $a^2 = 2b^2$ 。(依据: 等式两边同乘 b^2 。) 第 4 步: 由 $a^2 = 2b^2$, a^2 是 2 的倍数, 即 a^2 为偶。(依据: $2b^2$ 显然为 2 的倍数。) 第 5 步: **引理:** 若整数 c 满足 c^2 为偶, 则 c 为偶。证: 若 c 为奇, $c = 2k+1$, 则 $c^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$ 为奇, 矛盾。故 c 必为偶。(依据: 奇偶性定义。) 第 6 步: 由引理, a 为偶, 故存在整数 m 使 $a = 2m$ 。(依据: 偶数定义。) 第 7 步: 代回 $a^2 = 2b^2$: $4m^2 = 2b^2$, 两边同除以 2: $2m^2 = b^2$, 故 b^2 为偶。(依据: 代数化简。) 第 8 步: 再由引理, b 为偶, 故存在整数 t 使 $b = 2t$ 。(依据: 偶数定义。) 第 9 步: 于是 a, b 都能被 2 整除, 即 a, b 不互质 ($\gcd \geq 2$), **这与第 2 步的" a, b 互质"矛盾**。(依据: 互质定义。) 第 10 步: 故假设不成立, $\sqrt{2}$ 不是有理数, 即 $\sqrt{2}$ 是无理数。■

答: 证明如上, 矛盾点是" a, b 均被 2 整除, 与互质冲突"。

常见错误: ① 忘了先"约分到最简" (不要求互质, 就推不出最终矛盾); ② 引理" a^2 偶 $\Rightarrow a$ 偶"当成"显然"而不写依据——正因为它不显然, 第 06 章还会专门证; ③ 第 7 步的"两边同除以 2"漏掉, 直接写" b^2 偶"缺依据; ④ 矛盾点不明确 (要说"这与.....矛盾")。

向导便签

这一章咱们干了什么：把你从第 01 章起憋着的“凭什么”，正式变成了一套可用的工具——证明的四种招式：直接证明、反证法、数学归纳法、构造性证明。你不但知道了四招各在什么时候用，还拿到了每招的“书写格式模板”，并在严格化走廊里亲手复写了两个完整的证明（ $\sqrt{2}$ 无理数·反证法、前 n 项和·归纳法）。

钉死一句话：**证明不是魔法，是把“你懂的东西”按规则连起来；而“按规则连”有四种招式——正面推、反着假设推矛盾、推骨牌、造出来。**

记住：**读证明先问“用的哪一招”，写证明先套“格式模板”**。这两件事，是你以后 6 册里写和读一切严格化走廊的看家本领。下一章，我们把这个工具箱开进数论，去啃那个两千年的硬命题：**素数有无穷多个，拿什么证明？**

造物主手记

我曾踩的坑：

我以为“我验了前 100 个数都对”就能算证明，被老谟一句“你能验无穷个吗”堵住——**例证不是证明，覆盖所有情况才是**。我还差点把归纳法递推步里的“假设”当成耍赖——被点醒才知道那是“条件句”，不是“我相信它成立”。最险的是反证法：我差点推到一半没撞出矛盾就硬说“所以假设错”，老谟说矛盾必须真撞到礁石。

顿悟时刻：

真正开窍是看老谟推那排扑克牌。“第一张会倒”+“前面倒了后面一定倒”=整排都倒——**归纳法一下子不神秘了**。而更震撼的是，我发现我第 01 章做补码时干的其实就是“构造性证明”：我没证明负数有表示法，我直接造了一个。**原来我早就在用这些招式，只是不知道它们的名字。**

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A（计算，易→中）：归纳法书写立方和公式

用数学归纳法**完整书写**证明：对一切正整数 n ， $1^3+2^3+\dots+n^3 = [n(n+1)/2]^2$ 。

完整解答： **基础步：** $n=1$ 时，左边 $=1^3=1$ ；右边 $=[1\times 2/2]^2=1$ 。 $P(1)$ 成立。 **递推步：**设 IH： $1^3+\dots+n^3=[n(n+1)/2]^2$ 。要证 $P(n+1)$ ： $1^3+\dots+n^3+(n+1)^3 = [(n+1)(n+2)/2]^2$ 。 左边 $= [n(n+1)/2]^2 + (n+1)^3 = (n+1)^2[n^2/4 + (n+1)] = (n+1)^2[n^2/4 + (4n+4)/4] = (n+1)^2[(n^2+4n+4)/4] =$

$(n+1)^2[(n+2)^2/4] = [(n+1)(n+2)/2]^2$ 。右边一致，故 $P(n+1)$ 成立。由归纳法，命题对一切 n 成立。

■

题 B (计算, 中→难) : 反证法书写——证明 $\sqrt{3}$ 是无理数

用反证法**完整书写**证明 $\sqrt{3}$ 是无理数。(提示: 需引理" a^2 被 3 整除 $\Rightarrow a$ 被 3 整除", 可仿照" a^2 偶 $\Rightarrow a$ 偶"的最简论证: 若 a 不被 3 整除, 则 $a \equiv 1$ 或 $2 \pmod{3}$, 平方后 $\equiv 1 \pmod{3}$, 不为 0。)

完整解答: 证明 (反证法): ① 假设 $\sqrt{3}$ 是有理数, 则存在互质整数 $a, b, b \neq 0$, 使 $\sqrt{3}=a/b$ 。② 平方: $3=a^2/b^2$, 即 $a^2=3b^2$, 故 $3|a^2$ 。③ 引理: 若 $3|a^2$ 则 $3|a$ 。证: 若 a 不被 3 整除, 则 $a=3k \pm 1$, 平方 $a^2=9k^2 \pm 6k + 1 \equiv 1 \pmod{3}$, 故 $3 \nmid a^2$, 矛盾。故 $3|a$ 。④ 由引理, $a=3m$, 代回: $9m^2=3b^2$, 即 $3m^2=b^2$, 故 $3|b^2$, 再由引理 $3|b$ 。⑤ 于是 a, b 都能被 3 整除, 与"互质"矛盾。⑥ 故假设错, $\sqrt{3}$ 是无理数。■

题 C (概念, 检查理解) : 识别证明类型

老谟给出三个证明片段, 请指出各用哪一招 (直接/反证/归纳/构造), 并说明判断依据: (1) "设 n 为奇数, $n=2k+1$, 则 $n^2=4k^2+4k+1$ 为奇数, 故 n^2 不为偶数。" (这是在对偶地证" n^2 偶 $\Rightarrow n$ 偶") (2) "设 $n=1$ 时成立; 假设 n 时成立, 推出 $n+1$ 时成立, 故对一切 n 成立。" (3) "取集合 $\{1\}$, 它满足'非空且所有元素为 1', 故存在非空集合。" **完整解答:** (1) **直接证明 (用对偶命题)**——从" n 奇"直接推到" n^2 奇", 是正面推, 不是反证 (没先假设结论假)。 (2) **数学归纳法**——基础步+递推步的模板。 (3) **构造性证明**——直接造出集合 $\{1\}$ 验证性质。

预告

四招齐了: 正面推、反着假设推矛盾、推骨牌、造出来。你有了写证明的脚手架, 也有了读证明时"先看是哪一招"的眼睛。

下一章, 我们把工具箱开进**数论**——真刀真枪地证明那些关于整数、整除、素数的命题。尤其有一个命题, 从古希腊流传了两千年, 无数人试过, 却没人能"正面"证完: **素数到底有多少个?** 它靠的是反证法那招"反着假设", 而反证法里藏着一个绝妙的构造。**这一章, 你不但要证明这个命题, 还要亲手学会'怎么把证明写得每一步都有依据'——因为下一章之后, 这一册的地基和工具就都齐了, 你要带着它们去盖下一册《实数极限与连续》的楼了。**

第 06 章 · 证明实战——拿什么证明"素数有无穷多个"

本章从哪个问题出发：第 05 章你拿到了证明的四招。可招式在纸上，要见真章得开进一块"战场"。老谟抛来一个从古希腊流传两千年的命题——"**素数有无穷多个**"。没人能"正面"把它证完，可它确实能证。**它靠的是哪一招？那个绝妙的构造又藏在哪儿？**推导到哪个结论：你会在这一章把第 05 章的工具箱开进数论：**整除**的性质、**素数**的精确定义、用**反证法**证"素数有无穷多个"（欧几里得那招" $N = p_1 p_2 \cdots p_k + 1$ "）、用**对偶/直接证**" n^2 偶 $\Rightarrow n$ 偶"、并（叙述性地）认识**算术基本定理**。到章末，这一册的地基（数/逻辑/集合/函数）与工具（证明四招）全部集齐——**你带着它们，准备盖下一册《实数极限与连续》的楼。**

核心认知：**证明实战 = 把第 05 章的四招，用在"关于整数的命题"上。**这一章你亲手证明两个硬命题：**" n^2 偶 $\Rightarrow n$ 偶"**（用对偶命题直接证）和**"**素数有无穷多个**"（用欧几里得反证法——**先假设有限，再构造一个"列表外"的素数，撞出矛盾**）。你还会第一次尝到算术基本定理**"每个整数都能唯一分解成素数乘积"的味道。**证明不仅保护直觉，它本身就是发现新数学的方式——那个 $N+1$ 的构造，就是在反证法里"被逼出来"的新东西。**

引子

老谟把一叠写着数字的纸片摊在桌上，全是素数：2、3、5、7、11、13.....

"我要是写一张，写一张，永远写下去，"他说，"你能不能说，'迟早有一天我把所有素数都写完了'？"

我盯着那排数，手指在桌上敲到一半停住。".....好像不能。可你要我说出'为什么不能'，我又说不清——我总不能一张一张写到永远吧？"

"那这个问题就来了：**'素数到底有没有穷多个'——你打算拿什么证明？**"老谟把纸片拨到一边，空出桌面的中间，"你第 05 章学了四招。你猜，这一题，该用哪一招？"

"正面证它'无穷多个'....."我试着想，"那得证明'永远写不完'，可我正面证'无穷'，怎么证？总不能把无穷个都写出来吧。"

"你又说中了。'无穷'不是个能被'写出来'的东西，它是.....你靠近它就躲开、你怎么追都追不到的性质。"老谟顿了顿，"可要是反过来想——**先假设它有限呢？假设素数只有那么几个，然后我们从这个'有限'出发，看能不能推爆它。**"

我心里咯噔一下。反着假设，推出矛盾——这不是第 05 章那招"反证法"吗？可"假设素数有限"之后，到底能推出个什么矛盾来？我盯着桌面上那排 2、3、5、7，怎么也想不出，那排数怎么会自己"爆炸"。

对话引擎

第 1 轮：什么是素数——先把这个“战场”的地形说清

老谟：“开打之前，先认地形。你说‘素数’，可你知道素数的精确定义是什么吗？别急着背，你先想——它是‘只能被 1 和它自己整除的数’，对吗？那 1 是不是素数？”

造物主：“.....1 只能被 1 整除，它自己也是 1.....按‘只能被 1 和它自己整除’，1 好像应该算？可我又总觉得 1 不该是素数。”

老谟：“你的直觉对——**1 不是素数**，这是人为约定。数学家把‘素数’定义成：**大于 1 且除了 1 和它自身外没有其他正因数的正整数**。把 1 排除在外，不是因为它不够格，而是因为如果 1 算素数，那么“每个数唯一分解成素数乘积”就会出岔子（1 可以乘任意多个）。所以‘1 不是素数’是一条规定，不是发现。这正是你在第 01 章学到的那个词——约定。地形先认清：我们说的素数，是大于 1 的那个家族。”

造物主：“.....所以我这一章要证‘有无穷多个’的，是‘大于 1 且没有其他因数’的那一族数。它们被 1 排除在外。”

知识锚点：整除与素数——“战场”的定义

【概念深挖】**整除 (divisibility)**：设 a 、 b 为整数， $b \neq 0$ 。若存在整数 q 使 $a = bq$ ，则称 **b 整除 a** ，记作 $b \mid a$ ；称 b 是 a 的一个**因数 (divisor)**。若不存在这样的整数 q ，则 $b \nmid a$ 。

- 例如： $3 \mid 12$ ($12=3 \times 4$)； $5 \nmid 12$ (12 不是 5 的倍数)。

素数 (prime)：大于 1 且除了 1 和它自身外没有其他正因数的正整数。**合数 (composite)**：大于 1 且不是素数的正整数（即有至少一个大于 1 小于自身的因数）。

- **约定注记：1 既不是素数也不是合数**。这是人为约定——为了“唯一分解”定理成立（1 可被任意多个 1 相乘，会破坏唯一性）。“1 不是素数”不是发现，是规定。

为什么研究素数：素数是整数的“乘法原子”——任何大于 1 的整数都能分解成素数的乘积（这是本章末尾要叙述的**算术基本定理**）。所以“素数的个数”决定了整数的“基本构件”够不够用。

第 2 轮：整除的性质——直接证明的第一批战果

老谟：“地形认清了，先打几个小仗热身。我给你三条整除的性质，你试着用‘直接证明’证出来：第一， $a \mid b$ 且 $b \mid c$ ，则 $a \mid c$ (传递性)；第二， $a \mid b$ ，则 $a \mid (b \text{ 的任意倍数})$ ；第三， $a \mid b$ 且 $a \mid c$ ，则 $a \mid (b \pm c)$ 。三条你先挑一条证。”

造物主：".....我证传递性。 $a \mid b$ 就是 $b = a \cdot q_1$ ， $b \mid c$ 就是 $c = b \cdot q_2$ ，把 b 代进去： $c = (a \cdot q_1) \cdot q_2 = a \cdot (q_1 \cdot q_2)$ ，所以 $a \mid c$ 。成了，这是直接证明——从整除定义一路代。"

老谟："漂亮，这就是'直接证明'在数论里的标准用法：把'整除'翻译成'存在整数倍数'，再代数代入。你第 05 章学的直接证明，在这里落地了。这三条性质，就是后面证' n^2 偶 $\Rightarrow n$ 偶'、证素数性质时，随手就能用的砖。你注意到没有——证明不是'套公式'，是'把定义翻译成代数，再推'。"

知识锚点：整除的运算性质

【概念深挖】整除的常用性质（都可直接证明）：

- **传递性：**若 $a \mid b$ 且 $b \mid c$ ，则 $a \mid c$ 。
- **线性性：**若 $a \mid b$ 且 $a \mid c$ ，则对任意整数 s, t ， $a \mid (sb + tc)$ 。
- **倍数继承：**若 $a \mid b$ ，则 $a \mid (b \text{ 的任意倍数})$ 。

证明方法：把" $a \mid b$ "翻译成"存在整数 q 使 $b=aq$ "，再代数代入。**直接证明在数论里的主战场就是这个"翻译 + 代入"。**这些性质会在本章后面反复用到，也是第 02 册实数（数论基础）的常用砖。

第 3 轮： n^2 偶 $\Rightarrow n$ 偶——"对偶命题"这个取巧

老谟："热身完，来第一个真命题：**'若 n^2 是偶数，则 n 是偶数。'**你第 05 章学过直接证明、反证法。这个命题，直接证明推得动吗？"

造物主："..... n^2 偶，就是 $n^2=2m$ 。我想推出 n 偶，可我不能对 n^2 开平方吧？开平方不是整数运算.....好像直接推不动。"

老谟："推不动。那换个思路——' n^2 偶 $\Rightarrow n$ 偶'正面推不动，那你反过来想：**要证一个'若 P 则 Q '推不动时，数学家常去证它的'对偶命题'——'若 Q 错则 P 错'。**你说说，这个命题的'对偶'应该长什么样？"

造物主：".....对偶？' n^2 偶 $\Rightarrow n$ 偶'的对偶，就是' n 不偶（奇数） $\Rightarrow n^2$ 不偶（奇数）'？证这个，就是从 $n=2k+1$ 一路推 n^2 也是奇数——**好像比正面开方好推多了！**"

老谟："你摸到方法了：**正面推不动，就转去证它的对偶（逆否命题）——证完对偶，原命题自然成立。**至于'从 $n=2k+1$ 到 n^2 奇'的具体代数演算、以及'对偶与原命题等价'这件事怎么严格说明——**完整可复写证明在严格化走廊的定理 1 里，你翻过去对着空白纸补全。**我们先把这一招认下来：**很多' $\Rightarrow$ '型证明的突破口，是发现'正面难、对偶易'。**这个命题别看小，它是证 $\sqrt{2}$ 无理数的关键引擎（第 05 章你用过它当引理）——现在你终于正式把它证出来了。"

知识锚点：对偶命题（逆否命题）与“ n^2 偶 $\Rightarrow n$ 偶”

【概念深挖】**对偶命题 / 逆否命题 (contrapositive)**：命题“若 P 则 Q ”的**逆否命题**是“若 $\neg Q$ 则 $\neg P$ ”。**关键事实**：一个命题和它的逆否命题是等价的——证逆否成立，等于证原命题成立。

为什么用逆否：有些命题正面难证、逆否好证。例：“ n^2 偶 $\Rightarrow n$ 偶”正面难（不能开方），但逆否“ n 奇 $\Rightarrow n^2$ 奇”好证： $n=2k+1 \Rightarrow n^2=4k^2+4k+1=2(2k^2+2k)+1$ 是奇数。

证明类型标注：这属于“**直接证明（用对偶命题）**”——从“ n 奇”直接推到“ n^2 奇”，是正面推（没有先假设“ n^2 偶”为假再推矛盾，那是反证法）。**分清“反证法”（假设原结论假）与“证逆否”（证一个等价的正面命题）**是本章一个易混点：两者思路相近，但反证法假设“ $\neg Q$ ”推矛盾，证逆否是直接证“ $\neg Q \Rightarrow \neg P$ ”。

跨册衔接：这个“ n^2 偶 $\Rightarrow n$ 偶”及其变体（“ n^2 被 3 整除 $\Rightarrow n$ 被 3 整除”等），在第 02 册实数的无理数证明、以及大量整除性论证里会反复作为引理出现。

第 4 轮：素数无穷——先假设有限，再看“列表外”的构造

老谟：“现在上主菜。**命题**：素数有无穷多个。你第 05 章说过，正面证‘无穷’很难。我们反着来——假设素数只有有限多个： $p_1, p_2, \dots, p_k$ （这就是全部素数）。现在，别急着想矛盾在哪，你先想——凭这 k 个数，你能造出一个什么样的、它自己不在列表里的新数？”

造物主：“……我能造一个新的数？把这 k 个素数……乘起来？ $p_1 \times p_2 \times \dots \times p_k$ ？可乘起来还是那些素数的乘积，不新。要是……乘起来再加 1？ $N = p_1 \times p_2 \times \dots \times p_k + 1$ ？”

老谟：“你为什么想到‘加 1’？”

造物主：“因为……如果 N 只由 $p_1 \dots p_k$ 相乘而来，那它就能被这些素数整除；可加 1 之后，它除以任何一个 p_i 都会余 1，**谁都不整除它**——它就‘逃出’了这 k 个素数的魔爪？”

老谟：“你摸到那个绝妙构造的灵魂了。 $N = p_1 p_2 \dots p_k + 1$ 除以任何一个 p_i 都余 1，所以它不被列表里任何一个素数整除。可 N 是大于 1 的整数，它自己总得有素因数吧？那这个素因数是谁？它不能是列表里的任何一个——**所以它一定是列表外的一个新素数**。这就矛盾了：我们假设列表里是‘所有’素数，可 N 逼出了一个列表外的。**这个‘乘起来加 1’的构造，就是欧几里得两千年前那一下**。完整证明放严格化走廊，你现在抓住直觉：**反证法的‘矛盾’不是等来的，是‘造’出来的——那个 $N+1$ 就是被‘假设只有有限个’逼出来的新东西。**”

知识锚点：素数无穷——欧几里得的“N+1”构造

【概念深挖】**定理（素数有无穷多个）**：素数有无穷多个。**证明（欧几里得，反证法）**：

1. 假设素数只有有限个： $p_1, p_2, \dots, p_k$ （这就是所有素数的完整列表）。
2. 构造 $N = p_1 p_2 \cdots p_k + 1$ 。
3. $N > 1$ ，故 N 必有一个素因数 q （依据：算术基本定理的“存在性”部分，或更朴素地：对 N 反复分解因数，最终停在一个素数上）。
4. 若 q 在列表 $\{p_1, \dots, p_k\}$ 中，则 $q \mid N$ （构造）且 $q \mid p_1 p_2 \cdots p_k$ （ q 是其中一个因子），于是 $q \mid (N - p_1 p_2 \cdots p_k) = 1$ （整除的线性性/差的性质）。但 $q > 1$ 不可能整除 1，矛盾。
5. 故 q 不在列表中，即存在一个列表外的素数 q 。这与“ $p_1, \dots, p_k$ 是所有素数”矛盾。
6. 故假设不成立，**素数有无穷多个**。■

为什么 N 必有一个素因数： N 是大于 1 的整数。若 N 本身是素数，那它自己就是那个因数；若 N 是合数，则它能分解成更小的因数乘积，不断分解最终停在某个素数上。**这个“不断分解必到素数”的事实，正是算术基本定理的一部分**——本章先以叙述方式承认它（见后文“算术基本定理”），完整的严格证明是数论更深的内容。

跨册衔接：这个“假设有限 $\rightarrow$ 造一个列表外的新对象 $\rightarrow$ 矛盾”的模板，是**反证法**的经典范本。你在第 02 册、后续册里会反复见到“构造一个避开列表的新对象”这一招。

第 5 轮：算术基本定理——唯一分解，为什么它配得上“基本”二字

老谟：“证明完‘素数无穷’，你该问一句：**凭什么数学这么看重素数？** 答案藏在一个叫‘算术基本定理’的定理里——它说：**每个大于 1 的整数，都能唯一地分解成素数的乘积（不计顺序）**。比如 $12 = 2 \times 2 \times 3$ ，写成 $3 \times 2 \times 2$ 只是换顺序，本质是同一个分解。”

造物主：“……‘唯一’？那 12 除了 $2 \times 2 \times 3$ ，还有别的素数分解吗？比如 4×3 ？可 4 不是素数…… $12 = 2 \times 6 = 2 \times 2 \times 3$ ，好像最后都拆到 $2 \times 2 \times 3$ 。”

老谟：“你摸到‘唯一’的味道了——**中间怎么拆可能不同（ 2×6 、 3×4 ），但拆到素数的最终结果只有一个（ $2 \times 2 \times 3$ ）**。这就是为什么 1 不能被算作素数——如果 1 算素数，那 $12 = 2 \times 2 \times 3 = 2 \times 2 \times 3 \times 1 = 2 \times 2 \times 3 \times 1 \times 1 \dots$ 可以无限加 1，‘唯一’就碎了。**所以‘1 不是素数’这条约定，是为了保住‘唯一分解’**。这个定理‘基本’在哪儿？因为它说明素数是一切整数的‘乘法原子’——你研究素数，就是在研究所有整数的骨架。”

造物主：“……所以这一章我证明了‘原子足够多’（素数无穷），又听说‘每个数都是原子拼的’（算术基本定理）——素数和整数是互相呼应的两件事。”

老谟："正是。素数无穷说明'原子用不完'，算术基本定理说明'原子拼得成一切'。这两件事合起来，就是整数的乘法全景。算术基本定理的完整证明（涉及'唯一性'那半，要用到素数性质）本章先叙述，你在后续学习里会见到它的严格证明。"

知识锚点：算术基本定理（叙述）

【概念深挖】**算术基本定理 (Fundamental Theorem of Arithmetic)**：每个大于 1 的整数 n ，都能写成素数的乘积，且这种写法在**不计顺序**的意义下是**唯一**的。即 $n = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}$ ，其中 p_i 是互不相同的素数、 a_i 是正整数，且该表示唯一。

为什么 "1 不是素数" 是必要约定：若 1 算素数，则 n 的分解可任意乘 1 ($12=2 \times 2 \times 3=2 \times 2 \times 3 \times 1=\dots$)，"唯一性"被破坏。所以**"1 不是素数"不是发现，是为了让算术基本定理成立而设的约定**——这是"约定意识"在数论里的又一次落子。

存在性与唯一性：定理分两半——① **存在性**：每个 $n > 1$ 必有素数分解（反复分解到素数即可，直觉显然）；② **唯一性**：分解唯一（这半较难，需用素数性质，本章只叙述，不证全）。我们在"素数无穷"的证明里用到的"**N 必有一个素因数**"，正是存在性的直接应用。

第 6 轮：落点——全册收束：地基和工具，都齐了

老谟："把这一册的东西都收起来。从第 01 章的比特，到这一章的素数——你手上现在有什么？"

造物主：".....我数数。第 01 章，我学会了数——二进制、补码，还埋了'凭什么'的种子。第 02 章，逻辑——布尔代数，怎么判断。第 03 章，集合——把东西圈起来。第 04 章，函数——从一个到另一个。第 05 章，证明四招——直接、反证、归纳、构造。第 06 章，我把四招开进了数论，证了' n^2 偶 $\Rightarrow$ n 偶'、'素数无穷'，还听了算术基本定理。"

老谟："那你说，这一册的地基和工具，齐了吗？"

造物主：".....齐了。地基四件套（数/逻辑/集合/函数）+ 证明四招。我现在不但知道'数是什么、怎么判断、怎么圈、怎么映射'，还会'拿证明去给它们撑腰'了。"

老谟："这就不赖了。记住你手里这两套东西：四个地基概念，是你要盖的楼的'地基'；四个证明招式，是你'盖楼用的脚手架'。下一册《实数极限与连续》，你就要用这套脚手架，去盖第一座真正的大楼了——**极限**。那里的证明，会用到你今天练的反证法（很多'不存在'的极限结论）、归纳法（对一切正整数成立的不等式）、对偶命题（很多正反等价的论证）。这一册到此收束——你不是学会了'看别人证明'，你是学会了'自己写证明'。"

严格化走廊

位置说明：对话负责"直觉"——为什么反证法能证素数无穷。本走廊负责"严谨"——把数论的硬命题落成精确定义与两个**完整可复写**的证明，每个都标注"证明类型 + 依据"。这是本册证明工具库的"出师考"。

定义：整除、素数、合数（精确表述）

整除 (divisibility)：a、b 为整数， $b \neq 0$ 。若 $\exists q \in \mathbb{Z}$ 使 $a = bq$ ，则 $b \mid a$ ；否则 $b \nmid a$ 。

素数 (prime)：整数 $p > 1$ ，且 p 的正因数只有 1 与 p。**合数 (composite)**：整数 $n > 1$ ，且 n 不是素数（即 $\exists d$ 满足 $1 < d < n$ 且 $d \mid n$ ）。**1 的特殊地位**：1 既非素数也非合数（约定）。

最大公因数 (gcd)：a、b 的最大公因数 $\gcd(a,b)$ 是能同时整除 a、b 的最大正整数。若 $\gcd(a,b)=1$ ，称 a、b 互质 (coprime)。

定理 1 (n^2 偶 $\Rightarrow$ n 偶) ——证明类型：直接证明（用对偶命题）

前提：n 为整数。

结论：若 n^2 是偶数，则 n 是偶数。

证明（用对偶命题，即证" n 为奇 $\Rightarrow n^2$ 为奇"）：

- 对偶命题**：" n^2 偶 $\Rightarrow$ n 偶"的逆否是" n 奇 $\Rightarrow n^2$ 奇"。因为命题与其逆否等价，证后者即可。
- 设 n 为奇数（即假设 $\neg(n \text{ 偶})$ ）。
- 由奇数定义，存在整数 k 使 $n = 2k+1$ 。
- 计算 $n^2 = (2k+1)^2 = 4k^2 + 4k + 1 = 2(2k^2+2k) + 1$ 。（依据：完全平方展开、提取公因式 2）
- 由步骤 4， n^2 可以写成" $2 \times \text{整数} + 1$ "的形式，故 n^2 是奇数。
- 已证" n 奇 $\Rightarrow n^2$ 奇"，等价于其逆否" n^2 偶 $\Rightarrow$ n 偶"。■

依据标注：步 1 用"命题与其逆否等价"（对偶命题原理）；步 2 用"非偶即奇"的整数二分性质；步 3 用奇数定义；步 4 用代数运算；步 6 用逆否等价。**读法**：这个证明的巧劲在"换个等价命题证"。**证明类型是直接证明**（从" n 奇"直接推到" n^2 奇"），不是反证法——它没有先假设" n^2 偶"为假再推矛盾。分清这个区别，是本章重点。

定理 2 (素数有无穷多个) ——证明类型：反证法 + 构造

前提：素数为"大于 1 且除 1 和自身外无正因数"的整数；**引理：**任何大于 1 的整数都有一个素因数（算术基本定理的存在性部分，此处以叙述方式承认）。

结论：素数有无穷多个。

证明（反证法）：

1. **假设**素数只有有限多个，记完整列表为 $p_1, p_2, \dots, p_k$ 。
2. **构造** $N = p_1 p_2 \cdots p_k + 1$ 。（这一步的"加 1"是本证明的绝妙构造——它让 N 除以任何 p_i 都余 1。）
3. $N > 1$ （因为 $p_1 \geq 2$ ，乘积 ≥ 2 ，加 1 后 ≥ 3 ），由引理， N 有一个素因数 q 。
4. **分类讨论 q 是否在列表中：**
 - 若 $q \in \{p_1, \dots, p_k\}$ ：则 $q \mid p_1 p_2 \cdots p_k$ (q 是乘积中一个因子)，且 $q \mid N$ (q 是 N 的因数)。由整除的线性性/差的性质， $q \mid (N - p_1 p_2 \cdots p_k) = 1$ 。但 $q > 1$ 的素数不可能整除 1。**矛盾。**
 - 若 $q \notin \{p_1, \dots, p_k\}$ ：则存在一个列表外的素数 q ，这与" $p_1, \dots, p_k$ 是所有素数"的假设矛盾。
5. 两种情况都矛盾，故假设"素数只有有限多个"不成立。
6. **素数有无穷多个。■**

依据标注：步 1 用反证法设定；步 2 构造 N （关键）；步 3 用"任何大于 1 的整数有素因数"（叙述性引理）；步 4 用整除的性质（ q 整除两数之差）与"素数 >1 不能整除 1"；步 5 用矛盾律。**读法：**这个证明同时用了"反证法"（假设有限）和"构造"（ $N=p_1 \cdots p_k+1$ ）——是第 05 章四种招式中"反证法 + 构造"的组合拳。**注意那个 N 的构造是核心：**它既是"列表内矛盾"的引线，又是"列表外素数存在"的证据。**这正是反证法里'矛盾是造出来的'的最佳示范。**

反例/边界：数论命题的边界与使用红线

1. " n^2 偶 $\Rightarrow n$ 偶"的逆命题是否成立？逆命题" n 偶 $\Rightarrow n^2$ 偶"显然成立（ $n=2k \Rightarrow n^2=4k^2$ 偶）。但 ****" n^2 被 4 整除 $\Rightarrow n$ 被 4 整除"不成立****——反例： $n=2$ ， $n^2=4$ 被 4 整除，但 $n=2$ 不被 4 整除。**别把"偶"的特例推广到所有整除性。**
2. **反证法的矛盾必须来自"假设有限"本身：**欧几里得证明的矛盾，追根溯源来自"列表是全部素数"这个假设（ q 既必须在列表中又在列表外）。若没有"假设全部"这一条，构造 $N+1$ 只是"造一个新数"，不构成矛盾。**反证法的"矛盾"必须是被假设逼出来的。**
3. **"素数无穷"不等于"素数出现得很密"：**素数有无穷多个，但素数之间的间隔可以任意大（连续 $n-1$ 个合数 $n!+2, \dots, n!+n$ 的存在性可用构造证明）。**"无穷多"只说明数量，不说明分布密度。**

4. 算术基本定理的"唯一性"依赖"1 不是素数"这个约定：若把 1 算素数，唯一分解立刻失效。这是"约定意识"在数论里的关键体现——一条约定看似随意，却撑起一个定理。
5. "整除"的 0 的特殊性： $b \mid a$ 要求 $b \neq 0$ 。0 不能当除数；但"0 被任意非零数整除"（任意 $q \cdot b$ 可使 $a=0$ ）是另一回事，别混淆。

知识点总结

知识点（本章推出的核心概念）

- **整除 (divisibility)**： $b \mid a$ 当且仅当 $\exists q, a=bq$ 。是数论的"翻译语言"——把整除翻成"存在整数倍数"，再代数代入。
- **素数 / 合数**：素数 = 大于 1 且除 1 和自身外无正因数；**1 既非素数也非合数（约定）**。
- **对偶命题（逆否命题）**："若 P 则 Q"与"若 $\neg Q$ 则 $\neg P$ "等价；正面难证转证逆否。" n^2 偶 $\Rightarrow n$ 偶"用逆否" n 奇 $\Rightarrow n^2$ 奇"证。
- **素数无穷（欧几里得反证法）**：假设有限，构造 $N=p_1 \cdots p_k+1$ ，逼出列表外素数，矛盾。**反证法 + 构造的组合拳**。
- **算术基本定理（叙述）**：每个 $n>1$ 唯一分解成素数乘积。"**1 不是素数**"是为保唯一性的约定。
- **证明类型识别（数论语境）**：整除性质=直接证明；" n^2 偶 $\Rightarrow n$ 偶"=对偶直接；"素数无穷"=反证 + 构造。

历史结论（本章涉及的史料）

- **素数无穷（欧几里得，约公元前 300 年）**：《几何原本》第九卷给出"素数有无穷多个"的证明，用 $N=p_1 \cdots p_k+1$ 的反证法。**这是数学史上最经典的证明之一，两千多年无人超越其简洁**。
- **算术基本定理**：唯一分解的思想古希腊已有（欧几里得《几何原本》第七卷有雏形），现代严格形式由高斯（Gauss, 1801《算术研究》）系统证明。
- **$\sqrt{2}$ 无理数与对偶命题**：第 05 章证 $\sqrt{2}$ 时用的" a^2 偶 $\Rightarrow a$ 偶"，正是本章定理 1 的引理形式——两章在此衔接。

工程实践结论（本章知识在真实工程里怎么用）

- **素数在密码学**：RSA 加密依赖"大素数很难分解、但判断一个数是不是素数（素性测试）相对容易"。"**素数无穷**"保证了加密所需的素数永远够用。
- **整除/同余在哈希与校验**：哈希取模、CRC 校验码都依赖整除与同余运算（本章计算训练场的欧几里得除法）。

- **证明类型识别=算法分析直觉**：工程师看“为什么这个算法对”，常对应某类证明——递归用归纳、贪心用反证/交换论证、存在性算法用构造。**识别招式，就是拿到调试地图。**

计算训练场

位置说明：本章对话负责“证明的直觉”，这里负责“完整书写证明 + 数论计算”。下面三题重点是**把证明写成可复写格式、并用欧几里得除法做同余计算**——先自己写再对照。**难度递进：易→中→难。**

题 1（易→中）：欧几里得除法——求商余、做同余、判定素数

题干：用欧几里得除法（带余除法）计算：(a) 求 437 除以 12 的商和余数，写出 $437 = 12 \times q + r$ ($0 \leq r < 12$)。 (b) 由 (a) 判断 437 是否被 12 整除；求 $437 \bmod 12$ 。 (c) 计算 $2^{10} \bmod 7$ （即 1024 除以 7 的余数）。 (d) 判断 97 是否为素数（提示：只需检查不超过 $\sqrt{97}$ 的素数）。

完整分步解答：

(a) 带余除法：第 1 步：试商： $12 \times 36 = 432$, $12 \times 37 = 444 > 437$ ，故最大商 $q=36$ 。第 2 步：余数 $r = 437 - 432 = 5$ ，且 $0 \leq 5 < 12$ 。**答 (a)**： $437 = 12 \times 36 + 5$ ，商 36，余 5。

(b) 整除与 mod：第 1 步：余数 $r=5 \neq 0$ ，故 437 不被 12 整除 ($12 \nmid 437$)。第 2 步： $437 \bmod 12 = 5$ （余数即模值）。**答 (b)**： $12 \nmid 437$, $437 \bmod 12 = 5$ 。

(c) 同余：第 1 步： $1024 = 7 \times 146 + 2$ （因 $7 \times 146 = 1022$, $1024 - 1022 = 2$ ）。第 2 步： $1024 \bmod 7 = 2$ 。**答 (c)**： $2^{10} \bmod 7 = 2$ 。

(d) 素性判定：第 1 步： $\sqrt{97} \approx 9.8$ ，故只需检查 ≤ 9 的素数：2、3、5、7。第 2 步： $97 \bmod 2 = 1$ ； $97 \bmod 3 = 1$ （ $9+7=16$ 不被 3 整除）； $97 \bmod 5 = 2$ ； $97 \bmod 7 = 6$ （ $7 \times 13 = 91$, $97 - 91 = 6$ ）。第 3 步：均不为 0，故 97 无小于 $\sqrt{97}$ 的素因数。**答 (d)**：**97 是素数**。

常见错误：① 商取得过大或过小（要满足 $0 \leq r < \text{除数}$ ）；② 判定素数只查一部分、或查了超过 $\sqrt{n}$ 的数（浪费时间且可能误判）；③ 把“ $97 \bmod 3$ ”算错——用数位和 $9+7=16$ 判定更快。

题 2（中→难）：对偶命题完整书写——证明整除命题

题干：用**对偶命题**（逆否命题）**完整书写证明**：“若 $3 \mid n^2$ ，则 $3 \mid n$ ”（即 n^2 被 3 整除则 n 被 3 整除）。要求写出每一步依据。（提示：一个整数被 3 除，余数只能是 0、1、2。这正是本章第 3 轮“ n^2 偶 $\Rightarrow n$ 偶”的对偶证明思路，把“偶/2”换成“被 3 整除/3”。）

完整分步解答（可复写模板）：

命题：若 $3 \mid n^2$ ，则 $3 \mid n$ 。

证明（用对偶命题，即证“ $3 \nmid n \Rightarrow 3 \nmid n^2$ ”）：第 1 步：设 $3 \nmid n$ （即 n 不被 3 整除）。第 2 步：由带余除法， n 被 3 除的余数只能是 1 或 2（不为 0，因 $3 \nmid n$ ）。（依据：带余除法，余数 $\in \{0, 1, 2\}$ 。）第 3 步：**情形一： $n \equiv 1 \pmod{3}$** 。则 $n = 3k+1$ ， $n^2 = (3k+1)^2 = 9k^2+6k+1 = 3(3k^2+2k)+1$ ，故 n^2 被 3 除余 1，即 $3 \nmid n^2$ 。第 4 步：**情形二： $n \equiv 2 \pmod{3}$** 。则 $n = 3k+2$ ， $n^2 = (3k+2)^2 = 9k^2+12k+4 = 3(3k^2+4k+1)+1$ ，故 n^2 被 3 除余 1，即 $3 \nmid n^2$ 。第 5 步：两种情形下都有 $3 \nmid n^2$ ，故已证“ $3 \nmid n \Rightarrow 3 \nmid n^2$ ”。第 6 步：由对偶命题原理，其逆否“ $3 \mid n^2 \Rightarrow 3 \mid n$ ”成立。■

答：证明如上。关键是把“不被 3 整除”分成余 1、余 2 两种情形，分别算出 n^2 都余 1（即不被 3 整除）。

常见错误：① 只证了一种情形（余 1 或余 2），漏了另一种——必须两种情况都覆盖；② 展开 $(3k+2)^2$ 时漏项（应得 $9k^2+12k+4=3(3k^2+4k+1)+1$ ）；③ 忘了用对偶命题原理收尾（证了逆否却没说“等价于原命题”）。

题 3（难）：反证法完整书写——证明 $\sqrt{6}$ 是无理数（综合）

题干：用反证法完整书写证明： $\sqrt{6}$ 是无理数。要求：① 用到第 05 章已证的引理“ $2 \mid n^2 \Rightarrow 2 \mid n$ ”，并通过“2、3 互质”把 2 从 $3b^2$ 中分离出来；② 写出约分到最简、平方、传导、矛盾点每一步依据。

为什么换 $\sqrt{6}$ ：第 05 章已用同招式证过 $\sqrt{2}$ （引理“偶”）。 $\sqrt{6} = \sqrt{2} \cdot \sqrt{3}$ ，它的证明要在‘引理 $2 \mid n^2 \Rightarrow 2 \mid n$ ’之上，再多走一个‘2、3 互质分离素因子’的台阶——是“反证 + 引理 + 素因子分离”的更综合练习，避免与 $\sqrt{2}$ 完全重复。（提示：从 $a^2 = 6b^2$ 出发，先看 2 能推出什么；走到 $2m^2 = 3b^2$ 时，记得 2、3 互质。）

完整分步解答（可复写模板）：

命题： $\sqrt{6}$ 是无理数。

证明（反证法）：第 1 步：**假设** $\sqrt{6}$ 是有理数。第 2 步：由有理数定义，存在整数 a, b ($b \neq 0$ ，且 a, b 互质)，使 $\sqrt{6} = a/b$ 。（依据：任何有理数可约分到互质。）第 3 步：平方： $6 = a^2/b^2$ ，即 $a^2 = 6b^2$ 。（依据：等式两边同乘 b^2 。）第 4 步：由 $a^2 = 6b^2$ ，右边是 2 的倍数 ($6b^2 = 2 \cdot 3b^2$)，故 $2 \mid a^2$ 。（依据：整除定义。）第 5 步：由第 05 章引理“ $2 \mid n^2 \Rightarrow 2 \mid n$ ”， $2 \mid a$ ，故存在整数 m 使 $a = 2m$ 。（依据：引理 + 整除定义。）第 6 步：代回 $a^2 = 6b^2$ ： $4m^2 = 6b^2$ ，两边同除以 2： $2m^2 = 3b^2$ 。现在要把“2”从等式里分离出来：

- 左边 $2m^2$ 是偶数，故右边 $3b^2$ 也是偶数，即 $2 \mid 3b^2$ 。（依据：整除定义。）
- 因 2、3 互质 ($\gcd(2, 3)=1$)，且 $2 \mid 3 \cdot b^2$ ，故 $2 \mid b^2$ 。（依据：欧几里得引理——若素数 $p \mid uv$ 且 $p \nmid u$ ，则 $p \mid v$ ；此处 $2 \nmid 3$ 。）

- 由第 05 章引理 " $2 \mid n^2 \Rightarrow 2 \mid n$ ", $2 \mid b$, 故存在整数 t 使 $b = 2t$ 。第 7 步: 于是 a 、 b 都能被 2 整除, 即 a 、 b 不互质 ($\gcd \geq 2$), **这与第 2 步的" a 、 b 互质"矛盾。**(依据: 互质定义。) 第 8 步: 故假设不成立, **$\sqrt{6}$ 是无理数。** ■

答: 证明如上, 矛盾点是 " a 、 b 均被 2 整除, 与互质冲突"。关键在第 6 步用 "**2、3 互质**" 把 2 从 " $3b^2$ " 里分离出来——这是 $\sqrt{6}$ (非素数) 比 $\sqrt{2}/\sqrt{3}$ 多出的一个台阶。

常见错误: ① 忘了引用两处引理 ($2 \mid a^2 \Rightarrow 2 \mid a$ 是第 05 章、 $3 \mid a^2 \Rightarrow 3 \mid a$ 是题 2), 把 "素因子传导" 当显然; ② 第 6 步直接从 $2m^2 = 3b^2$ 跳 " $3 \mid b^2$ "——**要先分离 2 ($2 \mid 3b^2$, 因 2、3 互质得 $2 \mid b^2$)**, 不是直接看 3; ③ 没指出矛盾点 (要说 "这与互质矛盾")。

向导便签

这一章咱们干了什么: 把第 05 章的证明工具箱开进了数论。你认清了整除、素数的精确定义 ("1 不是素数" 是约定), 用对偶命题证了 " n^2 偶 $\Rightarrow n$ 偶", 用欧几里得反证法证了 "素数有无穷多个" (那个 $N+1$ 的构造), 还叙述性认识了算术基本定理。最后, 把这一册的整条线收束起来。

钉死一句话: **证明实战 = 把证明四招用在关于整数的命题上; 而 "素数无穷" 教了你最漂亮的一招——反证法里的矛盾不是等来的, 是 "假设有限" 逼着构造出来的。**

记住: **这一册到此收束——地基四件套 (数/逻辑/集合/函数) + 证明四招 (直接/反证/归纳/构造) 已全部集齐。** 你不再只会 "看别人证明", 你会 "自己写证明" 了。下一册《实数极限与连续》, 你就要用这套脚手架, 去盖第一座真正的大楼。

造物主手记

我曾踩的坑:

我差点把 1 当成素数, 被老谟点醒——"**1 不是素数**" 不是发现, 是为了保住 "唯一分解" 而设的约定。我还混淆了 "反证法" 和 "证对偶命题": 证 " n^2 偶 $\Rightarrow n$ 偶", 我一开始写成 "假设 n^2 偶, 然后.....", 其实直接证逆否 " n 奇 $\Rightarrow n^2$ 奇" 更干净。最让我卡壳的是 "素数无穷"——我盯着 $p_1 \dots p_k$ 怎么也想不到要 "乘起来加 1"; 被点破才发现, 那个 $N+1$ 是 "假设只有有限个" 逼出来的, 不是凭空想的。

顿悟时刻:

真正开窍是听懂那句 "**反证法的矛盾不是等来的, 是造出来的**"。欧几里得没等矛盾自己出现, 他动手造了一个 $N = p_1 \dots p_k + 1$, 硬是把 "列表外" 的素数逼了出来。原来证明不只是验证, 它本身就是发现新东西的方式——那个 $N+1$, 在假设里并不存在, 是 "反证" 这个动作把它生出来的。我第一次觉得, 证明不是背书, 是创造。

自测题

计算题须动笔完成，附完整解答自核；概念题用于检查理解。

题 A (计算, 易→中) : 欧几里得除法与同余

(a) 求 593 除以 16 的商和余数，写出带余除法式。(b) 求 $3^5 \bmod 11$ (即 243 除以 11 的余数)。(c) 判断 143 是否为素数。

完整解答： (a) $16 \times 37 = 592$, $16 \times 38 = 608 > 593$, 故 $593 = 16 \times 37 + 1$, 商 37、余 1。(b) $243 = 11 \times 22 + 1$ ($11 \times 22 = 242$), 余 1。 $3^5 \bmod 11 = 1$ 。(c) $143 = 11 \times 13$, 有非 1 且非自身的因数，故 **143 是合数** (不是素数)。

题 B (计算, 中→难) : 反证法完整书写——整除命题

用对偶命题完整书写证明："若 $5 \mid n^2$, 则 $5 \mid n$ "。

完整解答： **命题：**若 $5 \mid n^2$, 则 $5 \mid n$ 。 **证明 (对偶命题, 证 " $5 \nmid n \Rightarrow 5 \nmid n^2$ ") :** ① 设 $5 \nmid n$, 则 n 被 5 除的余数 $\in \{1, 2, 3, 4\}$ (非 0)。② 分别算 $n^2 \bmod 5$:

- $n \equiv 1: n^2 \equiv 1$;
- $n \equiv 2: n^2 \equiv 4$;
- $n \equiv 3: n^2 \equiv 9 \equiv 4$;
- $n \equiv 4: n^2 \equiv 16 \equiv 1$ 。四种情形 $n^2 \bmod 5$ 均为 1 或 4, 都不是 0, 故 $5 \nmid n^2$ 。③ 已证 " $5 \nmid n \Rightarrow 5 \nmid n^2$ ", 其逆否 " $5 \mid n^2 \Rightarrow 5 \mid n$ " 成立。■ (注: 此处用 "平方模 5 的余数" 验证, 与题 2 用展开式验证等价。)

题 C (概念, 检查理解) : 数论证明的类型识别

老谟给出三个证明片段, 请指出各用哪一招 (直接/反证/归纳/构造/对偶), 并说明判断依据: (1) "设 $a \mid b$ 且 $b \mid c$, 则 $b = aq$ 、 $c = bq'$, 代得 $c = a(qq')$, 故 $a \mid c$ 。" (2) "假设素数只有 $p_1, \dots, p_k$, 构造 $N = p_1 \cdots p_k + 1$, 逼出列表外素数, 矛盾。" (3) " n 奇 $\Rightarrow n = 2k + 1 \Rightarrow n^2 = 2(2k^2 + 2k) + 1$ 奇, 故 n^2 偶 $\Rightarrow n$ 偶。"

完整解答： (1) **直接证明**——从整除定义 ($\exists q, a = bq$) 翻译成代数代入, 正面推到结论。(2) **反证法 + 构造**——先假设有限 (反证), 再构造 $N + 1$ (构造), 逼出矛盾。(3) **对偶命题的直接证明**——直接证 " n 奇 $\Rightarrow n^2$ 奇" (逆否), 等价于原命题; 不是反证法 (没先假设 " n^2 偶" 为假)。

预告 (终章·全册收束)

这一册, 到此收束。

你从第 01 章那块只会开和关的芯片出发，一路走到这里：**数**（二进制、补码，以及第一次尝到“凭什么”）、**逻辑**（布尔代数，怎么判断）、**集合**（把东西圈起来）、**函数**（从一个到另一个）、**证明四招**（直接、反证、归纳、构造）、以及本章的**证明实战**（整除、素数无穷、算术基本定理）。**地基四件套 + 证明四招，全部集齐。**

这不是结束，而是真正的开始。下一册《实数极限与连续》，你就要用这套地基和脚手架，去盖第一座真正的大楼——**极限**。那里你遇到的每一个“极限存在”或“极限不存在”，都要靠你手上这两套东西来撑腰：证明里会用到反证法（很多“不存在”的极限结论）、数学归纳法（对一切正整数成立的不等式）、对偶命题（正反等价的论证）。**那些你在这两章亲手写过的证明，会成为你读下一册每一行严格化走廊时，心里稳稳的底气。**

《逻辑与数》· 计算答案速查页

本页汇总册内各章「计算训练场」与「自测题」的关键终值，供自测后核对。**使用纪律**：先动笔算完再对答案，不要边算边翻。若你算出的数与本页不符，先回看你的步骤，再回看章节解答——两种都符合才是真懂。标注：✓ 表示该值已逐项核算一致。

第 01 章 · 数——开关怎么表示量

计算训练场

- 题 1 (批量进制换算 + 补码加减)：(a) $13=1101✓$ 、 $42=101010✓$ 、 $100=1100100✓$ 、 $255=11111111✓$ ；(b) $13+42$ 的 8 位补码= $00110111=55✓$ (55 的补码一致)；(c) 8 位无符号与补码装 $100+200=300$ ，均被 **mod 256** 截断成 $44✓$
- 题 2 (取反加一 + 还原)：(a) -7 补码= $11111001✓$ ；(b) 再取反加一回 $00000111✓$ ；(c) $249=11111001✓$ 与 (a) 一致；(d) 由定理 2 证 $\sim x+1 \equiv 2^8-x✓$
- 题 3 (4 位加法器)： $1011+0110$ 逐位= $0001✓$ ； $11+6=17 \bmod 16=1✓$ ；补码读 $1011=-5$ 、 $0110=6$ ， $-5+6=1✓$ (同一电路两种语义)

自测题

- 题 A：(a) $29=11101$ 、 $64=1000000$ 、 $131=10000011$ ；(b) $29=00011101$ 、 $-29=11100011$ 、 $-1=11111111$ 、 $-128=10000000✓$ ；(c) $(-29)+(-1)=11100010=-30✓$ ($2^8-30=226$ 一致)
- 题 B：补码 11101011 还原= 21 ，故 $x=-21✓$ ； $2^8-21=235=11101011✓$
- 题 C：概念 (定理 2 思路： $x+\sim x=2^n-1 \Rightarrow \sim x+1=2^n-x$)

第 02 章 · 逻辑——开关怎么做判断

计算训练场

- 题 1 (化简 + 真值表)： $F=(A \wedge B) \vee (A \wedge \neg B) \vee (\neg A \wedge B)$ 化简= $A \vee B✓$ (分配律+互补律+同一律，4 行真值表全同)
- 题 2 (三变量真值表)： $G=\neg(A \wedge B) \vee C$ 与 $H=(\neg A \vee \neg B) \vee C$ ，8 行真值表**逐行全同**✓ (背后是德摩根律)

- 题 3 (NAND 表达 + 验证) : $f=(A \wedge B) \vee \neg C$ 改写成 $(P \uparrow P) \uparrow (Q \uparrow Q)$, $P=(A \uparrow B) \uparrow (A \uparrow B)$ 、 $Q=C \uparrow C$;
 $A=1, B=1, C=0$ 时原函数与 NAND 式均=1✓

自测题

- 题 A: $F=(A \vee B) \wedge (A \vee \neg B)$ 化简=**A**✓ (4 行真值表全等)
- 题 B: $g=\neg(A \vee B)$ 的纯 NAND 式 = $Q \uparrow Q$, $Q=(A \uparrow A) \uparrow (B \uparrow B)$; $A=1, B=0$ 时 $\neg(1 \vee 0)=\mathbf{0}$ 与 NAND 式=0✓
- 题 C: 概念 (真值表何时是证明/验证、构造 vs 验证、AND/OR 不完备而 NAND 完备)

第 03 章 · 集合——把东西圈起来

计算训练场

- 题 1 (幂集与笛卡尔积大小) : (a) $P(A)$ 共 **8** 个元素 ($A=\{a,b,c\}$) ; (b)
 $|P(A)|=\mathbf{8}$ ✓、 $|P(P(\emptyset))|=\mathbf{2}$ ✓、 $|A \times B|=\mathbf{6}$ ✓、 $|P(A \times B)|=\mathbf{64}$ ✓; (c) $P(\emptyset)=\{\emptyset\} \neq \emptyset$, $P(P(\emptyset))=\{\emptyset, \{\emptyset\}\}$
- 题 2 (双包含证明) : $A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$, 双向包含全成立✓; **$A=\emptyset$ 时空真, 分配律仍成立**✓
- 题 3 (幂集+笛卡尔积综合) : (a) $|P(X)|=\mathbf{16}$ ✓ (2^4) ; (b) 恰含 2 元素子集 $C(4,2)=\mathbf{6}$ 个: $\{1,2\}, \{1,3\}, \{1,4\}, \{2,3\}, \{2,4\}, \{3,4\}$ ✓; (c) $|P(X \times Y)|=\mathbf{256}$ ✓ (先算 $|X \times Y|=8$ 再 2^8) , 与 $P(X) \times P(Y)=\mathbf{64}$ (16×4) **区分**✓

自测题

- 题 A: (a) $|P(A)|=\mathbf{4}$ ✓; (b) $|A \times B|=\mathbf{6}$ 、 $|P(B)|=\mathbf{8}$ ✓; (c) 判真伪: $\emptyset \in P(A)$ 真、 $\{x\} \in P(A)$ 真、 $x \in P(A)$ 假、 $\{x\} \subseteq A$ 真
- 题 B: 双包含证 $(A \cap B)^c = A^c \cup B^c$, 两方向成立✓ (用德摩根逻辑版 " $x \notin A$ 或 $x \notin B$ ")
- 题 C: 概念 (双包含为何适用于无穷集合、 $\emptyset \subseteq$ 一切的空真)

第 04 章 · 函数——从一个到另一个

计算训练场

- 题 1 (判定单射/满射/双射) : (a) $f=3x-2$ **双射**, $f^{-1}=(x+2)/3$ ✓; (b) $g=x^2+1$ 非单射非满射、无反函数✓; (c) $h=e^x$ 对 $\mathbb{R}$ 单射非满射; 若陪域改 $(0, \infty)$ 则双射, 反函数 **$\ln$** ✓

- 题 2 (复合顺序 + 结合律) : $f=x+1$ 、 $g=2x$ 、 $h=x^2$: (a) $f \circ g=2x+1$ 、 $g \circ f=2x+2$ (不同) ✓; (b) $(f \circ g) \circ h$ 与 $f \circ (g \circ h)$ 都 $=2x^2+1$ ✓; (c) $x=3$ 时 $f \circ (g \circ h)=19$ ✓
- 题 3 (分段函数) : $f(x)=2x(x \geq 0)$ 、 $x-1(x < 0)$ 是单射; 两支像 $[0, \infty)$ 与 $(-\infty, -1)$ 合并 $=\mathbb{R} \setminus [-1, 0)$, **非满射** (漏 $[-1, 0)$) , **无反函数** ✓

自测题

- 题 A: (a) $f=x^3$ **双射**, $f^{-1}=\sqrt[3]{y}$ ✓; (b) $g=x^2-4$ 非单射非满射、非双射 ✓; (c) $h=2^x$ 对 $\mathbb{R}$ **非满射** (像 $(0, \infty)$) , 若陪域改 $(0, \infty)$ 则双射、反函数 $\log_2$ ✓
- 题 B: $f=x-1$ 、 $g=3x$ 、 $h=x^2$: $f \circ g=3x-1$ 、 $g \circ f=3x-3$ (不同) ✓; 结合律两边都 $=3x^2-1$ ✓, $x=2$ 时 $=11$ ✓
- 题 C: 概念 (x^2 为何无反函数、函数相等定义让结合律可证、直接 vs 构造)

第 05 章 · 证明方法入门——"显然"在数学里不算数

计算训练场 (完整书写证明)

- 题 1 (数学归纳法) : $1+2+\dots+n=n(n+1)/2$ ✓ (基础步 $n=1$: $1=1$; 递推步提公因式 $(n+1)$ 通分)
- 题 2 (归纳法·平方和) : $1^2+2^2+\dots+n^2=n(n+1)(2n+1)/6$ ✓ (关键代数: 通分 + 分解 $2n^2+7n+6=(n+2)(2n+3)$)
- 题 3 (反证法) : $\sqrt{2}$ 是**无理数** ✓ (假设有理 $\rightarrow a^2=2b^2 \rightarrow$ 引理 " a^2 偶 $\Rightarrow a$ 偶" $\rightarrow a, b$ 均偶 $\rightarrow$ 与互质矛盾)

严格化走廊示范 (可复写)

- 定理 2 (归纳) : $1+2+\dots+n = n(n+1)/2$ 对一切正整数成立 ✓
- 定理 1 (反证) : $\sqrt{2}$ 是无理数 ✓
- 两个反例 (归纳法"两步缺一不可") : 命题 $P(n)$: " $n=1$ "——基础步 $1=1$ 过、**递推步失败** ($n=1$ 推不出 $2=1$) ; 命题 $Q(n)$: " $n=n+1$ "——递推步空洞成立 (两边加 1) 、**基础步 $1=2$ 假** ✓

自测题

- 题 A (立方和) : $1^3+2^3+\dots+n^3=[n(n+1)/2]^2$ ✓
- 题 B (反证 $\sqrt{3}$) : $\sqrt{3}$ 是**无理数** ✓ (引理 " $3|a^2 \Rightarrow 3|a$ ", 余 $1/2$ 情形平方后 $\equiv 1 \pmod{3}$, a, b 均被 3 整除与互质矛盾)
- 题 C: 概念 (识别直接/归纳/构造)

第 06 章 · 证明实战——拿什么证明“素数有无穷多个”

计算训练场

- 题 1 (欧几里得除法 + 素性判定) : (a) $437 = 12 \times 36 + 5$ (商 36 余 5) ; (b) $12 \nmid 437$ 、 $437 \bmod 12 = 5$; (c) $1024 \bmod 7 = 2$; (d) **97 是素数** ($\sqrt{97} \approx 9.8$, 查 2/3/5/7 余 1/1/2/6 均非 0)
- 题 2 (对偶命题) : " $3 \mid n^2 \Rightarrow 3 \mid n$ " (证逆否" $3 \nmid n \Rightarrow 3 \nmid n^2$ ", 分余 1、余 2 两情形, n^2 均 $\equiv 1 \pmod 3$)
- 题 3 (反证 $\sqrt{6}$) : $\sqrt{6}$ 是**无理数** (引理" $2 \mid n^2 \Rightarrow 2 \mid n$ "; 到 $2m^2 = 3b^2$ 处用 2、3 互质把 2 分离出 $\rightarrow 2 \mid b^2 \rightarrow 2 \mid b \rightarrow a、b$ 均偶与互质矛盾)

自测题

- 题 A: (a) $593 = 16 \times 37 + 1$ (商 37 余 1) ; (b) $243 \bmod 11 = 1$; (c) $143 = 11 \times 13$, 是**合数**
- 题 B (对偶) : " $5 \mid n^2 \Rightarrow 5 \mid n$ " ($n \equiv 1, 2, 3, 4$ 时 $n^2 \bmod 5 = 1, 4, 4, 1$ 均非 0)
- 题 C: 概念 (识别直接/反证+构造/对偶)

使用说明

1. **难度标尺**: 每题分 (易/中/难) 见各章计算训练场题干; 本页只给终值, 不重复解答过程。
2. **证明为主、计算为辅**: 第 05、06 章重点是"完整书写证明", 终值多为"定理成立/该数性质"; 自测后除了对数字, 更要**对照章节里的"可复写模板"逐行核对自己的证明格式** (是否写全基础步/递推步、是否标注依据、是否明确指出矛盾点)。
3. **证明类型识别 (本册特色)**: 本册每一章的严格化走廊开头都有「证明类型标注」——直接/反证/归纳/构造/对偶。对答案时先问"这题用的哪一招", 再看数字: 看懂招式, 比看懂结论更进一步。
4. **模板卡复用**: 第 03 章那张「双包含」模板卡、第 05 章四种证明的书写格式模板, 是后面 6 册严格化走廊的通用脚手架。本页引用的每个证明终值, 都能回到对应模板卡上逐格核对。
5. 如发现本页某值与章节解答不一致, 先回看章节——本页终值已逐项核算, 若确认是章节解答有误, 欢迎反馈。